

我要在本地操作aws s3，s3的bucket要怎麼設定

已思考 9s >

下面我用「你要從本地電腦 (Windows/macOS/Linux) 用 AWS CLI/SDK 操作真正的 S3」來講 bucket 要怎麼設定；最後再補一小段「如果你其實是想在本地模擬 S3 (LocalStack) 」。

A. 先把「本地可以存取 S3」的權限準備好 (IAM)

1. 建立 IAM 使用者 (或用 IAM Identity Center/SSO)

- 新手最直觀：在 AWS Console → IAM → Users → Create user，勾選 *Programmatic access* (會拿到 Access key) 。
- 如果你們學校/公司用 SSO，就用 Identity Center 登入再給 CLI 用 (同理需要 S3 權限) 。 [AWS 文檔](#)

2. 給這個使用者一個最小可用的 S3 Policy

你可以先用 AWS Managed Policy：

- `AmazonS3FullAccess` (簡單但權限大)

或自己建一個「只給特定 bucket」的 policy，例如 (把 `your-bucket-name` 換成你的)：

json

複製程式碼

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListBucket",
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::your-bucket-name"
    },
    {
      "Sid": "ObjectRW",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
```

```
    "s3:PutObject",
    "s3:DeleteObject"
  ],
  "Resource": "arn:aws:s3:::your-bucket-name/*"
}
]
```

這種「bucket 層 + object 層」分開授權的概念，是 S3 權限的標準方式。 [AWS 文檔](#)

B. 建 bucket (Console 或 CLI 都行)

方法 1 : AWS Console 建

S3 → Create bucket :

1. **Bucket name** 全世界唯一
2. **Region** 選離你最近的 (台灣常用 `ap-northeast-1` 東京或 `ap-southeast-1` 新加坡)
3. **Block Public Access**
 - 若你只是自己/專案用：保持全部勾選 (封鎖公開)，最安全
 - 之後如果要放靜態網站或公開讀取，再來調整。

Block Public Access 會直接覆蓋 bucket policy/ACL，確保不會被意外公開。

[AWS 文檔 +1](#)
4. 其他建議設定 (看需求開)
 - **Versioning**：避免誤刪/可回溯
 - **Default encryption (SSE-S3 / SSE-KMS)**：資料自動加密
 - **Lifecycle rule**：舊檔自動轉 IA/Glacier 或刪除 (省錢)

方法 2 : AWS CLI 建

```
bash
```

 複製程式碼

```
aws s3 mb s3://your-bucket-name --region ap-northeast-1
```

`mb` 會建立 bucket，name 必須全球唯一。 [clarence.tw +1](#)

C. Bucket Policy (通常不用改，除非你要跨帳號/公開)

如果你 bucket 是私有、只讓你 IAM user 存取：

只要 IAM policy 有授權，bucket policy 可以先空著。

當你需要「跨帳號存取 / 某些角色可讀寫」時才加 bucket policy。AWS 官方有很多範本可照抄改 ARN。

[AWS 文檔 +1](#)

要公開讀取（例如靜態網站）才需要：

1. “關掉 Block Public Access 某些選項”
2. “寫 public read bucket policy

否則 policy 會被 Block Public Access 擋掉。

[AWS 文檔 +1](#)

D. 本地電腦設定 AWS CLI

1. 安裝 AWS CLI v2

- Windows: 官方 MSI
- macOS: `brew install awscli`

2. 設定 credentials

```
bash
```

[複製程式碼](#)

```
aws configure
```

依序輸入：

- AWS Access Key ID
- AWS Secret Access Key
- Default region name (如 `ap-northeast-1`)
- Default output format (`json`)

會寫到 `~/.aws/credentials` 和 `~/.aws/config`。

[AWS 文檔 +1](#)

如果你有多個帳號/專案，用 profile：

```
bash
```

[複製程式碼](#)

```
aws configure --profile myproj  
aws s3 ls --profile myproj
```

3. (可選) 用環境變數覆蓋

很適合寫腳本或 CI：

```
bash
```

[複製程式碼](#)

```
export AWS_ACCESS_KEY_ID=...  
export AWS_SECRET_ACCESS_KEY=...
```

```
export AWS_DEFAULT_REGION=ap-northeast-1
```

CLI 會優先用 env vars 。 [AWS 文檔 +1](#)

E. 測試你從本地可以操作 S3

bash

 複製程式碼

```
aws s3 ls
aws s3 ls s3://your-bucket-name
echo "hello" > test.txt
aws s3 cp test.txt s3://your-bucket-name/
aws s3 rm s3://your-bucket-name/test.txt
```

只要這些能跑通，你的 bucket 設定 + 本地權限就 OK 了。 [clarence.tw](#)

F. 如果你其實想「在本地模擬 S3」

有些人說「本地操作 S3」是指不連 AWS、在本地跑一個 S3 服務（方便測試）。這時候通常用 **LocalStack / MinIO**，然後 CLI 指到自訂 endpoint，例如：

bash

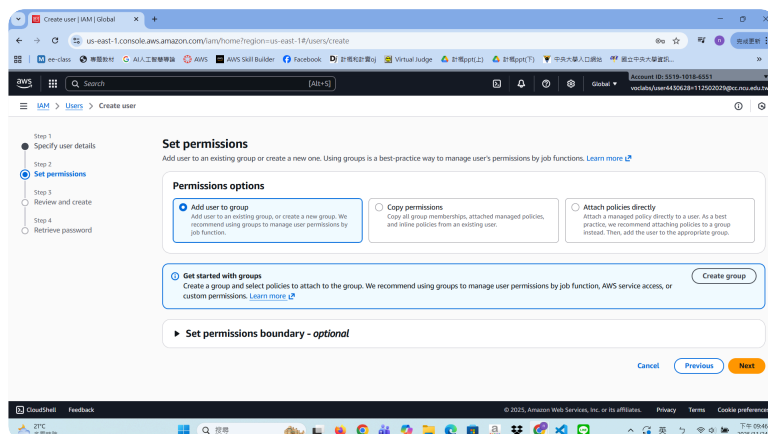
 複製程式碼

```
aws --endpoint-url http://localhost:4566 s3 ls
```

CLI 支援自訂 endpoint 設定。 [AWS 文檔 +1](#)

如果你願意告訴我你要的使用情境（例如：只是自己本機上傳下載？要給隊友共用？要公開當網站？），我可以直接幫你把 **IAM policy + bucket policy** 寫成你專案專用的最小權限版本。

     ...  資料來源



已思考 30s >

你這張圖是在 **Create user → Step 2: Set permissions**。你現在要做的是：先幫這個 user 掛上可操作 S3 的權限，然後之後拿到 Access Key 給本地 AWS CLI 用。

我直接照你畫面一步一步說：

1. 在這一頁先建立一個群組（推薦做法）

你目前選的是 **Add user to group**（左邊那個），OK。

1. 右邊點 **Create group**
2. **Group name** 你可以取：`s3-local-group`（隨便，但要好認）
3. 下方會出現 policies 清單，搜尋 **S3**
你有兩種選擇：

選擇 A：快速可用（新手）

勾 **AmazonS3FullAccess**

→ 你本地就能對所有 bucket 做任何事（簡單但權限大）。

選擇 B：只給特定 bucket（比較安全，作業/專題常用）

先不要勾任何 managed policy

→ 建完 group 之後再加「自訂 policy」（下面我也給你做法）。

4. 右下 **Create group** 完成
5. 回到同頁，你會看到剛剛建的 group，勾起來
6. 右下 **Next**

2. Step 3 Review and create → 建立 user

一路 Next → Create user。

3. 建完 user 後：做「Access Key」給本地用

到 IAM → Users → 點你的 user → Security credentials 分頁

1. 找到 **Access keys** 區塊
2. 點 **Create access key**
3. Use case 選 **Command Line Interface (CLI)**
4. Next → Create access key
5. 把 **Access key ID / Secret access key** 存起來 (Secret 只會顯示一次)

4. 本地電腦設定 AWS CLI

在你電腦終端機輸入：

```
bash
```

 複製程式碼

```
aws configure
```

依序貼上：

- Access key ID
- Secret access key
- Default region (例如 `ap-northeast-1`)
- output format (json)

測試：

```
bash
```

 複製程式碼

```
aws s3 ls
```

```
aws s3 ls s3://你的bucket名
```

5. (可選) 如果你要「只讓他用某個 bucket」

假設 bucket 叫 `my-class-bucket`

5-1 建 policy

IAM → Policies → Create policy → JSON 貼上：

json

複製程式碼

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListBucket",
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3::my-class-bucket"
    },
    {
      "Sid": "ObjectRW",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject"
      ],
      "Resource": "arn:aws:s3::my-class-bucket/*"
    }
  ]
}
```

→ Review policy → 存成 `my-class-bucket-rw`

5-2 把 policy 掛到 group

IAM → User groups → 點 `s3-local-group`

→ Permissions → Add permissions → Attach policies

→ 勾 `my-class-bucket-rw` → Add permissions

這樣本地只能碰這個 bucket。

小提醒（超常踩雷）

- S3 bucket 那邊 **Block Public Access** 不要關，除非你真的要公開當網站。
- Access key 千萬不要上傳到 GitHub 或報告裡。

你如果跟我說：

1. 你的 bucket 名字
2. 你要不要限制只用這個 bucket

我可以幫你上面的 policy 直接換成你的版本，讓你複製貼上就好。

📄 🍷 🗑️ ⬆️ ↺ ...

我已經建立好 bucket，不能直接操作嗎，一定要用 IAM user 嗎

可以直接操作「bucket」嗎？要看你在哪裡操作、用什麼身分操作。

S3 本身不會讓「任何人」直接連上去操作，一定要有 AWS 身分/憑證 (credentials) 才能碰到它。差別只是：不一定要用 IAM user，但一定要用某種 IAM 身分。

你可以不用 IAM user 的情況

1. 只在 AWS Console/CloudShell 上操作

- 你現在登入 AWS Console 的那個身分 (可能是 root / IAM role / SSO)
本來就已經有權限了，所以在 Console 的 S3 UI 就能直接上傳下載、建資料夾等。
- AWS CloudShell 也是同理：開 CloudShell 後下 `aws s3 ls` 就能用，不需要再建 IAM user。

✅ 結論：在 AWS 裡面操作，不一定要 IAM user。

但「本地電腦要操作 S3」一定需要憑證

你在自己電腦跑：

```
bash
```

📄 複製程式碼

```
aws s3 ls
```

```
aws s3 cp ...
```

這時候 AWS CLI 必須知道「你是誰」，所以要給它 credentials。

你可以選的 credentials 來源有：

A) IAM user 的 Access Key (最直覺)

- 你剛剛做的那套
- 優點：簡單、馬上可用
- 缺點：長期金鑰要小心保管

B) IAM Role + AssumeRole (常用在公司/專題)

- 先用一組「登入用」的身分 (可能還是 user 或 SSO)
- 再去 assume 一個 role 拿到 臨時憑證
- 優點：比長期 key 安全

C) AWS SSO / IAM Identity Center 登入 (學校/公司常用)

- 本地用：

```
bash
```

[複製程式碼](#)

```
aws configure sso
```

```
aws s3 ls
```

- 不會產生長期 access key，也是臨時憑證

D) EC2/Lambda 上的 Instance Role (不是你本地)

- 如果你是把程式跑在 EC2 上，那台機器掛 role 就能直接存取 S3
- 但這不適用「你自己的筆電」情境

✅ 結論：本地一定要某種 credentials，但不一定要 IAM user。

最重要的提醒

- 不要用 root 的 access key 在本地操作 (官方強烈不建議)
- 最佳做法：
 - 學生/個人：IAM user 最快
 - 團隊/正式專案：SSO 或 AssumeRole

如果你跟我說你現在的帳號類型是：

1. AWS Academy/voclabs ?
2. 自己的 AWS 帳號 ?
3. 學校/公司 SSO ?

我就能直接告訴你「你該用 A/B/C 哪一種」，然後給你對應的最短設定步驟。



```
User group was not created.  
User arn:aws:sts::551910186551:assumed-  
role/voclabs/user4430628=112502029@cc.ncu.edu.tw  
is not authorized to perform: iam:CreateGroup on  
resource: arn:aws:iam::551910186551:group/s3-local-  
group because no identity-based policy allows the  
iam:CreateGroup action
```

已思考 19s >

這個錯誤的意思很單純：你現在登入的身分是 **voclabs** 的 **assumed-role**，而這個 Lab 角色 **沒有 iam:CreateGroup 權限**，所以你不能在 IAM 裡自己建 User Group。

(AWS Academy / voclabs 常常會把 IAM 管理權限鎖起來，避免學生亂開權限。)

但你還是有幾條路可以走：

方案 1：不要建 group，改成「直接掛 policy 到 user」

回到你剛剛那頁 (Create user → Set permissions)：

1. 把選項改成右邊的

Attach policies directly

2. 下面搜尋 **S3**

先勾 **AmazonS3FullAccess** (最省事)

3. Next → Create user

這樣就不需要 CreateGroup 權限。

但前提是你的 Lab 角色至少要有 **iam:CreateUser** / **iam:AttachUserPolicy** 權限。

如果下一步又跳「not authorized to perform iam:CreateUser」，那代表 Lab 連 user 都不讓你建。

方案 2：用「Lab 已經給你的權限」直接操作 S3

你的 voclabs 角色通常已經有 **S3 權限**，只是不能管 IAM 而已。

你可以改用：

 **AWS Console**

S3 介面直接上傳 / 下載 / 建資料夾。

✅ CloudShell

Console 右上開 CloudShell (或左下 CloudShell)，直接打：

```
bash
```

 複製程式碼

```
aws s3 ls  
aws s3 ls s3://你的bucket  
aws s3 cp test.txt s3://你的bucket/
```

CloudShell 用的是你現在這個 voclabs role，所以不需要 access key，也不用 IAM user。

方案 3：如果 Lab 完全不讓你建 access key

那你就沒辦法在自己電腦用 AWS CLI 連到真 S3，因為本地一定要 credentials。
這時只能：

1. 用 CloudShell / Console 做所有 S3 操作
2. 或 請老師/助教/管理員幫你開
 - 建一個 IAM user + access key
 - 或幫你開 SSO/AssumeRole 給本地用

你現在可以先快速驗證哪個方案可行

你先去 IAM 看看你「能不能建 user」：

- IAM → Users → Create user
如果 **Create user 也被擋**
→ 代表這個 Lab 不給你任何 IAM 管理，直接跳方案 2/3。

如果你下一個錯誤訊息是什麼（例如 CreateUser / AttachUserPolicy 也被擋），貼給我，我就能直接判斷你這個 voclabs 到底開到哪裡，然後告訴你最佳解法。

     ...

User was not created.
User arn:aws:sts::551910186551:assumed-

```
role/voclabs/user4430628=112502029@cc.ncu.edu.tw
is not authorized to perform: iam:CreateUser on
resource:
arn:aws:iam::551910186551:user/testuser1124 because
no identity-based policy allows the iam:CreateUser
action
```

已思考 26s >

看這個錯誤就確定了：你的 **voclabs / AWS Academy** 角色被鎖住 **IAM 管理權限**，所以你不能 **CreateGroup**、也不能 **CreateUser**，當然也就不能拿 **Access Key**。

這不是你操作錯，是 Lab 設計本來就這樣。

所以結論是：

- ✅ 你可以用現在這個 **voclabs** 身分在 **AWS** 裡「直接操作 S3」
- ❌ 但你沒辦法在自己電腦用 **AWS CLI/SDK** 連到「真 S3」，因為本地一定要 **credentials**，而 Lab 不給你建立

下面給你可行的替代作法（你可以照需求選）：

方案 1：用 AWS Console / CloudShell 操作（最直接、Lab 允許）

Console

S3 → 你的 bucket → Upload/Download/建立資料夾...

這個你現在就能做。

CloudShell

在 Console 開 CloudShell，直接：

```
bash
```

 複製程式碼

```
aws s3 ls
aws s3 ls s3://你的bucket
aws s3 cp localfile.txt s3://你的bucket/
aws s3 cp s3://你的bucket/file.txt .
```

CloudShell 會用你現在的 **voclabs role**，所以不用 **key**。

方案 2：想從「本地」碰 S3？用 presigned URL 繞過憑證

你雖然不能在本地用 aws cli，但可以：

1. 在 CloudShell 產生可臨時使用的 URL
2. 用瀏覽器/curl 在你電腦上上傳或下載

2-1 下載 presigned URL (CloudShell)

```
bash
```

[複製程式碼](#)

```
aws s3 presign s3://你的bucket/要下載的檔案 --expires-in 3600
```

會吐出一條網址（有效 1 小時）。

你在本地貼到瀏覽器就能下載。

2-2 上傳 presigned URL (CloudShell 用 Python 產生 PUT URL)

CloudShell 裡先開個檔案 `gen_put_url.py`：

```
python
```

[複製程式碼](#)

```
import boto3, os
from botocore.config import Config

bucket = "你的bucket"
key = "upload_from_local.txt" # 你希望上傳後的檔名
expires = 3600

s3 = boto3.client("s3", config=Config(signature_version="s3v4"))
url = s3.generate_presigned_url(
    ClientMethod="put_object",
    Params={"Bucket": bucket, "Key": key},
    ExpiresIn=expires,
    HttpMethod="PUT"
)
print(url)
```

執行：

```
bash
```

[複製程式碼](#)

```
python3 gen_put_url.py
```

它會印出一條 PUT 用的 presigned URL。

然後你在**本地電腦**用 curl 上傳：

```
bash
```

[複製程式碼](#)

```
curl -X PUT -T 你本地的檔案.txt "剛剛那條URL"
```

這樣就等於本地上傳到真 S3 了，不需要 access key。

方案 3：如果作業「一定要本地 AWS CLI/SDK 直連」

那就只能請管理員開權限，因為 Lab 角色做不到：

- 請老師/助教幫你：
 - 建一個 IAM user 給你 access key
 - 或
 - 開 AWS SSO/Identity Center 讓你本地 `aws configure sso`
- 或你換用自己的 AWS 帳號（你自己有 root/iam 管理權限）

方案 4：只是要「本地測試 S3 API」不用連真 AWS

用本地 S3 模擬器：

- LocalStack
- MinIO

你的程式 endpoint 指到 localhost 就能測試，上線再換回 AWS。

你現在是做哪一種需求？

- 如果只是要交作業「成功上傳下載」→ 方案 1 或 2 就夠。
- 如果你是要跑 Python/Node 專案在本地直連 S3 → 就得走方案 3。

跟我說你作業要求哪種，我就幫你選最省事的路，順便把你的 bucket 名字代進範例指令。

     ...

[github pull 最新的檔案](#)

可以，用 `git pull` 就是把 GitHub 上最新的 commit 抓下來並合併到你本地。下面給你最常用、保險的流程（Windows/macOS/Linux 都通用）。

1. 先確認你在專案資料夾

終端機 / PowerShell：

```
bash
```

[複製程式碼](#)

```
cd 你的專案路徑
```

2. 看一下目前在哪個分支

```
bash
```

[複製程式碼](#)

```
git branch
```

有 `*` 的那條就是你現在的分支，例如 `main` 或 `master`。

3. 拉最新檔案

如果你在 `main/master`

```
bash
```

[複製程式碼](#)

```
git pull origin main
```

或

```
bash
```

[複製程式碼](#)

```
git pull origin master
```

如果你在別的分支（例如 `dev`）

```
bash
```

[複製程式碼](#)

```
git pull origin dev
```

`origin` 是遠端 repo 的預設名字

`main/dev` 是遠端分支名

4. 更保險的做法 (避免亂合併)

如果你怕 pull 直接 merge 造成衝突，可以分兩步：

```
bash

git fetch origin
git merge origin/main
```

 複製程式碼

(分支換成你的)

5. 如果你本地有改檔案，pull 可能失敗

情況 A：你改了但還沒 commit

先存起來：

```
bash

git stash
git pull origin main
git stash pop
```

 複製程式碼

情況 B：pull 時出現 conflict

你會看到類似：

```
pgsql

CONFLICT (content): Merge conflict in xxx
```

 複製程式碼

做法：

1. 打開衝突檔案，把 <<<<<<< ===== >>>>>>> 中間內容整理成你要的樣子
2. 然後：

```
bash

git add .
git commit -m "resolve conflict"
```

 複製程式碼

6. 一次把所有遠端分支更新到本地 (可選)

 複製程式碼

bash

 複製程式碼

```
git pull --all
```

7. 你只是要「抓最新但不要自動合併」

用 fetch :

bash

 複製程式碼

```
git fetch origin
```

然後你想合併哪個分支再 merge 。

如果你願意貼：

1. `git remote -v` 的結果
2. `git branch` 的結果

我可以直接告訴你你該打哪一行 (main 還是 master 或 dev) ，避免你拉錯分支。

     ...

```
PS C:\Users\Neo Yeh\Desktop\雲端實務\FinalProject>
git pull origin chenpoyo0219
remote: Enumerating objects: 21, done.
remote: Counting objects: 100% (21/21), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 21 (delta 1), reused 21 (delta 1), pack-
reused 0 (from 0)
Unpacking objects: 100% (21/21), 5.73 KiB | 254.00
KiB/s, done.
From https://github.com/liu-chun-
wu/AWS_final_project
* branch      chenpoyo0219 -> FETCH_HEAD
* [new branch] chenpoyo0219 ->
origin/chenpoyo0219
fatal: refusing to merge unrelated histories
```

```
PS C:\Users\Neo Yeh\Desktop\雲端實務\FinalProject>
甚麼問題
```

這個錯誤 `fatal: refusing to merge unrelated histories` 的意思是：

你本地這個 repo 的 Git 歷史，和遠端 `origin/chenpoyo0219` 這個分支 完全沒有共同祖先（不同的起點），
Git 預設不允許把「兩個無關的專案」硬合在一起，所以拒絕 merge。

常見原因（你這種課堂/專題超常見）：

1. 你本地資料夾是你自己 `git init` 新開的，或從別處拷貝來的，不是從這個 GitHub repo clone 的。
2. 你本地原本連到另一個 repo，後來才改 remote。
3. 遠端那個分支是用「不同專案」拆出來的（不同 root commit）。

下面給你 三種處理方式，看你想要哪種結果：

✅ 方案 A：我就是要把遠端內容「合進我本地現有的檔案」

那就允許無關歷史合併：

powershell

 複製程式碼

```
git pull origin chenpoyo0219 --allow-unrelated-histories
```

如果出現 conflict，就照 Git 提示去解衝突再 commit。

這會把「你本地的歷史」和「遠端的歷史」接在一起，
以後就算是同一個專案了。

✅ 方案 B：我本地沒什麼重要改動，只想「直接變成遠端最新版」

這是最乾淨、最多人要的情境（等於你本地用遠端覆蓋）：

powershell

 複製程式碼

```
git fetch origin
git reset --hard origin/chenpoyo0219
```

⚠ 注意：`reset --hard` 會把你本地目前的檔案/commit 全部改成遠端那個分支的樣子，
本地未提交的改動會消失。
如果你怕不見，先備份或用 `git stash`。

✅ 方案 C：你其實應該重新 clone (最安全、最不會亂)

如果你本地 repo 來源很亂，最穩的做法是：

1. 先把你現在資料夾改名備份 (例如 `FinalProject_backup`)
2. 重新 clone 正確 repo：

powershell

複製程式碼

```
cd C:\Users\Neo Yeh\Desktop\雲端實務
git clone https://github.com/liu-chun-wu/AWS_final_project.git FinalProject
cd FinalProject
git checkout chenpoyo0219
```

小補充：為什麼你會遇到？

你的輸出有這行：

arduino

複製程式碼

```
* [new branch] chenpoyo0219 -> origin/chenpoyo0219
```

代表你本地以前 **沒有** 這個遠端分支的歷史；

現在抓下來後，Git 發現它跟你本地目前分支完全不同根，所以才報錯。

我建議你選哪個？

- 你本地只是放一份作業、想更新成 GitHub 最新 → 用 方案 B
- 你本地有自己寫的東西要保留，又想把遠端拉進來 → 用 方案 A
- 不確定本地怎麼來的/怕弄亂 → 用 方案 C

你可以直接照你想要的方案貼指令跑；

如果你跑完有 `conflict` 或新的錯誤訊息，再貼我，我幫你對症解。

📄 🍌 🗨 📶 🔄 ...

我要把本地的程式部屬到aws lambda，lambda要怎麼設定

已思考 13s >

你要把本地程式部署到 AWS Lambda，核心就三件事：

(1) 打包程式 → (2) 建 Lambda function → (3) 設定執行角色/handler/資源與觸發器。

我下面用最常見的「ZIP 上傳」方式一步一步講，最後補 SAM/容器的另一種部署法。

(另外你之前是 voclabs/AWS Academy 帳號：那種常常不能建 IAM user/role，我也會告訴你在 Lab 裡要怎麼選角色。)

0. 先確認你要用的 Runtime (例：Python / Node.js)

Lambda 需要你指定 Runtime，然後 handler 寫法會跟著變。

官方支援的 runtime 例如 Python、Node.js、Java、Go 等。

[AWS 文檔 +1](#)

1. 本地打包 (ZIP)

1-1 Python 專案 (含套件)

假設你的入口檔是 `lambda_function.py`，handler 叫 `lambda_handler`

bash

 複製程式碼

```
# 在你的專案資料夾  
pip install -r requirements.txt -t .  
zip -r function.zip .
```

重點：套件要一起打進 zip 裡，不然 Lambda 上跑會找不到套件。

1-2 Node.js 專案

bash

 複製程式碼

```
npm install  
zip -r function.zip .
```

1-3 只是一支單檔 (沒外部套件)

直接把那個檔案 zip 起來也可以。

2. 在 AWS Console 建 Lambda Function

Lambda → Create function

1. Author from scratch

2. Function name：你要的名字
3. Runtime：選你對應的語言版本
4. Architecture：一般選 x86_64 (預設)

5. Execution role (很重要)：

[AWS 文檔 +1](#)

- 正常 AWS 帳號：選

Create a new role with basic Lambda permissions

(它會給 CloudWatch Logs 權限)

- **AWS Academy/voclabs**：你多半不能建 role

→ 請選 **Use an existing role**，通常叫 **LabRole**

(這是 Lab 允許你用的)

按 Create function。

3. 上傳你的 ZIP 程式

進到你剛建好的 function → **Code** 分頁

右側 **Upload from** → **.zip file** → 上傳 `function.zip`

[AWS 文檔](#)

4. 設定 Handler / Runtime settings

同一頁往下 **Runtime settings** → Edit [AWS 文檔](#)

- Python handler 格式：

`檔名.函式名`

例：`lambda_function.lambda_handler`

- Node.js handler 格式：

`檔名.export的function`

例：`index.handler`

存檔。

5. 設定記憶體、Timeout、環境變數

到 **Configuration** 分頁：

5-1 General configuration

- Memory：128MB 起跳（看你運算量）
- Timeout：預設 3 秒，必要時調大（最多 15 分鐘） AWS 文檔

5-2 Environment variables（如果你有 API key、bucket 名、參數）

Configuration → Environment variables → Edit → Add AWS 文檔 +1

6. 測試你的 Lambda

到 **Test** 分頁

1. Create new test event（選一個範本或自己貼 JSON）
2. Test
3. 看 Execution result / CloudWatch Logs

7.（選用）設定觸發器 Trigger

如果你要「上傳 S3 檔案就自動跑 Lambda」：

Lambda function → **Add trigger** → 選 S3

- Bucket：選你的 bucket
- Event type：PUT（上傳）
- Prefix/Suffix：可選（如只觸發 .jpg）
存檔後，S3 上傳檔案會觸發 Lambda。 AWS 文檔

注意：這時 Lambda 會自動加一條 resource-based policy 允許 S3 呼叫。 AWS 文檔

8. 另一種正式部署法：AWS SAM（推薦做專題/多人協作）

如果你希望「本地一鍵部署 + 版本控管」：

1. 安裝 SAM CLI
2. 在專案有 `template.yaml`
3. 部署：

```
bash
```

 複製程式碼

```
sam build
sam deploy --guided
```

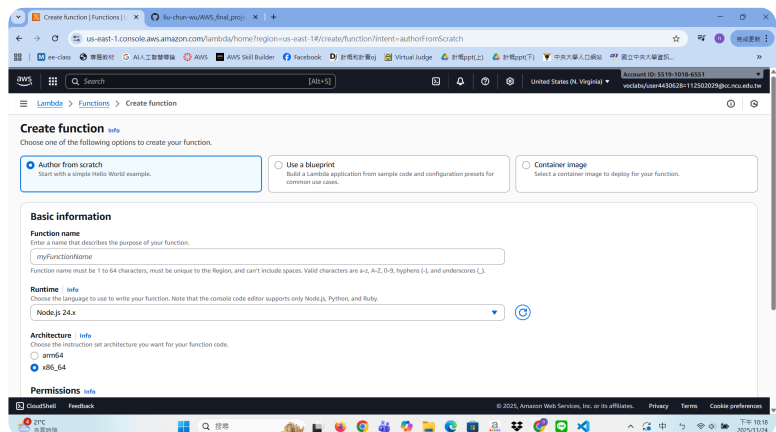
SAM 會幫你打包、上傳到 S3、建立/更新 Lambda。 [AWS 文檔 +2](#)

最常見踩雷清單

1. handler 寫錯 (檔名或 function 名不一致)
2. 依賴套件沒打進 zip (Python/Node 超常見)
3. Execution role 權限不足
 - 例如要讀 S3 卻沒給 `s3:GetObject`
4. voclabs 限制：不能建 IAM role/user → 用 LabRole

如果你把「你的專案語言 + 入口檔名/函式名 (handler)」貼我，
我可以直接幫你寫出**正確 handler 字串**，以及你需要的最小權限 role policy (例如只讀你的某個 bucket)。

📄 🍌 🗨️ ↗️ ↺️ ... 📖 資料來源



怎麼設定

你現在在 **Lambda → Create function (Author from scratch)** 這頁，照你畫面這樣設定就行，我一步一步對應欄位講：

這一頁要填什麼？

1. Author from scratch

你已經選了左邊 **Author from scratch** ☒ 保持這樣。

2. Basic information

Function name

- 這裡填你的函數名字 (英數/底線/減號都可，不能空格)
- 例： `finalproject-api` 或 `aws-final-lambda`

Runtime

- 你現在選 **Node.js 24.x** ☒
只要你本地程式是 Node.js 就保持這個版本。
- 如果你本地是 Python 才改成 Python (但你現在看起來是 Node)

Architecture

- 你選 **x86_64** ☒ (預設即可)
- 除非你知道要用 ARM (省錢/某些套件需求)，不然不用改。

3. Permissions (在畫面再往下滑一點就看得得到)

因為你是 **voclabs / AWS Academy** 帳號，你不能自己建 role，所以一定要這樣選：

- 下拉 **Permissions**
- 選 **Use an existing role**
- 在 role 下拉選單選 **LabRole**
(通常 voclabs 就只給這個可以用)

你不要選「Create a new role...」
那個會被擋 (你前面 IAM 就被鎖了)。

4. 右下角按 Create function

這頁就完成了。

建完 function 後，下一步怎麼做？

進到你的 function 之後：

A. 上傳你的本地程式 ZIP

Code 分頁右側：

1. Upload from → .zip file
2. 上傳你在本地打包好的 `function.zip`

(Node.js 專案打包方式：在專案資料夾 `npm install` 後 `zip -r function.zip .`)

B. 設定 Handler

往下找到 **Runtime settings** → **Edit**

- Node.js handler 格式：

檔名.匯出的函式名

例子：

- 如果入口是 `index.js`，你裡面有
`exports.handler = async (event) => {...}`

那 handler 就填：

```
pgsql
```

 複製程式碼

```
index.handler
```

C. 測試

Test 分頁 → Create test event → Test

看有沒有正常回傳/Log。

如果你是要讓 Lambda 讀寫 S3

因為你用的是 **LabRole**，它不一定有 S3 權限。

你可以先直接測，如果出現 `AccessDenied` 再跟我說錯誤訊息，我教你怎麼在 Lab 的限制下補權限（通常是老師預先給的 policy 或用既有 role）。

你現在先照上面把 **Permissions** 選 **LabRole** → **Create function**。

建好後如果你願意貼：

1. 你的入口檔名（例如 `index.js` / `app.js`）
2. 你 export 的 handler 函式名稱

我就能幫你把手串一次填對，避免跑不起來。

     ...

Additional configurations
要設定嗎

已思考 9s >

通常 **不用**在「Additional configurations」這一步就設定，先用預設建立 function 就好；因為那些都是「進階/特定需求才要開」，而且 **建立後隨時都能回來改**。

我用你現在 (voclabs + Node.js) 最常見情況，幫你快速判斷每個常見選項要不要動：

先別動、留預設的情況 (90% 作業/專題)

只要你是：

- 第一次部署
- 只是要跑一段程式 / API / 讀寫 S3
- 沒有要連 RDS/EC2 私有網路
- 沒有特別要限制併發或加密需求

➡ 直接不管 Additional configurations，滑過去按 Create function。

只有在以下情況才需要設定

1. VPC

什麼時候要設？

- 你 Lambda 需要連 RDS / ElastiCache / EC2 私有子網

➡ 才要設 VPC。

什麼時候不要設？

- 只是要上網、呼叫外部 API、讀 S3

➡ **不要設 VPC** (設了反而可能沒網路、變慢)。

2. Function URL / API Gateway

要不要現在設？

- 如果你馬上要做 HTTP API 才開

- 不然先不開也沒差，之後再加 Trigger 就好。
-

3. Tags

- 只是作業/專題，不一定要
 - 如果老師有要求資源標記再加。
-

4. Encryption / KMS

- 你沒特別敏感資料或合規要求
→ 不用動（預設 AWS 會幫你做基本加密）。
-

5. Concurrency / Async / DLQ

- 這些都是流量大、怕爆掉或要錯誤排隊才會調
→ 作業階段 不用設。
-

6. Ephemeral storage / Monitoring

- 預設通常夠
 - 真的爆 storage 或要更細監控再回來調。
-

你現在的最佳做法

- ✓ 先不設定 Additional configurations → Create function
 - ✓ 建好後上傳 zip、改 handler、Test 跑起來
 - ? 真的需要連資料庫/做 API 再回來加 VPC 或 Trigger
-

如果你願意貼一下你這支 Lambda 要做什麼（例如：讀 S3、做 API、連資料庫、爬外部網站），我可以直接跟你說哪些 Additional configurations 要開、開哪幾個就夠。



```
import json

def lambda_handler(event, context):
    # TODO implement
    return {
        'event': event,
        'body': json.dumps('Hello from Lambda!')
    }
輸出event的格式
```

你這段 Lambda :

py

 複製程式碼

```
import json

def lambda_handler(event, context):
    return {
        'event': event,
        'body': json.dumps('Hello from Lambda!')
    }
```

你問「輸出 event 的格式」= **event** 本身是什麼樣的 JSON 結構。

答案是：**event** 的格式完全取決於你的 Trigger (觸發來源)。不同來源，event 長得不一樣。

下面我給你最常見的幾種 event 範例，跟你怎麼把它漂亮輸出。

1) 最簡單：Console 手動 Test (自訂事件)

你在 Lambda console 按 Test、自己填 JSON 的話
event 就是你填的那份。

例如你 Test event 填：

json

 複製程式碼

```
{
  "name": "Neo",
```



```
"id": 1124
}
```

那 event 就會是 Python dict :

py

 複製程式碼

```
{"name": "Neo", "id": 1124}
```

2) API Gateway (HTTP API / REST API) 事件

HTTP API v2 常見格式 (最常用)

event 會長這樣 (簡化版) :

json

 複製程式碼

```
{
  "version": "2.0",
  "routeKey": "GET /hello",
  "rawPath": "/hello",
  "rawQueryString": "a=1&b=2",
  "headers": { "host": "...", "user-agent": "..." },
  "requestContext": {
    "http": {
      "method": "GET",
      "path": "/hello",
      "sourceIp": "1.2.3.4"
    }
  },
  "queryStringParameters": { "a": "1", "b": "2" },
  "body": null,
  "isBase64Encoded": false
}
```

3) S3 上傳觸發事件

若你設定 S3 PUT 觸發 Lambda , event 會像這樣 :

json

 複製程式碼

```
{
  "Records": [
    {
      "eventSource": "aws:s3",
      "eventName": "ObjectCreated:Put",
      "s3": {
        "bucket": { "name": "your-bucket" },
        "object": { "key": "test.txt", "size": 123 }
      }
    }
  ]
}
```

你通常會取：

py

 複製程式碼

```
bucket = event["Records"][0]["s3"]["bucket"]["name"]
key = event["Records"][0]["s3"]["object"]["key"]
```

4) 直接看 event 長什麼樣 (漂亮印出)

在 Lambda 裡加這行：

py

 複製程式碼

```
import json

def lambda_handler(event, context):
    print("EVENT:\n", json.dumps(event, ensure_ascii=False, indent=2))
    return {
        "event": event,
        "body": json.dumps("Hello from Lambda!")
    }
```

跑 Test 後去 **Monitor → CloudWatch Logs** 就能看到完整 event。

5) 小提醒：如果你是掛 API Gateway

API Gateway 期待的回傳格式是：

py

```
return {  
    "statusCode": 200,  
    "headers": {"Content-Type": "application/json"},  
    "body": json.dumps({"msg": "Hello from Lambda!"})  
}
```

你現在回傳 `{"event": event, "body": ...}` 在 Console Test 會正常看到，
但如果接 API Gateway，可能會被當成 非 proxy 格式。

如果你跟我說你這支 Lambda 的 trigger 是哪一種 (S3? API Gateway? 手動測試?)，我可以給你「那一種 event 的完整真實範本」，以及你該怎麼取值。



Status: Failed
Test Event Name: test1

Response:

```
{  
  "errorMessage": "Unable to import module  
'lambda_function': No module named 'flask'",  
  "errorType": "Runtime.ImportModuleError",  
  "requestId": "",  
  "stackTrace": []  
}
```

The area below shows the last 4 KB of the execution log.

Function Logs:

```
[WARNING] 2025-11-24T14:30:58.543Z  
LAMBDA_WARNING: Unhandled exception. The most  
likely cause is an issue in the function code. However, in  
rare cases, a Lambda runtime update can cause  
unexpected function behavior. For functions using  
managed runtimes, runtime updates can be triggered  
by a function change, or can be applied automatically.  
To determine if the runtime has been updated, check
```

the runtime version in the INIT_START log entry. If this error correlates with a change in the runtime version, you may be able to mitigate this error by temporarily rolling back to the previous runtime version. For more information, see

<https://docs.aws.amazon.com/lambda/latest/dg/runtimes-update.html>

```
[ERROR] Runtime.ImportModuleError: Unable to
import module 'lambda_function': No module named
'flask'
```

Traceback (most recent call last):

INIT_REPORT Init Duration: 144.10 ms Phase: init

Status: error Error Type: Runtime.ImportModuleError

[WARNING] 2025-11-24T14:30:58.663Z

LAMBDA_WARNING: Unhandled exception. The most likely cause is an issue in the function code. However, in rare cases, a Lambda runtime update can cause unexpected function behavior. For functions using managed runtimes, runtime updates can be triggered by a function change, or can be applied automatically. To determine if the runtime has been updated, check the runtime version in the INIT_START log entry. If this error correlates with a change in the runtime version, you may be able to mitigate this error by temporarily rolling back to the previous runtime version. For more information, see

<https://docs.aws.amazon.com/lambda/latest/dg/runtimes-update.html>

```
[ERROR] Runtime.ImportModuleError: Unable to
import module 'lambda_function': No module named
'flask'
```

Traceback (most recent call last):

INIT_REPORT Init Duration: 91.77 ms Phase: invoke

Status: error Error Type: Runtime.ImportModuleError

START RequestId: 7b036e2f-b197-4d6f-801f-

86f4cc5802f8 Version: \$LATEST

END RequestId: 7b036e2f-b197-4d6f-801f-

86f4cc5802f8

REPORT RequestId: 7b036e2f-b197-4d6f-801f-

86f4cc5802f8 Duration: 103.06 ms Billed Duration:

104 ms Memory Size: 128 MB Max Memory Used: 46

MB Status: error Error Type:

Runtime.ImportModuleError

Request ID: 7b036e2f-b197-4d6f-801f-86f4cc5802f8

已思考 13s >

你這個錯誤很明確：**Lambda** 在載入你的 `lambda_function.py` 時，**import** 到 `flask`，但部署包裡沒有 **flask** 套件，所以直接在 `init` 階段爆掉。

`No module named 'flask'` = 你的 zip 只上傳程式碼，沒有把依賴套件一起打包。

下面給你 3 種修法，照你情況選最快的。

方案 1（最常用）：把 flask 一起打進 ZIP 再上傳

在你本地專案資料夾（有 `lambda_function.py` 那層）做：

Windows / PowerShell 範例：

powershell

 複製程式碼

`cd` 你的專案資料夾

1) 安裝套件到目前資料夾

`pip install flask -t .`

2) 重新把整個資料夾打包

`Compress-Archive -Path * -DestinationPath function.zip -Force`

macOS / Linux：

bash

 複製程式碼

`cd` 你的專案資料夾

`pip install flask -t .`

`zip -r function.zip .`

然後回 Lambda console → Upload zip 覆蓋上去，再 Test。

Flask 是純 Python 套件，不吃編譯，直接在 Windows 打包也 OK。

方案 2：用 CloudShell 打包（你是 voclabs 很推薦）

你之前 IAM 被鎖，所以用 CloudShell 最穩：

1. AWS Console 開 **CloudShell**
2. 上傳你的程式檔 (或 git clone 專案)
3. 在 CloudShell 裡：

```
bash
```

[複製程式碼](#)

```
cd 你的專案資料夾  
pip install flask -t .  
zip -r function.zip .
```

4. 下載 `function.zip` 再上傳到 Lambda

方案 3：用 Lambda Layer (如果你帳號允許建立 Layer)

Console → Lambda → Layers → Create layer

上傳一個只放 flask 的 layer，再在 function → Add a layer。

但 **voclabs/academy** 有時候 layer 也會被限制，所以我更建議用方案 1/2。

重要提醒：如果你是在 Lambda 跑「Flask 伺服器」

Lambda 不能像本地一樣 `app.run()` 開伺服器。

正確做法是：

- 用 API Gateway 觸發 Lambda
- 你的 Flask app 要用 adapter 轉成 Lambda handler

常見是 `awscli` / `serverless-wsgi` / `mangum` (ASGI)

如果你貼你現在 `lambda_function.py` 裡 Flask 的寫法 (有沒有 `app = Flask(__name__)`、`routes`、`app.run()`)，我可以直接幫你改成 Lambda 正確可跑的版本。

你先選 **方案 1 或 2** 重新打包上傳，應該就能解掉 `flask` 找不到的問題。

如果上傳後又出現新錯誤 (例如 handler 或 `app.run` 相關)，把 log 貼我，我接著帶你修。

[複製](#) [點讚](#) [收藏](#) [分享](#) [刷新](#) ...

```
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import sqlite3
import requests
from PIL import Image
# from db import init_db, insert_record
# init_db()
load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"
app = Flask(__name__)

# 本地儲存目錄
LOCAL_STORAGE = './generated_files'
os.makedirs(f'{LOCAL_STORAGE}/images',
exist_ok=True)
os.makedirs(f'{LOCAL_STORAGE}/audios',
exist_ok=True)

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少
prompt"}), 400

        headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
        payload = {"inputs": prompt}
```

```
response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

content_type = response.headers.get('Content-
Type', '')

if 'application/json' in content_type:
    error_data = response.json()
    return jsonify({
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# 嘗試解析圖片
try:
    image =
Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

# 儲存到本地
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S
")
filename = f"
```



```
{timestamp}_{uuid.uuid4().hex[:8]}.png"
    local_path = f"
{LOCAL_STORAGE}/images/{filename}"
    image.save(local_path, format="PNG")

    # 本地 URL
    local_url =
f"http://localhost:5000/generated_files/images/{filena
me}"

    # 寫入 SQLite
    # insert_record(user_id, prompt, local_url)

    return jsonify({
        "success": True,
        "local_url": local_url,
        "reply": f"Flask 收到圖片 prompt: {prompt}"
    })

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")

        filename = "ta.txt"
        # 模擬生成圖片，先回傳本地 txt URL
        # local_url =
f"http://localhost:5000/generated_files/test.txt"
        local_url =
f"http://localhost:5000/generated_files/audios/{filenam
e}"

        return jsonify({
            "success": True,
            "local_url": local_url,
            "reply": f"Flask 收到音樂 prompt: {prompt}"
        })
```

```
except Exception as e:
    return jsonify({"success":False, "error":str(e)}),500
# 呼叫索取生成結果(圖片/音樂)
@app.route('/generated_files/<file_type>/<filename>',
methods=['GET'])
def serve_file(file_type, filename):
    """
    file_type: 'images' 或 'audios'
    filename: 檔案名稱
    """
    # 驗證子資料夾
    if file_type not in ["images", "audios"]:
        return jsonify({"success": False, "error": "不支援的檔案類型"}), 400

    directory = os.path.join(LOCAL_STORAGE, file_type)
    return send_from_directory(directory, filename)

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL query 取得 user_id
        conn = sqlite3.connect("bot_records.db")
        c = conn.cursor()

        if user_id:
            c.execute("SELECT id, user_id, timestamp, prompt, file_url FROM records WHERE user_id=? ORDER BY id DESC", (user_id,))
        else:
            c.execute("SELECT id, user_id, timestamp, prompt, file_url FROM records ORDER BY id DESC")

        rows = c.fetchall()
        conn.close()

        return jsonify({
            "success": True,
            "records": [
                {"id": r[0], "user_id": r[1], "timestamp": r[2],
                "prompt": r[3], "file_url": r[4]}
            ]
        })
```

```
        for r in rows
    ]
})
except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)
這是我要打包的程式，還有沒有需要安裝的
```

你這份程式需要另外打包的第三方套件主要就這幾個（你 import 到、而且不是 Python 內建的）：

你一定要裝進 ZIP 的

1. Flask

- 你有 `from flask import ...`

2. python-dotenv

- 你有 `from dotenv import load_dotenv`

3. requests

- 你有 `import requests`

4. Pillow

- 你有 `from PIL import Image`

不用裝的（Python 內建）

`json, os, io, uuid, datetime, sqlite3` 都是標準庫，不用 pip 安裝。

requirements.txt 可以寫這樣

txt

複製程式碼

```
Flask
python-dotenv
requests
Pillow
```

然後本地打包 (在專案根目錄) :

bash

 複製程式碼

```
pip install -r requirements.txt -t .  
zip -r function.zip .
```

但我一定要提醒：這份「原生 Flask 伺服器」不能直接在 Lambda 跑

你現在程式最後有：

py

 複製程式碼

```
if __name__ == '__main__':  
    app.run(...)
```

Lambda 不會開 server、也不能用 localhost，所以你就算把套件打包好，也會遇到：

- 沒有一直跑的 web server
- `http://localhost:5000/...` 在 Lambda 沒意義
- 寫檔只能寫 `/tmp` (其他路徑唯讀)

你要做兩個小改動才會是 Lambda 版

1. 加一個 Lambda handler (用 awsgi 轉 Flask)

多裝一包 `awsgi`：

requirements.txt 變：

txt

 複製程式碼

```
Flask  
python-dotenv  
requests  
Pillow  
awsgi
```

2. 檔案最下面改成：

py

 複製程式碼

```
import awsgi
```

```
def lambda_handler(event, context):  
    return awscli.response(app, event, context)
```

然後把 `app.run(...)` 那段刪掉/註解。

之後用 **API Gateway 觸發 Lambda**，你的 Flask route 才會被打到。

另外兩個「Lambda 必改點」（不然功能邏輯會怪）

1. dotenv 在 Lambda 通常不用了

你可以把 token 改成在 Console → Environment variables 設定，

`load_dotenv()` 留著也行，但只是多餘，不會壞。

2. 本地存檔要改到 /tmp 或 S3

- Lambda 只能寫 `/tmp`（容量預設 512MB）

所以：

py

複製程式碼

```
LOCAL_STORAGE = '/tmp/generated_files'
```

- 真正要永久保存/給前端下載，建議改存 S3 再回傳 **presigned URL**（你現在回 `localhost` 下載一定失敗）

如果你要我幫你「直接改成可在 Lambda + API Gateway 跑的版本（含 S3 存檔回傳 URL）」，你跟我說：

1. 你 API Gateway 用 HTTP API 還是 REST API

2. 你的 S3 bucket 名稱

我就直接幫你把整份程式改好 + handler 一次到位。

📄 🍌 🗨 📶 🔄 ...

pip: 無法辨識 'pip' 詞彙是否為 Cmdlet、函數、指令檔或可執行程式的名稱
。請檢查名稱拼字是否正確，如果包含路徑的話，請確認路徑是否正確，然後再試一次。
位於 線路:1 字元:1
+ pip install -r requirement.txt -t.

```
+ ~~~  
+ CategoryInfo          : ObjectNotFound: (pip:String) [],  
CommandNotFoun  
dNotFoundException  
+ FullyQualifiedErrorId :  
CommandNotFoundException  
  
PS C:\Users\Neo Yeh\Desktop\雲端實務  
\FinalProject\backend\src>
```

你這個錯誤代表 **PowerShell** 找不到 **pip** 指令。在 Windows 很常見，通常用下面幾種方式就能解。

另外你指令裡寫的是 `requirement.txt`，多數專案是 `requirements.txt`（有 s）。如果檔名真的是 `requirement.txt` 也可以，但要確定拼對。

✅ 方法 1（最推薦）：用 `python -m pip`

在你現在這個資料夾直接打：

powershell

 複製程式碼

```
python -m pip install -r requirements.txt -t .
```

如果你的檔名是 `requirement.txt`，就改成：

powershell

 複製程式碼

```
python -m pip install -r requirement.txt -t .
```

`python -m pip` 不需要 `pip` 在 PATH 裡，也能正常跑。

✅ 方法 2：用 Windows 的 `py` launcher

如果 `python` 也不行，用：

powershell

 複製程式碼

```
py -m pip install -r requirements.txt -t .
```

✅ 方法 3：先確認 Python 有沒有裝好

在 PowerShell 輸入：

```
powershell
```

[複製程式碼](#)

```
python --version
```

或

```
powershell
```

[複製程式碼](#)

```
py --version
```

- 如果有版本號 (例如 Python 3.12.x) → 代表 Python 有裝，可以回去用方法 1/2。
- 如果顯示找不到 → 代表你 Python 沒裝或沒加 PATH。

✅ 方法 4：pip 在 Scripts 目錄，用完整路徑

通常 pip 在：

```
makefile
```

[複製程式碼](#)

```
C:\Users\<你帳號>\AppData\Local\Programs\Python\Python3x\Scripts\pip.exe
```

你可以先找你的 Python 安裝位置，然後這樣跑：

```
powershell
```

[複製程式碼](#)

```
& "C:\Users\Neo Yeh\AppData\Local\Programs\Python\Python312\Scripts\pip.exe" inst
```

(路徑依你實際版本改)

✅ 補充：建議你用虛擬環境 (避免汙染系統)

在 backend\src 這層：

```
powershell
```

[複製程式碼](#)

```
python -m venv .env  
.env\Scripts\activate
```

```
python -m pip install -r requirements.txt -t .
```

最後再打包

裝好套件後：

powershell

 複製程式碼

```
Compress-Archive -Path * -DestinationPath function.zip -Force
```

常見踩雷提醒

1. requirements 檔名要對

- 大多是 requirements.txt

2. 你現在在 backend\src

- 請確認 lambda_function.py (或你的入口檔) 跟 requirements 在同一層再裝/打包

3. 如果裝 Pillow 失敗 (少見)

- 用 python -m pip install Pillow -t . 單獨裝一次

你先試 **方法 1** 那行，若還報錯，把輸出貼我，我直接幫你判斷是 Python 沒裝、還是路徑/檔名問題。

     ...

bash: Compress-Archive: command not found

你現在是在 **bash / Git Bash / WSL** 裡跑指令，所以 **Compress-Archive** (PowerShell 指令) 當然找不到。

在 bash 要用 **zip** (或用 PowerShell 來壓) 。

 在 bash 裡壓成 function.zip

先確認你在「要打包的那一層」(有 `lambda_function.py` 和你剛 `pip` 裝進來的套件資料夾那層)

```
bash
```

[複製程式碼](#)

```
pwd
```

```
ls
```

然後壓縮：

```
bash
```

[複製程式碼](#)

```
zip -r function.zip .
```

- 代表把目前資料夾全部打包進 zip。

如果你的 bash 沒有 zip 指令

方案 A：裝 zip (WSL / Linux)

```
bash
```

[複製程式碼](#)

```
sudo apt update
```

```
sudo apt install zip
```

```
zip -r function.zip .
```

方案 B：改用 PowerShell 壓

在 PowerShell (不是 bash) 開同一個資料夾，跑：

```
powershell
```

[複製程式碼](#)

```
Compress-Archive -Path * -DestinationPath function.zip -Force
```

小提醒：打包前要先把套件裝進同一層

在 bash 同一層跑 (記得用 `python -m pip`)：

```
bash
```

[複製程式碼](#)

```
python -m pip install -r requirements.txt -t .  
zip -r function.zip .
```

如果你願意貼一下 `pwd` 和 `ls` 的結果，我可以幫你確認「目前位置對不對、zip 會不會漏打包」。



我用右鍵壓縮是一樣的效果嗎

已思考 8s >

可以，用右鍵壓縮也可以，重點是壓縮「的方式」而不是指令本身 🍌

對 Lambda 來說，只要是標準 `.zip` 檔就行，不管你是用：

- `zip -r function.zip .`
- PowerShell 的 `Compress-Archive`
- 檔案總管右鍵 → 送到 → 壓縮 (zip) 資料夾

都可以，但一定要注意資料夾層級：

✅ 正確做法 (右鍵版)

假設你的結構像這樣：

text

複製程式碼

```
backend\src\  
  lambda_function.py  
  requirements.txt  
  flask/...  
  requests/...  
  PIL/...  
  dotenv/...
```

1. 先用檔案總管打開到 `backend\src` 這一層
(要看到 `lambda_function.py` 那一層)

- 在這個資料夾裡 按 `Ctrl + A` 全選全部檔案和資料夾
(不要整個往上一層選 `src` 資料夾)
- 在「已選取的檔案上」右鍵 →
送到 (Send to) → 壓縮 (zipped) 資料夾
- 把產生的 zip 改名成 `function.zip` , 上傳到 Lambda

這樣壓出來的 zip 打開後, 最上層會直接是 :

text

 複製程式碼

```
lambda_function.py
flask/...
requests/...
PIL/...
...
```

Lambda 解壓後就能直接 `import flask` 、找到 `lambda_function.py` 。

✗ 錯誤常見做法

如果你在「上一層」直接對整個資料夾右鍵壓縮, 例如 :

- 對 `src` 右鍵 → 壓縮 → 變成 `src.zip`

那 zip 裡會長這樣 :

text

 複製程式碼

```
src/
  lambda_function.py
  flask/...
  ...
```

Lambda 解壓後看到的是 `/var/task/src/lambda_function.py` ,
但 handler 設定的是 `lambda_function.lambda_handler` ,
→ 會找不到 module (`Unable to import module 'lambda_function'`) 。

快速檢查

壓好之後打開 `function.zip` 看一下 :

- ✓ 最上層看得到 `lambda_function.py` → OK

- **✗** 最上層只有一個資料夾 (例如 `src/` , 點進去才看到檔案) → 要重壓一次, 用「選內容壓縮」的方式。

你如果方便, 截一張「你目前資料夾內容」的畫面 (看到 `lambda_function.py` 的那層) 給我, 我可以直接跟你說哪一層要右鍵壓縮, 避免上傳後又遇到 `import` 問題。

< 1/2 >      ...

```
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import sqlite3
import requests
from PIL import Image
import awsgi
# from db import init_db, insert_record
# init_db()
load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"
app = Flask(__name__)

# 本地儲存目錄
LOCAL_STORAGE = './generated_files'
os.makedirs(f'{LOCAL_STORAGE}/images',
exist_ok=True)
os.makedirs(f'{LOCAL_STORAGE}/audios',
exist_ok=True)

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")
```

```
user_id = data.get("user_id", "unknown")

if not prompt:
    return jsonify({"success": False, "error": "缺少
prompt"}), 400

headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
payload = {"inputs": prompt}

response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

content_type = response.headers.get('Content-
Type', '')

if 'application/json' in content_type:
    error_data = response.json()
    return jsonify({
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# 嘗試解析圖片
try:
    image =
Image.open(io.BytesIO(response.content))
except Exception as e:
```

```
        return jsonify({
            "success": False,
            "error": f"圖片解析失敗: {str(e)}"
        }), 500

    # 儲存到本地
    timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
    local_path = f"
{LOCAL_STORAGE}/images/{filename}"
    image.save(local_path, format="PNG")

    # 本地 URL
    local_url =
f"http://localhost:5000/generated_files/images/{filena
me}"

    # 寫入 SQLite
    # insert_record(user_id, prompt, local_url)

    return jsonify({
        "success": True,
        "local_url": local_url,
        "reply": f"Flask 收到圖片 prompt: {prompt}"
    })

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")

        filename = "ta.txt"
        # 模擬生成圖片，先回傳本地 txt URL
        # local_url =
```

```
f"http://localhost:5000/generated_files/test.txt"
    local_url =
f"http://localhost:5000/generated_files/audios/{filename}"
e}"

    return jsonify({
        "success": True,
        "local_url": local_url,
        "reply": f"Flask 收到音樂prompt: {prompt}"
    })

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500
# 呼叫索取生成結果(圖片/音樂)
@app.route('/generated_files/<file_type>/<filename>',
methods=['GET'])
def serve_file(file_type, filename):
    """
    file_type: 'images' 或 'audios'
    filename: 檔案名稱
    """
    # 驗證子資料夾
    if file_type not in ["images", "audios"]:
        return jsonify({"success": False, "error": "不支援的檔案類型"}), 400

    directory = os.path.join(LOCAL_STORAGE, file_type)
    return send_from_directory(directory, filename)

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
        query 取得 user_id
        conn = sqlite3.connect("bot_records.db")
        c = conn.cursor()

        if user_id:
            c.execute("SELECT id, user_id, timestamp,
prompt, file_url FROM records WHERE user_id=?
ORDER BY id DESC", (user_id,))
        else:
            c.execute("SELECT id, user_id, timestamp,
```

```
prompt, file_url FROM records ORDER BY id DESC")

rows = c.fetchall()
conn.close()

return jsonify({
    "success": True,
    "records": [
        {"id": r[0], "user_id": r[1], "timestamp": r[2],
"prompt": r[3], "file_url": r[4]}
        for r in rows
    ]
})
except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

要改成lambda

已思考 24s >

可以，這支 Flask 要改成 Lambda 版其實只要 **改三個重點**：

1. 不要再 `app.run()` 開伺服器 (Lambda 不開 server)
2. 加一個 `lambda_handler`，用 `awscli` 把 Flask 接到 Lambda
3. 檔案跟 SQLite 不能寫在現在的路徑，要改寫 `/tmp` (Lambda 只有 `/tmp` 可寫)

我先直接給你「改好的版本」，再幫你畫重點。

✅ 改成 Lambda 版的完整程式 (假設檔名 `lambda_function.py`)

python

複製程式碼

```
from flask import Flask, request, jsonify, send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import sqlite3
import requests
from PIL import Image
```



```
import awsgi # 用來把 Flask app 包成 Lambda handler

# from db import init_db, insert_record
# init_db()
load_dotenv()
HuggingFace_Token = os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
HuggingFace_URL = "https://router.huggingface.co/hf-inference/models/stabilityai/

app = Flask(__name__)

# ✅ Lambda 上可以寫檔的只有 /tmp
LOCAL_STORAGE = '/tmp/generated_files'
os.makedirs(f'{LOCAL_STORAGE}/images', exist_ok=True)
os.makedirs(f'{LOCAL_STORAGE}/audios', exist_ok=True)

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json or {}
        prompt = data.get("prompt", "沒有收到圖片prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少 prompt"}), 400

        headers = {"Authorization": f"Bearer {HuggingFace_Token}"}
        payload = {"inputs": prompt}

        response = requests.post(HuggingFace_URL, headers=headers, json=payload,

        if response.status_code != 200:
            return jsonify({
                "success": False,
                "error": f"Hugging Face API 錯誤: {response.status_code}",
                "details": response.text
            }), 500

        content_type = response.headers.get('Content-Type', '')

        if 'application/json' in content_type:
            error_data = response.json()
```

```

        return jsonify({
            "success": False,
            "error": "Hugging Face 回傳錯誤",
            "details": error_data
        }), 500

    if 'image' not in content_type:
        return jsonify({
            "success": False,
            "error": f"非預期的回傳格式: {content_type}"
        }), 500

    # 嘗試解析圖片
    try:
        image = Image.open(io.BytesIO(response.content))
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"圖片解析失敗: {str(e)}"
        }), 500

    # ✅ 儲存在 /tmp 底下
    timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.png"
    local_path = f"{LOCAL_STORAGE}/images/{filename}"
    image.save(local_path, format="PNG")

    # ⚠️ Lambda 沒有 localhost，這裡改成「相對路徑」，前端自己組完整 URL
    relative_url = f"/generated_files/images/{filename}"

    # 寫入 SQLite (如果要寫 DB，檔案路徑也應該放 /tmp 或用 RDS/S3，這裡先略)
    # insert_record(user_id, prompt, relative_url)

    return jsonify({
        "success": True,
        "file_path": relative_url,
        "reply": f"Flask 收到圖片prompt: {prompt}"
    })

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

```

```
@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        data = request.json or {}
        prompt = data.get("prompt", "沒有收到圖片prompt")

        # 這裡先假裝有一個 txt 放在 audios 裡
        filename = "ta.txt"
        # 你可以改成真的生成 audio 檔案存 /tmp
        relative_url = f"/generated_files/audios/{filename}"

        return jsonify({
            "success": True,
            "file_path": relative_url,
            "reply": f"Flask 收到音樂prompt: {prompt}"
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

# 呼叫索取生成結果(圖片/音樂)
@app.route('/generated_files/<file_type>/<filename>', methods=['GET'])
def serve_file(file_type, filename):
    """
    file_type: 'images' 或 'audios'
    filename: 檔案名稱
    """
    # 驗證子資料夾
    if file_type not in ["images", "audios"]:
        return jsonify({"success": False, "error": "不支援的檔案類型"}), 400

    directory = os.path.join(LOCAL_STORAGE, file_type)
    return send_from_directory(directory, filename)

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL query 取得 user_id

        # ⚠️ Lambda 上直接在當前目錄建 sqlite 檔會有權限問題，建議也放 /tmp
        db_path = os.path.join('/tmp', 'bot_records.db')
```

```

conn = sqlite3.connect(db_path)
c = conn.cursor()

# 這裡最好在程式啟動時先建立 table，這裡簡單保底一下：
c.execute("""
    CREATE TABLE IF NOT EXISTS records (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id TEXT,
        timestamp TEXT,
        prompt TEXT,
        file_url TEXT
    )
""")

if user_id:
    c.execute(
        "SELECT id, user_id, timestamp, prompt, file_url FROM records WHERE (user_id,)"
    )
else:
    c.execute("SELECT id, user_id, timestamp, prompt, file_url FROM records")

rows = c.fetchall()
conn.close()

return jsonify({
    "success": True,
    "records": [
        {"id": r[0], "user_id": r[1], "timestamp": r[2], "prompt": r[3],
         for r in rows
        }
    ]
})

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

#  Lambda 的入口：用 awsgi 把 Flask app 包成 Lambda handler
def lambda_handler(event, context):
    return awsgi.response(app, event, context)

```

Lambda Console 的 Handler 設定：

如果你的檔名是 `lambda_function.py`，就填：

text

 複製程式碼

```
lambda_function.lambda_handler
```

必須一起打包進 ZIP 的套件 (requirements.txt)

你的 `requirements.txt` 建議寫：

txt

 複製程式碼

```
Flask
python-dotenv
requests
Pillow
awscli
```

在 有 `lambda_function.py` 的那一層：

bash

 複製程式碼

```
python -m pip install -r requirements.txt -t .
# 然後用右鍵或 zip -r 把目前所有檔案壓成 function.zip
```

上傳這個 `function.zip` 到 Lambda。

API Gateway / 前端 URL 怎麼用？

- 部署到 Lambda + API Gateway 之後，API 的 base URL 會像：

text

 複製程式碼

```
https://xxxxx.execute-api.ap-northeast-1.amazonaws.com/prod
```

- 你呼叫 `/generate-image` 時，前端就打：

text

 複製程式碼

```
POST https://xxxxx.execute-api.../prod/generate-image
```

- 後端回傳 `file_path: "/generated_files/images/xxxx.png"`
前端要自己組成：

```
text
```

 複製程式碼

```
https://xxxxx.execute-api.../prod/generated_files/images/xxxx.png
```

去 GET 圖片。

(如果你要我順便幫你畫 API Gateway 那邊要怎麼接這支 Lambda，我也可以一步一步幫你配。)

你現在下一步建議：

1. 把程式改成上面這版
2. 確認 `requirements.txt` 有那 5 個套件
3. 在「有 `lambda_function.py` 的那一層」裝套件、壓 zip
4. 上傳到 Lambda、handler 設 `lambda_function.lambda_handler`
5. 先在 Lambda console 裡 Test 一次 (用一個包含 prompt 的事件)，看有沒有成功回應

測完如果有新的 error message (`Runtime.ImportModuleError`、awscli 相關、IO 錯誤之類)，直接把錯誤貼給我，我可以針對那個錯誤幫你修到通。

     ...

圖片要存在s3bucket

已思考 33s >

OK，我幫你把「圖片存本機」改成「圖片上傳到 S3 bucket」，而且是 Lambda 可以跑的版本 🙌

1. Lambda 這邊要先準備兩件事

(1) 在 Lambda 設定環境變數

到 Lambda → 你的 function → **Configuration** → **Environment variables** 新增：

- `HUGGINGFACE_TOKEN`：你原本的 key
- `S3_BUCKET_NAME`：你要存圖片的 bucket 名字 (例：`my-finalproject-bucket`)

(2) Lambda 的執行角色 (LabRole) 要有 S3 權限

如果可以調整 policy (或老師幫你設) , 最小權限大概這樣 (bucket 名字記得換掉) :

json

 複製程式碼

```
{
  "Effect": "Allow",
  "Action": [
    "s3:PutObject",
    "s3:GetObject"
  ],
  "Resource": "arn:aws:s3:::my-finalproject-bucket/*"
}
```

你是 voclabs 的 LabRole , 這一段可能要老師預先配好 , 如果沒有權限就會看到 AccessDenied 。

2. 程式 : 修改成「圖片存 S3」的 Lambda 版

需要的套件 (requirements.txt)

txt

 複製程式碼

```
Flask
python-dotenv
requests
Pillow
awscli
```

不用安裝 boto3 , Lambda 內建已經有。

修改後的 lambda_function.py 範例 (重點是 generate_image)

python

 複製程式碼

```
from flask import Flask, request, jsonify, send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import sqlite3
import requests
```

```
from PIL import Image
import awsgi
import boto3 # ✅ 新增: S3 用

load_dotenv()

HuggingFace_Token = os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
HuggingFace_URL = "https://router.huggingface.co/hf-inference/models/stabilityai/

# ✅ 從環境變數讀取 S3 bucket 名
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
AWS_REGION = os.getenv("AWS_REGION", "ap-northeast-1")

app = Flask(__name__)

# ✅ 建立 S3 client
s3 = boto3.client("s3")

# (如果還有要用到本地檔案, 可用 /tmp; 圖片現在改丟 S3, 就不用 LOCAL_STORAGE 了)
# LOCAL_STORAGE = '/tmp/generated_files'
# os.makedirs(f'{LOCAL_STORAGE}/images', exist_ok=True)
# os.makedirs(f'{LOCAL_STORAGE}/audios', exist_ok=True)

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json or {}
        prompt = data.get("prompt", "沒有收到圖片prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少 prompt"}), 400

        headers = {"Authorization": f"Bearer {HuggingFace_Token}"}
        payload = {"inputs": prompt}

        # 呼叫 HuggingFace 產圖
        response = requests.post(HuggingFace_URL, headers=headers, json=payload,

        if response.status_code != 200:
            return jsonify({
```



```

        "success": False,
        "error": f"Hugging Face API 錯誤: {response.status_code}",
        "details": response.text
    }), 500

content_type = response.headers.get('Content-Type', '')

if 'application/json' in content_type:
    error_data = response.json()
    return jsonify({
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# 嘗試解析圖片 (可選，其實你也可以直接把 response.content 丟到 S3)
try:
    image = Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

# 生成檔名 & S3 key
timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = f"images/{filename}"

# ✅ 上傳到 S3
try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content,          # 直接用原始 bytes
        ContentType="image/png"
    )

```

```

except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

#  建立一個 1 小時有效的 presigned URL 給前端下載
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

# 也可以順便記錄到 DB:file_url 存 s3_key 或 presigned_url
# insert_record(user_id, prompt, s3_key)

return jsonify({
    "success": True,
    "s3_key": s3_key,
    "download_url": presigned_url,
    "reply": f"Flask 收到圖片prompt: {prompt}"
})

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

# 其餘 /generate-audio、/history 你可以先保留或再調整成 S3 / RDS 版本

def lambda_handler(event, context):
    # awsgi 把 API Gateway 事件轉給 Flask app
    return awsgi.response(app, event, context)

```

Lambda Handler 設定 (再次提醒)

如果檔名叫 `lambda_function.py` :

- **Handler** 填 : `lambda_function.lambda_handler`

3. 部署流程小 recap (你現在的情況)

1. 在 `backend/src` (有 `lambda_function.py` 的那層) 放好 :

- `lambda_function.py` (這一版)
- `requirements.txt` (那 5 個套件)

2. 在同一層執行 (PowerShell / bash 擇一) :

bash

複製程式碼

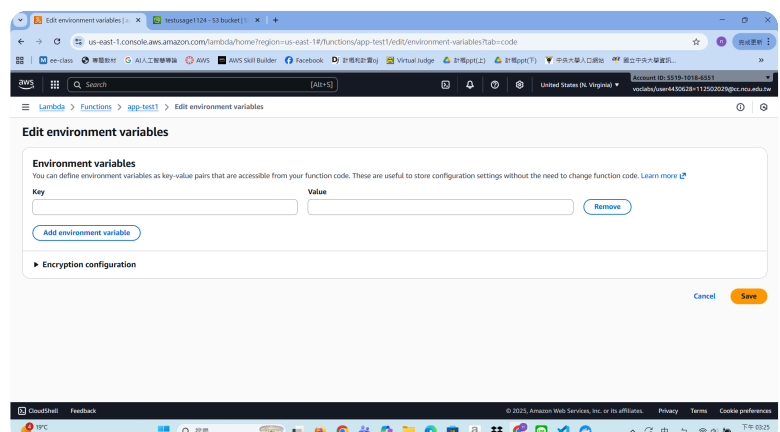
```
python -m pip install -r requirements.txt -t .  
# 安裝完後，用右鍵「全選」→ 右鍵壓縮成 function.zip  
# 或在 bash: zip -r function.zip .
```

3. 到 Lambda :

- Code → Upload from → .zip file → 上傳 `function.zip`
- Runtime settings → Handler : `lambda_function.lambda_handler`

4. 先用 Test 事件測試 (記得 event 要長得像 API Gateway HTTP 事件，或先用簡單的一般 event 測試 `awscli`)

如果你願意把「S3 bucket 名字」跟「你現在 Lambda 是用 HTTP API 還是還沒接 API Gateway」跟我說，我可以順便幫你寫一個 **完整測試用的 API Gateway event JSON**，讓你在 Lambda Console 的 Test 直接看到這支 `/generate-image` 跑起來。



是這個環境變數嗎，怎麼填

對，就是這個畫面要填 🍌

你程式裡用到的環境變數有這幾個：

python

複製程式碼

```
HuggingFace_Token = os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
AWS_REGION = os.getenv("AWS_REGION", "ap-northeast-1")
```

所以在這個畫面這樣填：

1. HUGGINGFACE_TOKEN

1. 在 **Key** 打： `HUGGINGFACE_TOKEN`
2. 在 **Value** 打：你在本地用的 Hugging Face Token (整串複製貼上)

2. SUNO_TOKEN (如果目前沒用到，可以先不填或留空)

如果你之後會用 Suno，就加一個：

- Key： `SUNO_TOKEN`
- Value：你的 Suno token (如果沒有就先不要加這一個)

3. S3_BUCKET_NAME

這個一定要有，因為 Lambda 要知道圖片丟哪個 bucket：

- Key： `S3_BUCKET_NAME`
- Value：你的 S3 bucket 名字，例如：
`finalproject-image-bucket`
(就是你在 S3 建的那個名稱，完全一樣)

4. AWS_REGION (可加可不加)

程式裡有：

python

 複製程式碼

```
AWS_REGION = os.getenv("AWS_REGION", "ap-northeast-1")
```

你現在 Lambda 在 **us-east-1** (畫面右上角寫 United States (N. Virginia)) ,
建議順便加一個讓它對得起來：

- Key : **AWS_REGION**
- Value : **us-east-1**

實際操作步驟

1. 在這個畫面：
 - 第一列：Key = **HUGGINGFACE_TOKEN** , Value = 你的 HF token
2. 按 **Add environment variable**
 - 第二列：Key = **S3_BUCKET_NAME** , Value = 你的 bucket 名
3. 再按 **Add environment variable** (可選)
 - 第三列：Key = **AWS_REGION** , Value = **us-east-1**
4. 如果要用 Suno 就再加一列 **SUNO_TOKEN**
5. 右下按 **Save**

存好後，Lambda 下次執行，`os.getenv(...)` 就會讀到這些值了，不用再改程式。

你可以先把 **HUGGINGFACE_TOKEN + S3_BUCKET_NAME + AWS_REGION** 這三個填好存檔，

如果之後 Test 又有新的錯誤訊息 (例如 S3 AccessDenied 或 Hugging Face 回傳錯誤) ，
把 log 貼給我，我再幫你往下調。

     ...

↳ `s3_key = f"images/{filename}"`

什麼意思

啊～你是問這行：

python

 複製程式碼

```
s3_key = f"images/{filename}"
```

那我拆開講 📌

1. 什麼是 `s3_key` ？

在 S3 裡，每個檔案不是叫「檔名」，而是叫 **Object Key**，就是「在 bucket 裡的完整路徑字串」。

例：

- bucket 名：`my-bucket`
- key：`images/20251125_abc123.png`

這個物件就好像放在：

text

複製程式碼

my-bucket

└─ images/

└─ 20251125_abc123.png

所以 `s3_key` 這個變數，就是你要把圖片存到 bucket 裡的路徑。

2. `f"images/{filename}"` 的意思

假設 `filename = "20251125_abcd1234.png"`，

那這行：

python

複製程式碼

```
s3_key = f"images/{filename}"
```

就會變成：

python

複製程式碼

```
s3_key = "images/20251125_abcd1234.png"
```

也就是：「在 bucket 裡的 `images` 資料夾底下，叫這個檔名」。

之後上傳會這樣用：

python

 複製程式碼

```
s3.put_object(  
    Bucket=S3_BUCKET_NAME,  
    Key=s3_key,  
    Body=response.content,  
    ContentType="image/png"  
)
```

= 把圖片丟到：

你設定的 bucket / `images/那個檔名`

3. 如果我不要 `images/` 這個資料夾呢？

可以，改成：

python

 複製程式碼

```
s3_key = filename
```

那檔案就會直接放在 bucket 根目錄，不會多一層 `images`。

4. 也可以客製更多層

例如你想依照 user 或日期分類：

python

 複製程式碼

```
s3_key = f"{user_id}/images/{filename}"  
# 或  
s3_key = f"{datetime.date.today()}/images/{filename}"
```

就會變成：

text

 複製程式碼

```
<bucket> / 112502029/images/20251125_abcd1234.png
```

如果你跟我說 **你的 bucket 名字**，我可以直接幫你寫出實際會長什麼樣子的 S3 路徑跟下載 URL，讓你比較有感覺。



```
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import sqlite3
import requests
from PIL import Image
import awsgi
import boto3

# from db import init_db, insert_record
# init_db()
load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"
app = Flask(__name__)

s3 = boto3.client("s3") # 建立 s3 client

# 本地儲存目錄
# LOCAL_STORAGE = './generated_files'
# os.makedirs(f'{LOCAL_STORAGE}/images',
exist_ok=True)
# os.makedirs(f'{LOCAL_STORAGE}/audios',
exist_ok=True)

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")
```



```
user_id = data.get("user_id", "unknown")

if not prompt:
    return jsonify({"success": False, "error": "缺少
prompt"}), 400

headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
payload = {"inputs": prompt}

response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

content_type = response.headers.get('Content-
Type', '')

if 'application/json' in content_type:
    error_data = response.json()
    return jsonify({
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# 嘗試解析圖片
try:
    image =
Image.open(io.BytesIO(response.content))
except Exception as e:
```

```
        return jsonify({
            "success": False,
            "error": f"圖片解析失敗: {str(e)}"
        }), 500

    # 儲存到本地
    # timestamp =
    datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    # filename = f"
    {timestamp}_{uuid.uuid4().hex[:8]}.png"
    local_path = f"
    {LOCAL_STORAGE}/images/{filename}"
    image.save(local_path, format="PNG")

    # 本地 URL
    # local_url =
    f"http://localhost:5000/generated_files/images/{filename}"

    # 存到 S3
    timestamp =
    datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"
    {timestamp}_{uuid.uuid4().hex[:8]}.png"
    s3_key = filename

    try:
        s3.put_object(
            Bucket=S3_BUCKET_NAME,
            Key=s3_key,
            Body=response.content,    # 直接用原始 bytes
            ContentType="image/png"
        )
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"上傳 S3 失敗: {str(e)}"
        }), 500

    # 寫入 SQLite
    # insert_record(user_id, prompt, local_url)
```

```
return jsonify({
    "success": True,
    "local_url": local_url,
    "reply": f"Flask 收到圖片 prompt: {prompt}"
})

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500
```

已思考 36s >

你這版有幾個地方一定會炸掉，我先點出問題，再直接給你「修好、可以存 S3 的完整版本」。

你這段程式目前的問題

在你貼的這一版裡：

python

 複製程式碼

```
# LOCAL_STORAGE 被註解掉了
# LOCAL_STORAGE = './generated_files'

...

# 儲存到本地
local_path = f"{LOCAL_STORAGE}/images/{filename}" # ❌ LOCAL_STORAGE 沒定義
image.save(local_path, format="PNG")

...

s3_key = filename # ✅ 這個 OK
...
Bucket=S3_BUCKET_NAME, # ❌ S3_BUCKET_NAME 沒定義
...

return jsonify({
    "success": True,
    "local_url": local_url, # ❌ local_url 沒定義
```

```
...  
})
```

所以一定會出現 `NameError`：

- `LOCAL_STORAGE` 沒定義
- `S3_BUCKET_NAME` 沒定義
- `local_url` 沒定義
- 而且 Lambda 上也不需要再存本地檔案。

正確的作法：只存 S3，不存本地

1. 全域變數區先加上 `S3_BUCKET_NAME`

python

複製程式碼

```
load_dotenv()  
HuggingFace_Token = os.getenv("HUGGINGFACE_TOKEN")  
Suno_Token = os.getenv("SUNO_TOKEN")  
HuggingFace_URL = "https://router.huggingface.co/hf-inference/models/stabilityai/  
  
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME") # ✅ 從環境變數讀 bucket 名  
app = Flask(__name__)  
  
s3 = boto3.client("s3")
```

2. `generate_image` 改成「直接把 `response.content` 丟去 S3，再回傳下載網址」

python

複製程式碼

```
@app.route("/generate-image", methods=["POST"])  
def generate_image():  
    try:  
        data = request.json or {}  
        prompt = data.get("prompt", "沒有收到圖片prompt")  
        user_id = data.get("user_id", "unknown")  
  
        if not prompt:  
            return jsonify({"success": False, "error": "缺少 prompt"}), 400
```

```
headers = {"Authorization": f"Bearer {HuggingFace_Token}"}
payload = {"inputs": prompt}

# 呼叫 Hugging Face 產圖
response = requests.post(HuggingFace_URL, headers=headers, json=payload,

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤: {response.status_code}",
        "details": response.text
    }), 500

content_type = response.headers.get('Content-Type', '')

if 'application/json' in content_type:
    error_data = response.json()
    return jsonify({
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# (可選) 確認一下確實是圖片
try:
    Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

# ✅ 產生檔名 & S3 key
timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = f"images/{filename}" # 存在 bucket 的 images/ 底下
```

```
#  上傳到 S3
try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content,      # 用原始 bytes
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

#  產生一個 presigned URL (下載網址) 當作 local_url 回傳給前端
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={"Bucket": S3_BUCKET_NAME, "Key": s3_key},
        ExpiresIn=3600 # 有效 1 小時
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "s3_key": s3_key,
    "local_url": presigned_url, # 這樣前端不用改 key 名稱
    "reply": f"Flask 收到圖片prompt: {prompt}"
})

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500
```

3. 檔案最後要有 Lambda 入口

如果這支要部署在 Lambda，檔案最後記得有：

python

 複製程式碼

```
def lambda_handler(event, context):  
    return awsgl.response(app, event, context)
```

Handler 設成：

text

 複製程式碼

```
lambda_function.lambda_handler
```

(假設檔名叫 `lambda_function.py`)

4. 環境變數 & 權限檢查 (再提醒一次)

在你的那個環境變數畫面填：

- `HUGGINGFACE_TOKEN` = 你的 HF token
- `S3_BUCKET_NAME` = 你建立好的 S3 bucket 名字
- `AWS_REGION` (可選) = `us-east-1` (你現在區域)

Lambda 的執行角色 (LabRole) 要有：

- `s3:PutObject` / `s3:GetObject` 對 `arn:aws:s3:::你的bucket/*`

如果你想確認「你這個 bucket 名 + key 真的有東西」，你可以在 S3 介面看看有沒有 `images/xxxx.png`，或者之後貼一次新的錯誤訊息（例如 `AccessDenied` / `NoSuchBucket`），我就可以幫你對應到權限或名稱問題。

     ...

s3.put_object有把圖放進去嗎

已思考 17s >

短答案：只要 `s3.put_object` 沒拋錯、你的 bucket 名字跟權限都對，圖片就真的已經放進 S3 了。

但要「眼睛看到」，你可以做幾個小檢查。

1. `s3.put_object` 成功的判斷

你現在程式裡有這段：

python

 複製程式碼

```
try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content,
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500
```

如果 **進到** `except`，你就會在回傳 JSON 裡看到 **"上傳 S3 失敗: ..."**，那當然沒成功。

如果沒有進 `except`，就表示 S3 有回 200 OK，理論上物件已經在 bucket 裡了。

你也可以把回傳值印出來 debug 一下：

python

 複製程式碼

```
res = s3.put_object(
    Bucket=S3_BUCKET_NAME,
    Key=s3_key,
    Body=response.content,
    ContentType="image/png"
)
print("S3 PUT RESULT:", res)
```

在 CloudWatch Logs 裡可以看到 `ETag` 之類的資訊。

2. 去 S3 Console 看

1. 開 S3，在左上選跟 Lambda 一樣的 Region (你是 `us-east-1`)
2. 點你的 bucket (就是 `S3_BUCKET_NAME` 填的那個名字)
3. 看物件列表：

- 如果你用 `s3_key = f"images/{filename}"`
 - 會看到一個 `images/` 的「資料夾」
 - 點進去裡面應該有 `2025xxxx_xxxxxxxx.png` 那種檔名
- 如果你是 `s3_key = filename`
 - 檔案直接在 bucket 根目錄

注意：console 顯示的「資料夾」只是用 key 裡的 `/` 假裝出來的；實際上 S3 只有 key 一長串。

3. 也可以用 AWS CLI / CloudShell 檢查

在 CloudShell (或有權限的 CLI) 打：

```
bash
```

[複製程式碼](#)

```
aws s3 ls s3://你的bucket名稱/  
aws s3 ls s3://你的bucket名稱/images/
```

如果有看到剛剛生成的檔名，代表 `put_object` 確實有放進去。

4. 最容易踩雷幾點 (看你有沒有中)

1. S3_BUCKET_NAME 打錯 or 環境變數沒設

- 會直接丟 exception (通常是 `NoSuchBucket` 或 `Parameter validation failed`)，會被你的 `except` 接到。

2. Lambda 執行角色沒有 `s3:PutObject` 權限

- 會是 `AccessDenied`，一樣會被 `except` 接到。

3. 你在 S3 Console 選錯 Region

- 上方那個 Region 要跟 Lambda & bucket 一樣。

5. 建議加一行 log 幫你確認

在 `put_object` 後面加：

```
python
```

[複製程式碼](#)

```
print(f"Uploaded to S3 bucket={S3_BUCKET_NAME}, key={s3_key}")
```

之後每次跑 Lambda，在 CloudWatch Logs 就能清楚看到「傳到哪個 bucket / key」，再去 console 用同樣的路徑找就不會迷路。

你可以先跑一次成功的產圖請求，

然後：

1. 看回傳 JSON 有沒有 `"success": true`，沒有「上傳 S3 失敗」
2. 去 S3 Console → 找那個 bucket → 展開 `images/` 看看有沒有新檔案

如果還是看不到，直接把 Lambda 回傳 JSON 或錯誤訊息貼給我，我可以幫你判斷是「沒傳上去」還是「傳錯地方」。



Status: Failed

Test Event Name: test1

Response:

```
{
  "errorMessage": "Unable to import module
'lambda_function': No module named 'awsgi'",
  "errorType": "Runtime.ImportModuleError",
  "requestId": "",
  "stackTrace": []
}
```

The area below shows the last 4 KB of the execution log.

Function Logs:

[WARNING] 2025-11-25T07:43:30.235Z

LAMBDA_WARNING: Unhandled exception. The most likely cause is an issue in the function code. However, in rare cases, a Lambda runtime update can cause unexpected function behavior. For functions using managed runtimes, runtime updates can be triggered by a function change, or can be applied automatically. To determine if the runtime has been updated, check the runtime version in the INIT_START log entry. If this error correlates with a change in the runtime version,

you may be able to mitigate this error by temporarily rolling back to the previous runtime version. For more information, see

<https://docs.aws.amazon.com/lambda/latest/dg/runtimes-update.html>

[ERROR] Runtime.ImportModuleError: Unable to import module 'lambda_function': No module named 'awsgi'

Traceback (most recent call last):

INIT_REPORT Init Duration: 140.58 ms Phase: init

Status: error Error Type: Runtime.ImportModuleError

[WARNING] 2025-11-25T07:43:30.353Z

LAMBDA_WARNING: Unhandled exception. The most likely cause is an issue in the function code. However, in rare cases, a Lambda runtime update can cause unexpected function behavior. For functions using managed runtimes, runtime updates can be triggered by a function change, or can be applied automatically. To determine if the runtime has been updated, check the runtime version in the INIT_START log entry. If this error correlates with a change in the runtime version, you may be able to mitigate this error by temporarily rolling back to the previous runtime version. For more information, see

<https://docs.aws.amazon.com/lambda/latest/dg/runtimes-update.html>

[ERROR] Runtime.ImportModuleError: Unable to import module 'lambda_function': No module named 'awsgi'

Traceback (most recent call last):

INIT_REPORT Init Duration: 92.27 ms Phase: invoke

Status: error Error Type: Runtime.ImportModuleError

START RequestId: ebb69b15-a5df-4c2a-ade2-

7c0b09e032a4 Version: \$LATEST

END RequestId: ebb69b15-a5df-4c2a-ade2-

7c0b09e032a4

REPORT RequestId: ebb69b15-a5df-4c2a-ade2-

7c0b09e032a4 Duration: 101.66 ms Billed Duration:

102 ms Memory Size: 128 MB Max Memory Used: 46

MB Status: error Error Type:

Runtime.ImportModuleError

Request ID: ebb69b15-a5df-4c2a-ade2-7c0b09e032a4

已思考 15s >

很好，這個錯誤其實很單純：

`No module named 'awscli'` = 你的部署 zip 裡沒有 `awscli` 這個套件

也就是說：你已經在程式裡 `import awscli` 了，但在打包時沒有把 `awscli` 一起塞進去。

1 先確認 requirements.txt

在跟 `lambda_function.py` 同一層的檔案裡（通常是 `backend/src`），`requirements.txt` 裡一定要有這幾行（至少）：

txt

 複製程式碼

```
Flask
python-dotenv
requests
Pillow
awscli
```

有多的沒差，但 要確定有 `awscli` 這一行。

2 在「有 lambda_function.py 的那一層」裝套件

打開終端機 / PowerShell，到 有 `lambda_function.py` 的那個資料夾：

powershell

 複製程式碼

```
cd C:\Users\Neo Yeh\Desktop\雲端實務\FinalProject\backend\src
```

然後執行（不要用 `pip`，用 `python -m pip` 比較穩）：

powershell

 複製程式碼

```
python -m pip install -r requirements.txt -t .
```

執行完後，在這個資料夾底下應該會多出一堆資料夾，例如：

- flask
- awscli

- PIL
- requests
- dotenv (或 python_dotenv)

3 重新打包 zip (重點：要選內容壓縮)

在檔案總管開到 `backend\src` 這一層：

1. 按 `Ctrl + A` 全選所有 檔案 + 資料夾 (包含 `lambda_function.py` 、 `flask` 、 `awscli` ...)
2. 在「已選取的其中一個檔案」上按右鍵 → 送到 → 壓縮(zipped)資料夾
3. 產生的 zip 改名叫 `function.zip`

打開 `function.zip` 確認：

最上層要直接看到 `lambda_function.py` 、 `flask/` 、 `awscli/` ，
不要是 `src/lambda_function.py` 這種多一層資料夾。

4 回 Lambda 上傳新的 zip

1. Lambda → 你的 function → Code
2. 右上角 **Upload from** → **.zip file** → 選剛剛的 `function.zip`
3. 確認 Runtime 設的是 **Python**，Handler 是：

text

 複製程式碼

```
lambda_function.lambda_handler
```

4. 儲存後再按 Test。

如果這次還是 `ImportModuleError`，但換成別的 module (例如 `Flask`、`Pillow`)，就把新的錯誤訊息貼給我，我可以看是哪個套件還沒被打進 zip。

如果變成其他類型錯誤 (例如 S3 許可、`HuggingFace` 回傳錯誤)，一樣把 log 貼過來，我幫你看下一步要修哪裡。

     ...

↳ 在「有 `lambda_function.py` 的那一層」裝套件

這樣會重裝已有的套件嗎

不會把你電腦「全域」的套件搞壞，頂多只是 **在那個資料夾裡重新放一次一樣的套件**，是安全的 👍

說細一點：

- `python -m pip install -r requirements.txt -t .`
 - 只會安裝到目前這一層資料夾 (`-t .` 的意思是「安裝到這個資料夾」)
 - 不會去刪你原本 Python 環境裡的套件，也不會動到其他專案
- 如果那個資料夾裡 **已經有對應的套件版本**：
 - pip 可能會判斷「已滿足需求」就跳過
 - 或是 **再覆蓋一次同樣的檔案**，效果就是「重新複製一遍」，不會出事，只是多花幾秒鐘
- 在做 Lambda 打包時，**反覆跑這行是正常操作**
每次改 requirements 或懷疑少套件，就再跑一次，確保 `lambda_function.py` 同一層下面一定有：
 - `flask/`
 - `awscli/`
 - `PIL/`
 - `requests/`
 - `dotenv` 或 `python_dotenv` 之類的資料夾

所以你可以很安心地再跑一次那行指令，不用怕會「重裝壞掉」，頂多只是覆蓋到同版本而已。

接下來重點還是：裝完後 **重新壓 zip → 上傳 Lambda**，就能把 awscli、Flask 那些一起帶上去。

📄 🍴 🔖 ↗️ ↺ ...

Status: Failed

Test Event Name: test_1

Response:

```
{
  "errorMessage": "Unable to import module
'lambda_function': No module named 'awscli'",
  "errorType": "Runtime.ImportModuleError",
```

```
"requestId": "",  
"stackTrace": []  
}
```

The area below shows the last 4 KB of the execution log.

Function Logs:

[WARNING] 2025-11-25T08:02:01.893Z

LAMBDA_WARNING: Unhandled exception. The most likely cause is an issue in the function code. However, in rare cases, a Lambda runtime update can cause unexpected function behavior. For functions using managed runtimes, runtime updates can be triggered by a function change, or can be applied automatically. To determine if the runtime has been updated, check the runtime version in the INIT_START log entry. If this error correlates with a change in the runtime version, you may be able to mitigate this error by temporarily rolling back to the previous runtime version. For more information, see

<https://docs.aws.amazon.com/lambda/latest/dg/runtimes-update.html>

[ERROR] Runtime.ImportModuleError: Unable to import module 'lambda_function': No module named 'awsgi'

Traceback (most recent call last):

INIT_REPORT Init Duration: 139.42 ms Phase: init

Status: error Error Type: Runtime.ImportModuleError

[WARNING] 2025-11-25T08:02:02.010Z

LAMBDA_WARNING: Unhandled exception. The most likely cause is an issue in the function code. However, in rare cases, a Lambda runtime update can cause unexpected function behavior. For functions using managed runtimes, runtime updates can be triggered by a function change, or can be applied automatically. To determine if the runtime has been updated, check the runtime version in the INIT_START log entry. If this error correlates with a change in the runtime version, you may be able to mitigate this error by temporarily rolling back to the previous runtime version. For more information, see

<https://docs.aws.amazon.com/lambda/latest/dg/runti>

```
mes-update.html
[ERROR] Runtime.ImportModuleError: Unable to
import module 'lambda_function': No module named
'awscli'
Traceback (most recent call last):
INIT_REPORT Init Duration: 92.05 ms Phase: invoke
Status: error Error Type: Runtime.ImportModuleError
START RequestId: a44ae7b8-ed94-47e9-a368-
cc88ae710dc2 Version: $LATEST
END RequestId: a44ae7b8-ed94-47e9-a368-
cc88ae710dc2
REPORT RequestId: a44ae7b8-ed94-47e9-a368-
cc88ae710dc2 Duration: 103.39 ms Billed Duration:
104 ms Memory Size: 128 MB Max Memory Used: 46
MB Status: error Error Type:
Runtime.ImportModuleError

Request ID: a44ae7b8-ed94-47e9-a368-cc88ae710dc2
我存的結構是這樣：
同一層有package、app.py、lambda_function.py、
bot.py
所有套件都在package裡
```

已思考 29s >

好，現在問題超清楚了：

你的 `awscli` 的確有被裝進 `zip`，但被放在 `package/` 這個子資料夾裡，`Python` 預設不會去那個資料夾找，所以 `import awscli` 還是失敗。

`Lambda` 解壓 `zip` 之後，只會自動把 `zip` 最上層 放進 `sys.path`，不會自動幫你把 `package/` 也加進來。

你說現在結構是這樣（`zip` 打開大概長這樣）：

text

 複製程式碼

```
lambda_function.py
app.py
bot.py
package/
  awscli/...
  flask/...
```



```
requests/...  
...
```

但 Python import 的路徑只看這一層 (/var/task) :

```
text
```

[複製程式碼](#)

```
lambda_function.py  
app.py  
bot.py
```

所以它找不到 `awscli` 。

做法一 (推薦) : 讓套件跟 `lambda_function.py` 放「同一層」

最單純也是最接近官方做法 :

不要再用 `-t package` , 改用 `-t .` , 讓套件直接裝在目前資料夾根目錄。

1. 到有 `lambda_function.py` 的那一層

例如 :

```
powershell
```

[複製程式碼](#)

```
cd C:\Users\Neo Yeh\Desktop\雲端實務\FinalProject\backend\src
```

這層要看得得到 :

```
text
```

[複製程式碼](#)

```
lambda_function.py  
app.py  
bot.py  
requirements.txt
```

2. 重新把套件裝到「這一層」

```
powershell
```

[複製程式碼](#)

```
python -m pip install -r requirements.txt -t .
```

注意這裡是 `-t .` , 不是 `-t package`
跑完之後, 這一層會多出一堆資料夾 :

text

 複製程式碼

```
lambda_function.py
app.py
bot.py
awsgi/
flask/
PIL/
requests/
...
```

如果原本有 `package/` 夾，可以留著沒差，也可以刪掉，重點是 `awsgi` 那些要直接在根目錄。

3. 用「選內容」的方式重新壓 zip

在檔案總管打開 `backend\src`：

1. `Ctrl + A` 全選所有檔案 + 資料夾 (包含 `lambda_function.py` 、 `awsgi/` 、 `flask/`)
2. 在其中一個檔案上按右鍵 → 送到 → 壓縮(zipped)資料夾
3. 把 zip 改名成 `function.zip`

打開 `function.zip` 檢查最上層要長這樣：

text

 複製程式碼

```
lambda_function.py
app.py
bot.py
awsgi/
flask/
PIL/
requests/
...
```

不要再有一層 `src/` 或 `package/` 把它們包住。

4. 上傳這個 zip 到 Lambda

- Code → Upload from → .zip file → 選 `function.zip`
- Runtime settings → Handler = `lambda_function.lambda_handler`
- 存檔 → 再 Test

這樣 `import awsgi` 就會成功了。

做法二 (不改 zip 結構，程式裡手動把 `package/` 加進 `sys.path`)

如果你真的想保留「所有套件都放在 `package/`」，也可以在 `lambda_function.py` 最前面手動加路徑：

python

 複製程式碼

```
import os
import sys

# 把當前檔案同一層的 package/ 資料夾加進 sys.path
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
sys.path.append(os.path.join(CURRENT_DIR, "package"))

from flask import Flask, request, jsonify, send_from_directory
from dotenv import load_dotenv
import io, uuid, datetime
import sqlite3
import requests
from PIL import Image
import awsgi
import boto3
```

這樣 Python 在 import 的時候就會去 `package/` 裡找 `awsgi`。

但注意：

zip 裡結構至少要是：

text

 複製程式碼

```
lambda_function.py
package/
  awsgi/
  flask/
  ...
```

而不是再多一層：

text

 複製程式碼

```
src/
  lambda_function.py
  package/
```

```
awscli/  
...
```

推薦你現在怎麼做？

因為你已經卡在 `awscli` 好幾輪，我建議直接走「做法一」：

1. 在 `backend\src` 這層重裝套件到 `-t .`
2. 用全選 → 右鍵壓縮成 `function.zip`
3. 上傳覆蓋原本的 Code

如果這次再出錯，就會是下一個 module（例如 `Flask` / `Pillow`）或進到程式邏輯問題，錯誤訊息就會往下走，我再幫你解下一關。



Status: Failed

Test Event Name: test_1

Response:

```
{  
  "errorMessage": "module 'awscli' has no attribute  
'response'",  
  "errorType": "AttributeError",  
  "requestId": "ed74fd95-6c36-4e66-b68e-  
47b0c4566b05",  
  "stackTrace": [  
    " File \"/var/task/lambda_function.py\", line 4, in  
lambda_handler\n    return awscli.response(app, event,  
context)\n"  
  ]  
}
```

The area below shows the last 4 KB of the execution log.

Function Logs:

```
START RequestId: ed74fd95-6c36-4e66-b68e-  
47b0c4566b05 Version: $LATEST  
[ERROR] AttributeError: module 'awscli' has no attribute  
'response'
```

```
Traceback (most recent call last):
  File "/var/task/lambda_function.py", line 4, in
    lambda_handler
    return awsgi.response(app, event, context)
END RequestId: ed74fd95-6c36-4e66-b68e-47b0c4566b05
REPORT RequestId: ed74fd95-6c36-4e66-b68e-47b0c4566b05 Duration: 11.37 ms Billed Duration: 95 ms Memory Size: 128 MB Max Memory Used: 36 MB Init Duration: 83.06 ms

Request ID: ed74fd95-6c36-4e66-b68e-47b0c4566b05
```

已思考 47s >

這個錯誤其實是「裝錯套件」造成的，不是你寫的 `lambda_handler` 有問題 🤖

```
module 'awsgi' has no attribute 'response'
```

代表：現在確實有一個叫 `awsgi` 的模組被匯入到了，但那個模組裡 **沒有** `response` 這個函式。

原因幾乎可以確定是——

1 你在 requirements 裡裝的是 `awsgi`（錯的那個）

PyPI 上有兩個不同的東西：

- `aws-wsgi`：正確的這個，安裝後可以 `import awsgi`，而且裡面有 `awsgi.response(...)` PyPI +1
- `awsgi`：完全不同的舊套件，是一個 async WSGI server，裡面沒有 `response` 這個 function PyPI

你現在的情況就是安裝了後者，所以 `import awsgi` 成功，但裡面沒有 `response`，就噴這個錯。

✅ 解法：改成安裝 `aws-wsgi`，重新打包

第一步：改 requirements.txt

打開跟 `lambda_function.py` 同一層的 `requirements.txt`，把裡面跟 `awsgi` 有關的改掉：

txt

複製程式碼

```
Flask
python-dotenv
requests
Pillow
aws-wsgi      # ☒ 要寫這個，而不是 awsgi
```

如果原本有寫 `awsgi` 那一行，把它刪掉或改成 `aws-wsgi`。

第二步：重新安裝到「同一層資料夾」

在有 `lambda_function.py` 的資料夾打開 Terminal / PowerShell，例如：

```
powershell

cd C:\Users\Neo Yeh\Desktop\雲端實務\FinalProject\backend\src
python -m pip install -r requirements.txt -t .
```

 複製程式碼

跑完之後，在這一層目錄底下要看到一堆資料夾，例如：

- `awsgi/` (這次是正確的 `awsgi`)
- `flask/`
- `requests/`
- `PIL/`
- ...

第三步：重新打包 function.zip

一樣在 `backend\src` 這一層，用檔案總管：

1. `Ctrl + A` 全選所有檔案 + 資料夾 (含 `lambda_function.py`、`awsgi/` 等)
2. 右鍵 → 送到 → 壓縮 (zip) 資料夾
3. 把壓好的 zip 改名成 `function.zip`

打開 `function.zip` 檢查看一下：最上層要直接看到：

```
text

lambda_function.py
app.py
bot.py
awsgi/
```

 複製程式碼

```
flask/  
PIL/  
requests/  
...
```

第四步：上傳到 Lambda

1. Lambda → 你的 function → Code → **Upload from** → **.zip file** → 選剛剛的 `function.zip`
2. Runtime settings :
 - Handler = `lambda_function.lambda_handler`
3. 儲存後再按 Test

程式最後的 handler 應該長這樣：

python

複製程式碼

```
def lambda_handler(event, context):  
    return awsgi.response(app, event, context, base64_content_types={"image/png"})
```

這是官方範例的用法，可以支援回傳 PNG 圖片給 API Gateway。 PyPI +1

照這四步做完，`awsgi.response` 那個 `AttributeError` 就會消失。

接下來如果再出現新的錯誤（例如 S3 權限、HuggingFace 回傳錯誤、event 格式不對），把新的錯誤 log 再貼一次給我，我幫你看下一關要修什麼。

📄 👍 🗨️ ⬆️ ↺️ ...

我要怎麼刪除原本的awsgi

已思考 19s >

重點是：你不用把電腦裡「全域」的 `awsgi` 卸載，只要把「要打包給 Lambda 的那個資料夾裡」的舊 `awsgi` 刪掉就好，然後裝正確的 `aws-wsgi` 再重打包。

我分步做給你：

1. 先刪掉「專案資料夾裡」舊的 awsgi

到你要打包的那一層（有 `lambda_function.py` 的那層），例如：

powershell

 複製程式碼

```
cd C:\Users\Neo Yeh\Desktop\雲端實務\FinalProject\backend\src
```

這裡應該可以看到類似：

text

 複製程式碼

```
lambda_function.py
app.py
bot.py
requirements.txt
awsgi/
awsgi-XXXXX.dist-info/    (如果有)
flask/
PIL/
...
```

刪檔方式 1：用檔案總管（最直覺）

1. 用檔案總管打開 `backend\src`
2. 找到資料夾：
 - `awsgi`
 - 以及任何像 `awsgi-0.0.0.dist-info` 這種名稱的資料夾（有就一起刪掉）
3. 直接右鍵刪除

刪檔方式 2：用 PowerShell 刪

在 `backend\src` 這層：

powershell

 複製程式碼

```
Remove-Item -Recurse -Force awsgi
Remove-Item -Recurse -Force awsgi-*.dist-info # 有的話
```

2. 修改 requirements.txt：確定是 `aws-wsgi` 不是 `awsgi`

打開 `requirements.txt`，改成類似這樣（重點在最後一行）：

txt

 複製程式碼

```
Flask
python-dotenv
requests
Pillow
aws-wsgi
```

千萬不要再有一行寫 `awscli`，那是錯的那個套件名字。

3. 在同一層重新安裝「正確的」aws-wsgi

還是在 `backend\src` 這一層：

powershell

 複製程式碼

```
python -m pip install -r requirements.txt -t .
```

跑完之後，你會看到又多出一個 `awscli/` 資料夾，但這次是來自 `aws-wsgi`，裡面就會有我們要用的 `response` 函式。

4. 再次打包 & 上傳 Lambda

1. 在 `backend\src`：Ctrl + A 全選所有檔案 & 資料夾
2. 右鍵 → 送到 → 壓縮 (zip) 資料夾
3. 改名成 `function.zip`
4. 回 Lambda Code 頁面 → Upload from → .zip file → 選 `function.zip`
5. Handler 確認是：`lambda_function.lambda_handler`

程式最後一定要是這樣：

python

 複製程式碼

```
def lambda_handler(event, context):
    return awscli.response(app, event, context)
```

再按 Test，這次就不會再噴 `module 'awscli' has no attribute 'response'` 了。

如果再出現新的錯誤（例如 S3 AccessDenied 或 HuggingFace 相關），把新的 log 貼上來，我再幫你看下一步。



```
PIL          certifi-2025.11.12.dist-info  flask-
3.1.2.dist-info  jinja2-3.1.6.dist-info
requirement.txt  websockets
app.py          charset_normalizer          httptools
markupsafe      sample          websockets-
15.0.1.dist-info
aws_wsgi-0.2.7.dist-info  charset_normalizer-3.4.4.dist-
info  httptools-0.7.1.dist-info  markupsafe-3.0.3.dist-
info  spec          werkzeug
bin          click          idna
pillow-12.0.0.dist-info  test3.zip          werkzeug-
3.1.3.dist-info
blinker        click-8.3.1.dist-info  idna-
3.11.dist-info  pillow.libs          urllib3
blinker-1.9.0.dist-info  db.py
itsdangerous    python_dotenv-1.2.1.dist-info
urllib3-2.5.0.dist-info
bot.py          dotenv          itsdangerous-
2.2.0.dist-info  requests          uvloop
certifi        flask          jinja2
requests-2.32.5.dist-info  uvloop-0.22.1.dist-info
這些套件應該沒錯了吧
```

已思考 39s >

你現在這個資料夾裡的套件「快要對了」，但還差一點點，所以 Lambda 才還是找不到

`awsgi.response`。

先看你貼的清單（我簡化一下）：

text

複製程式碼

PIL

app.py

aws_wsgi-0.2.7.dist-info

```
bin
blinker
certifi
charset_normalizer
flask
...
python_dotenv-1.2.1.dist-info
requests
urllib3
...
requirement.txt
...
```

✓ 已經 OK 的部分

- flask / flask-3.1.2.dist-info ✓
- PIL / pillow-12.0.0.dist-info ✓
- requests / requests-2.32.5.dist-info ✓
- dotenv / python_dotenv-1.2.1.dist-info ✓
- aws_wsgi-0.2.7.dist-info ✓ (代表你有裝 aws-wsgi 這個套件)

✗ 但少了最關鍵的一個：awscli 目錄

照正常來說，裝完 aws-wsgi 之後，在同一層應該要有一個資料夾：

text

複製程式碼

```
awscli/
  __init__.py
  compat.py
  ...
```

你貼出來的清單**沒有** awscli 這個資料夾，所以 Python 雖然看到有 aws_wsgi-0.2.7.dist-info，

但實際可 import 的 module awscli 沒被放進來，Lambda 才會說：

```
module 'awscli' has no attribute 'response'
```

(實際上是 import 到了某個空殼 / 舊版的 awscli，或者完全沒有 awscli package)

幫你整理下一步要做什麼

1 確定 requirements.txt 寫的是 aws-wsgi

打開你那個 requirement.txt (沒有 s 那個) , 內容至少要像這樣 :

txt

 複製程式碼

```
Flask
python-dotenv
requests
Pillow
aws-wsgi
```

如果裡面有一行寫成 awsgi (沒有 dash) , 把那行改掉 , 只保留 aws-wsgi 那一行。

2 在同一層「強制重裝 aws-wsgi」一次

在有 lambda_function.py / app.py 的那一層開終端機 (你現在看到那堆套件的那層) :

powershell

 複製程式碼

```
python -m pip install aws-wsgi -t . --force-reinstall --no-cache-dir
```

這行的效果是 :

- 只安裝 aws-wsgi 一個套件
- 強制重新解壓 (就算之前裝過也會蓋掉)
- 裝到「目前資料夾」這一層

執行完之後 , 再看一次這個資料夾的內容 , 你應該會多看到一個 :

text

 複製程式碼

```
awscli/
aws_wsgi-0.2.7.dist-info
...
```

只要這個 awscli/ 出現 , 就對了。

3 確定 lambda_function.py 頭跟尾長這樣

檔案開頭要有 :

python

 複製程式碼

```
from flask import Flask, request, jsonify, send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import sqlite3
import requests
from PIL import Image
import awsgi
import boto3
```

檔案最後要有：

python

 複製程式碼

```
def lambda_handler(event, context):
    return awsgi.response(app, event, context)
```

4 重新壓 zip，上傳 Lambda

1. 在這一層按 `Ctrl + A` (全選所有檔案 + 資料夾，包括 `lambda_function.py`、`awsgi/`、`flask/` 等)
2. 右鍵 → 送到 → 壓縮 (zipped) 資料夾
3. 改名成 `function.zip`
4. 去 Lambda → Code → Upload from → .zip file → 選 `function.zip`
5. Runtime settings → Handler：`lambda_function.lambda_handler`
6. 存檔 → Test 一次

小總結

- 你現在 `Flask / Pillow / requests / dotenv` 都 OK 了
- 還差的就是：要看到一個 `awsgi/` 資料夾出現在同一層
- 照上面的 `python -m pip install aws-wsgi -t . --force-reinstall --no-cache-dir` 跑一次，再重壓 zip，上 Lambda 就能跨過這關

你跑完第 2 步之後，如果願意再貼一次目錄清單 (看起來有沒有 `awsgi/`)，我可以幫你再確認一次結構沒問題，這樣你上傳後比較不會又踩到奇怪的坑。

     ...

Status: Failed

Test Event Name: test_1

Response:

```
{
  "errorMessage": "name 'app' is not defined",
  "errorType": "NameError",
  "requestId": "8a7155c8-21bf-409a-8072-69a549fb5df2",
  "stackTrace": [
    " File \"/var/task/lambda_function.py\", line 4, in\nlambda_handler\n  return awscli.response(app, event,\ncontext)\n"
  ]
}
```

The area below shows the last 4 KB of the execution log.

Function Logs:

START RequestId: 8a7155c8-21bf-409a-8072-69a549fb5df2 Version: \$LATEST

[ERROR] NameError: name 'app' is not defined

Traceback (most recent call last):

File \"/var/task/lambda_function.py\", line 4, in
lambda_handler

return awscli.response(app, event, context)

END RequestId: 8a7155c8-21bf-409a-8072-69a549fb5df2

REPORT RequestId: 8a7155c8-21bf-409a-8072-69a549fb5df2 Duration: 2.64 ms Billed Duration: 100 ms Memory Size: 128 MB Max Memory Used: 38 MB Init Duration: 96.88 ms

Request ID: 8a7155c8-21bf-409a-8072-69a549fb5df2

已思考 31s >

好～現在這個錯誤就真的很單純了：

```
NameError: name 'app' is not defined
```

= Lambda 在執行

```
awscli.response(app, event, context)
```

的時候，找不到叫 `app` 的全域變數。

也就是說：你現在的 `lambda_function.py` 裡根本沒有定義 `app = Flask(__name__)`，或是它在別的檔案（`app.py`）裡，但沒 `import` 進來。

你說同一層有：`package/`、`app.py`、`lambda_function.py`、`bot.py`

很有可能是：Flask 的程式都在 `app.py`，Lambda handler 在 `lambda_function.py`。

那我們就按這個情況來修，給你最簡單、乾淨的作法 🙌

做法 A（推薦）：app 留在 `app.py`，`lambda_function` 只負責接到 Lambda

1 確認 `app.py` 長這樣（重點是全域有一個 `app = Flask(__name__)`）

簡化示意（你可以保留原本的 `routes` / `generate-image` 等邏輯）：

python

複製程式碼

```
from flask import Flask, request, jsonify, send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import sqlite3
import requests
from PIL import Image
import boto3

load_dotenv()
HuggingFace_Token = os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
HuggingFace_URL = "https://router.huggingface.co/hf-inference/models/stabilityai/"
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")

app = Flask(__name__) # ✅ 這個一定要在檔案最上層

s3 = boto3.client("s3")
```

```
# 這裡放你的 /generate-image、/generate-audio、/history 這些 @app.route(...)  
# ...
```

重點只有一個：在 `app.py` 的全域層級有一個 `app = Flask(__name__)`，不要放在 `if name == 'main'` 裡面。

2 把 `lambda_function.py` 改成只有「import app + handler」

`lambda_function.py` 裡可以簡單寫成這樣：

python

複製程式碼

```
import awsgi  
from app import app # ✅ 從 app.py 匯入那個 Flask 實體  
  
def lambda_handler(event, context):  
    return awsgi.response(app, event, context)
```

就這樣，超短。

注意：這裡的 `from app import app` 指的是「檔名叫 `app.py` 裡面的變數 `app`」，所以檔名一定要叫 `app.py`，而且裡面要有 `app = Flask(__name__)`。

3 再重新打包 zip + 上傳

確認資料夾長這樣（在你要打包的那一層）：

text

複製程式碼

```
lambda_function.py  
app.py  
bot.py          (可有可無)  
flask/  
PIL/  
requests/  
awsgi/  
...
```

然後：

1. `Ctrl + A` 全選所有檔案 + 資料夾
2. 右鍵 → 送到 → 壓縮 (zip) 資料夾
3. 改名成 `function.zip`
4. 回 Lambda → Code → Upload from → .zip file → 選這個 `function.zip`
5. Runtime settings → Handler 填：`lambda_function.lambda_handler`
6. Save → 再 Test 一次

這樣 `lambda_handler` 執行時就會先 import 到 `app.py` 裡的 `app`，`NameError` 就會消失。

做法 B：所有程式都寫在 `lambda_function.py` (不另外用 `app.py`)

如果你懶得分檔案，也可以把之前那支大 Flask 程式整份貼進 `lambda_function.py`，並確保：

python

 複製程式碼

```
app = Flask(__name__)  
# ...  
def lambda_handler(event, context):  
    return awsgi.response(app, event, context)
```

而且 handler 一定寫在 `app` 定義「後面」。

如果你願意，把現在的 `lambda_function.py` 和 `app.py` 內容貼出來（或說你選 A 還是 B），我可以幫你把兩支檔案改成「可以直接複製貼上」的最終版，這樣你只要打包上傳就行。

     ...

Status: Failed

Test Event Name: test_1

Response:

```
{  
  "errorMessage": "httpMethod",  
  "errorType": "KeyError",  
  "requestId": "7d4fc6de-9a7e-4d54-8b32-f51e83ebfc02",  
}
```

```

"stackTrace": [
  " File \"/var/task/lambda_function.py\", line 5, in
lambda_handler\n  return awscli.response(app, event,
context)\n",
  " File \"/var/task/awsgcli/__init__.py\", line 172, in
response\n  output = app(envron(event, context),
sr)\n",
  " File \"/var/task/awsgcli/__init__.py\", line 121, in
envron\n  'REQUEST_METHOD':
event['httpMethod'],\n"
]
}

```

The area below shows the last 4 KB of the execution log.

Function Logs:

START RequestId: 7d4fc6de-9a7e-4d54-8b32-f51e83ebfc02 Version: \$LATEST

[ERROR] KeyError: 'httpMethod'

Traceback (most recent call last):

File "/var/task/lambda_function.py", line 5, in
lambda_handler

return awscli.response(app, event, context)

File "/var/task/awsgcli/__init__.py", line 172, in response
output = app(envron(event, context), sr)

File "/var/task/awsgcli/__init__.py", line 121, in envron
'REQUEST_METHOD': event['httpMethod'],

END RequestId: 7d4fc6de-9a7e-4d54-8b32-f51e83ebfc02

REPORT RequestId: 7d4fc6de-9a7e-4d54-8b32-f51e83ebfc02 Duration: 36.58 ms Billed Duration: 1337 ms Memory Size: 128 MB Max Memory Used: 104 MB Init Duration: 1300.16 ms

Request ID: 7d4fc6de-9a7e-4d54-8b32-f51e83ebfc02

```

lambda_function.py
import awscli
from app import app

```

```

def lambda_handler(event, context):
    return awscli.response(app, event, context)

```

```
app.py
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import sqlite3
import requests
from PIL import Image
import awsgi
import boto3

# from db import init_db, insert_record
# init_db()
load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
app = Flask(__name__)

s3 = boto3.client("s3") # 建立 s3 client

# 本地儲存目錄
# LOCAL_STORAGE = './generated_files'
# os.makedirs(f'{LOCAL_STORAGE}/images',
exist_ok=True)
# os.makedirs(f'{LOCAL_STORAGE}/audios',
exist_ok=True)

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少
```

```
prompt"}), 400

headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
payload = {"inputs": prompt}

response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

content_type = response.headers.get('Content-
Type', '')

if 'application/json' in content_type:
    error_data = response.json()
    return jsonify({
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# 嘗試解析圖片
try:
    image =
Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500
```

```
# 儲存到本地
# timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
# filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
# local_path = f"
{LOCAL_STORAGE}/images/{filename}"
# image.save(local_path, format="PNG")

# 本地 URL
# local_url =
f"http://localhost:5000/generated_files/images/{filename}"

# 存到 S3
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = filename

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content, # 直接用原始 bytes
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

# 寫入 SQLite
# insert_record(user_id, prompt, local_url)

return jsonify({
    "success": True,
    # "local_url": local_url,
```

```

        "reply": f"Flask 收到圖片prompt: {prompt}"
    })

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

"""

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片prompt")

        filename = "ta.txt"
        # 模擬生成圖片，先回傳本地 txt URL
        # local_url =
        f"http://localhost:5000/generated_files/test.txt"
        local_url =
        f"http://localhost:5000/generated_files/audios/{filename}"

        return jsonify({
            "success": True,
            "local_url": local_url,
            "reply": f"Flask 收到音樂prompt: {prompt}"
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500
# 呼叫索取生成結果(圖片/音樂)
@app.route('/generated_files/<file_type>/<filename>',
methods=['GET'])
def serve_file(file_type, filename):

    # file_type: 'images' 或 'audios'
    # filename: 檔案名稱

    # 驗證子資料夾
    if file_type not in ["images", "audios"]:
        return jsonify({"success": False, "error": "不支援的檔案類型"}), 400

```

```
directory = os.path.join(LOCAL_STORAGE, file_type)
return send_from_directory(directory, filename)

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
        query 取得 user_id
        conn = sqlite3.connect("bot_records.db")
        c = conn.cursor()

        if user_id:
            c.execute("SELECT id, user_id, timestamp,
prompt, file_url FROM records WHERE user_id=?
ORDER BY id DESC", (user_id,))
        else:
            c.execute("SELECT id, user_id, timestamp,
prompt, file_url FROM records ORDER BY id DESC")

        rows = c.fetchall()
        conn.close()

        return jsonify({
            "success": True,
            "records": [
                {"id": r[0], "user_id": r[1], "timestamp": r[2],
"prompt": r[3], "file_url": r[4]}
                for r in rows
            ]
        })
    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500
"""

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

很好，這次錯誤已經不是套件問題了，而是「測試事件(event) 的格式不對」。

```
KeyError: 'httpMethod'
```

= awscli 在把 Lambda event 轉成 Flask 需要的 WSGI 請求時，
需要 `event["httpMethod"]`，但你的 event 裡根本沒有這個欄位。

awscli 預期你丟進來的是「API Gateway (REST API) proxy event」，
但你現在在 Lambda console 用的 Test event 是隨便一個一般 JSON (沒有
httpMethod)。

一句話結論

現在程式本身 OK，

你要改的是「測試事件的格式」，要長得像 API Gateway 的 HTTP 請求。

做法 1：在 Lambda Console 用「API Gateway Proxy」範本測試

1. 到 Lambda 的 Test 頁面
2. 在「Test event」那裡按「Edit」或「Create new test event」
3. 在 Template 下拉裡選一個像：
 - API Gateway AWS Proxy或名稱中有「proxy」的 API Gateway 範本
4. 它會自動幫你生一個長得像這樣的 JSON (我幫你改成適合 `/generate-image` 的版本)：

json

複製程式碼

```
{
  "resource": "/generate-image",
  "path": "/generate-image",
  "httpMethod": "POST",
  "headers": {
    "Content-Type": "application/json"
  },
  "queryStringParameters": null,
  "pathParameters": null,
  "stageVariables": null,
  "requestContext": {
    "resourcePath": "/generate-image",
    "httpMethod": "POST",
    "path": "/prod/generate-image"
  },
}
```



```
"body": "{\"prompt\": \"一隻可愛的貓咪\", \"user_id\": \"test-user\"}\",  
"isBase64Encoded": false  
}
```

5. 存成 `test_generate_image` 之類的名字

6. 按 **Test**

只要 event 裡有 `httpMethod`、`path` 等欄位，

`awsgi.response(app, event, context)` 就能正常把它轉給你的 Flask

`@app.route("/generate-image", methods=["POST"])`。

做法 2：之後接上真正的 API Gateway 再測

你現在是「在 Lambda console 裡人為按 Test」，

正式情境其實會是：

1. 建一個 **API Gateway (REST API)**
2. 設成 **Lambda Proxy integration**，指到這個 Lambda
3. 用 Postman / 前端 `POST https://xxxx.execute-api.../prod/generate-image`

那時候 API Gateway 自己會幫你組好 `event` (含 `httpMethod`、`path`、`headers`、`body`)，

你的 `lambda_handler → awsgi.response(app, event, context)` 就會自動正常運作。

小檢查：現在程式架構 OK 嗎？

- `app.py` 裡有：

```
python  
  
app = Flask(__name__)  
@app.route("/generate-image", methods=["POST"])  
def generate_image():  
    ...
```

 複製程式碼

- `lambda_function.py` 非常短：

```
python  
  
import awsgi  
from app import app
```

 複製程式碼

```
def lambda_handler(event, context):  
    return awscli.response(app, event, context)
```

- zip 裡最上層有：`lambda_function.py`, `app.py`, `awscli/`, `flask/`, `requests/`, `PIL/`
...

這樣就對了，現在只差「用對的 event 來測」。

你可以先照上面「做法 1」把 Test event 換成我那個 JSON，再按一次 Test；
如果這次進到你的 `/generate-image` 裡卻出現 HuggingFace 或 S3 的錯誤，把新的 log 貼出來，我再幫你把那一段調好。



Status: Succeeded

Test Event Name: test_2

Response:

```
{  
  "statusCode": "500",  
  "headers": {  
    "Content-Type": "application/json",  
    "Content-Length": "121"  
  },  
  "isBase64Encoded": false,  
  "body": "{\\\"details\\\":\\\"{\\\"error\\\":\\\"Invalid  
username or password.\\\"}\\\"}\\\",\\\"error\\\":\\\"Hugging Face  
API \\u932f\\u8aa4: 401\\\",\\\"success\\\":false}\\n\"  
}"
```

The area below shows the last 4 KB of the execution log.

Function Logs:

```
START RequestId: 853c5095-5fe7-48eb-b2ca-  
15b510f91806 Version: $LATEST  
END RequestId: 853c5095-5fe7-48eb-b2ca-  
15b510f91806  
REPORT RequestId: 853c5095-5fe7-48eb-b2ca-  
15b510f91806 Duration: 382.93 ms Billed Duration:
```

383 ms Memory Size: 128 MB Max Memory Used: 104 MB

Request ID: 853c5095-5fe7-48eb-b2ca-15b510f91806

現在這個結果代表：

👉 Lambda + Flask + awscli + S3 都正常在跑了

錯的是 呼叫 Hugging Face API 回傳 401：「Invalid username or password」，也就是「Token 無效或權限不對」。

從 body 解出來是：

json

複製程式碼

```
{
  "success": false,
  "error": "Hugging Face API 錯誤: 401",
  "details": "{\"error\":\"Invalid username or password.\"}"
}
```

所以現在要修的是「HUGGINGFACE_TOKEN」，不是 Lambda 設定本身。

你現在要做的事（修 Hugging Face Token）

1. 確認 Lambda 的環境變數 HUGGINGFACE_TOKEN

到 Lambda → 你的 function → Configuration → Environment variables，看：

- Key：HUGGINGFACE_TOKEN
- Value：是不是你在 Hugging Face 網站生成的 存取 Token (Access Token)

常見錯誤：

- 貼成「使用者帳號密碼」（No ❌）
- 貼成「整串字串含引號」，像 "hf_abcdef..."（應該是沒有引號）
- 多了空白： hf_abcdef... 或 hf_abcdef...

正確的 Value 範例（大致長這樣）：

text

複製程式碼

```
hf_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

2. 如果不確定 token 對不對，建議重新生一個

到 Hugging Face 網站 (你平常登入的帳號)：

1. 右上角頭像 → Settings (設定)
2. 左邊選 **Access Tokens**
3. 點 **New token**
4. 建議：
 - Name 隨便 (例如 `lambda-finalproject`)
 - Role：選 **Read** 或 **Inference** (有讀取/推論權限)
5. 建立後會看到一串 `hf_` 開頭的字串
只會顯示一次，務必馬上複製起來

然後回 Lambda：

- 把 `HUGGINGFACE_TOKEN` 的 Value 換成這一串
- 儲存 (Save)

3. 確認 URL 是可以用你這種 token 的

你現在的程式用的是：

```
python
```

[複製程式碼](#)

```
HuggingFace_URL = "https://router.huggingface.co/hf-inference/models/stabilityai/
```

這是 Hugging Face 的 Router / Inference API，用 Bearer Token 是 OK 的，但要滿足幾點：

- 模型 `stabilityai/stable-diffusion-xl-base-1.0` 不在你帳號的禁用/收費限制內
- 你的帳號有權限用 Inference API (某些情況要綁信用卡或有配額)

如果你在本機 (用同一個 token) 打這個也 401，就代表你 token 本身有問題或帳號權限不足。

你可以在本機簡單測一下 (用同一個 token)：

```
python
```

[複製程式碼](#)

```
import requests
```

```
url = "https://router.huggingface.co/hf-inference/models/stabilityai/stable-diffu
```

```
headers = {"Authorization": "Bearer hf_你的token"}
payload = {"inputs": "a cat"}

r = requests.post(url, headers=headers, json=payload)
print(r.status_code)
print(r.text[:500])
```

- 如果本機也 401 → token / 帳號問題
- 本機 200 OK · Lambda 401 → Lambda 的環境變數沒設好 (例如留到舊值、貼錯字串)

4. 快速 Debug : 印出有沒有拿到 Token (不要印真正內容)

你也可以在 Lambda 的程式裡暫時加一行：

python

 複製程式碼

```
print("HUGGINGFACE_TOKEN exists:", HuggingFace_Token is not None)
```

這樣在 CloudWatch Logs 就會看到是不是至少有讀到環境變數：

- `True` → 有讀到 (但可能內容錯)
- `False` → 代表環境變數 Key 名錯 / 沒儲存成功

(提醒：絕對不要 `print(HuggingFace_Token)`，避免 key 被 log 出去。)

5. 現在 Lambda 端「功能流程」已經 OK 了

你剛剛：

- awsgi 問題解決 
- app import 問題解決 
- event 格式改成 API Gateway proxy 
- Lambda 成功執行，並正確回傳 500 + Hugging Face 的錯誤內容 

所以現在只剩：

把 HUGGINGFACE_TOKEN 換成正確可用的 API token

你可以先照上面步驟重新產一個 Hugging Face token → 更新 Lambda 環境變數 → 再跑一次 Test。

如果這次 statusCode 變成 200 / 回傳 `success: true`，就代表整條鏈 (Lambda + Flask + HF + S3) 全部打通了。

如果還是 401，把你現在的 HUGGINGFACE_TOKEN 設定方式描述給我 (不用貼真實 token)，我可以幫你看是不是哪裡多空白或 Key 打錯。



這個error是在程式的哪一段

是這段 📌

在你的 `app.py` 裡 `generate_image()` 這一段程式碼：

python

複製程式碼

```
headers = {"Authorization": f"Bearer {HuggingFace_Token}"}
payload = {"inputs": prompt}

response = requests.post(HuggingFace_URL, headers=headers, json=payload, timeout=

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤: {response.status_code}",
        "details": response.text
    }), 500
```

對應關係說明

- 你看到的 Lambda 回傳：

json

複製程式碼

```
{
  "statusCode": "500",
```

```
"body": "{\\"details\\":\\"{\\\\"error\\\\":\\\\"Invalid username or password.\\\\"}"}"
```

- 就是這裡的 `if response.status_code != 200:` 被觸發，因為 Hugging Face 回 401 :
 - `error` 欄位 → `f"Hugging Face API 錯誤: {response.status_code}"`
 - `details` 欄位 → `response.text` (裡面是 `{"error":"Invalid username or password."}`)
 - HTTP 狀態碼 500 → return 後面的 `, 500`

所以：

- 程式是哪一段出錯？
 - 👉 `generate_image()` 裡「檢查 Hugging Face 回應碼」的那段 `if response.status_code != 200:`。
- 真正的根本原因？
 - 👉 Hugging Face 那邊認為你的 Token 不對 (或沒權限)，回 401，你的程式照設計把它包成 `success: false` + 500 丟回來。

📄 🍌 🗨️ ↗️ ↻️ ...

Status: Succeeded
Test Event Name: test_2

Response:

```
{
  "statusCode": "500",
  "headers": {
    "Content-Type": "application/json",
    "Content-Length": "153"
  },
  "isBase64Encoded": false,
  "body": "{\\"details\\":\\"{\\\\"error\\\\":\\\\"Invalid username or password.\\\\"}\\",\\"error\\":\\"Hugging Face API \\\u932f\\u8aa4: 401\\",\\"location\\":\\"status_code != 200\\",\\"success\\":false}\\n"
}
```

The area below shows the last 4 KB of the execution log.

Function Logs:

```
START RequestId: 96aa467e-dab5-412a-82e9-3e93a2ed2c75 Version: $LATEST
END RequestId: 96aa467e-dab5-412a-82e9-3e93a2ed2c75
REPORT RequestId: 96aa467e-dab5-412a-82e9-3e93a2ed2c75 Duration: 319.63 ms Billed Duration: 1496 ms Memory Size: 128 MB Max Memory Used: 103 MB Init Duration: 1175.51 ms
```

Request ID: 96aa467e-dab5-412a-82e9-3e93a2ed2c75
A function update is still in progress so the invocation went to the previously deployed code and configuration.
回傳不要有\

已思考 24s >

你現在看到的那些 `\` 不是程式多加了一層斜線在傳給前端，而是：

Lambda Console 在顯示 `body` 這個欄位（它本身是一個 JSON 字串）時，又再把裡面的 JSON 用字串方式顯示，所以才會看到很多 `\`。

實際 HTTP 回傳長這樣（給前端看的原始 body）：

json

 複製程式碼

```
{
  "success": false,
  "error": "Hugging Face API 錯誤: 401",
  "details": "{\"error\":\"Invalid username or password.\"}",
  "location": "status_code != 200"
}
```

這裡 `details` 本身是一個「字串」，裡面又剛好是 JSON → 所以必須 `escape`，才會有一堆 `\`。

如果你想要回傳的 JSON 裡不要有這些反斜線，做法是：

👉 把 `details` 改成真正的「物件」，而不是「字串」。

要改的程式段落

你現在 `generate_image` 裡大概是這樣：

python

 複製程式碼

```
if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤: {response.status_code}",
        "details": response.text,    # 👉 這裡是字串
        "location": "status_code != 200"
    }), 500
```

把這段改成「優先把 Hugging Face 的錯誤當 JSON 物件丟回去」：

python

 複製程式碼

```
import json

...

if response.status_code != 200:
    # 優先嘗試把 HF 回傳當 JSON 解析
    try:
        details = response.json()    # 變成 dict，而不是純文字
    except Exception:
        details = {"raw": response.text}    # 解析失敗就包成一個欄位

    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤: {response.status_code}",
        "details": details,          # 👉 這裡變成物件 / dict
        "location": "status_code != 200"
    }), 500
```

這樣回傳給前端會變成：

json

 複製程式碼

```
{
  "success": false,
  "error": "Hugging Face API 錯誤: 401",
  "details": {
    "error": "Invalid username or password."
  },
}
```

```
"location": "status_code != 200"
}
```

⚠ 在 Lambda Console 的「Response」區塊裡，因為它又把整個 body 當成一個字串來顯示，

還是會看到一些 escape，但真正前端收到的 HTTP body 就是不帶 `\` 的正常 JSON，前端 `JSON.parse(body)` 之後 `details.error` 會是正常字串。

如果你願意，把你現在 `generate_image()` 裡完整的 `return` 那段丟給我，我可以直接幫你改成「錯誤 / 成功都回同一種乾淨 JSON 格式」的版本，直接貼回去用就行。

📄 🍴 🔗 ↗️ ↺ ⋮

401意思

401 在這邊指的是 HTTP 狀態碼 401 – Unauthorized (未授權)。

用人話講就是：

「伺服器知道你想做什麼，但你提供的身分驗證 (帳號/密碼/Token) 是錯的或無效，所以拒絕幫你做。」

在你現在這個情況裡：

- 你請求 Hugging Face 的 API
- 帶的 header 是：

http

📄 複製程式碼

Authorization: Bearer HUGGINGFACE_TOKEN

- Hugging Face 回：

json

📄 複製程式碼

```
{"error": "Invalid username or password."}
```

- HTTP 狀態碼 = 401

所以超精準翻譯就是：

你這個 `HUGGINGFACE_TOKEN` 對 Hugging Face 來說是「無效憑證」
→ 不是正確的 Access Token / 權限不夠 / 貼錯 / 少東西
→ 所以回你 401 Unauthorized

順便補一個很常混的：

- **401**：沒通過身分驗證 (token/密碼錯、沒帶 Authorization)
- **403**：有身分，但「沒有權限」做這件事 (例如沒有用這個模型的權限)

你現在是 **401** → 去檢查這幾件事：

1. Lambda 環境變數裡 `HUGGINGFACE_TOKEN`
 - 是不是一整串 `hf_XXXXXXX`
 - 有沒有多空白、少字、貼錯帳號的 token
2. 用同一個 token 在你本機跑一個簡單 `requests.post` 測試
 - 本機也 401 → token/帳號本身有問題
 - 本機 200 OK、Lambda 401 → Lambda 那邊環境變數設錯

如果你願意，可以跟我說：

- 你在 Hugging Face 那邊「token 是怎麼生的」(選什麼 scope)
我可以一步一步跟你對照，確認你生的是可以用來 call 這個 model API 的那種 token。

📄 🍌 🗨 📶 ↻ ...

我要讓本地的discordbot.py觸發aws lambda(也就是剛剛的lambdafunction)，再啟動app.py

已思考 35s >

你現在其實已經做到這樣：

- Lambda 裡：`lambda_function.py` 用 `awsgi.response(app, event, context)` 去呼叫 `app.py` 裡的 Flask `app` ✓
- Test event 丟進去，可以跑到 `/generate-image`，只是 HuggingFace 401 (token 問題) ✓

接下來你要的是：

「本地的 `discordbot.py` 去觸發這個 Lambda (也就是 Flask 的 `/generate-image`) 。」

關鍵觀念：

在 Lambda 裡不用「啟動 `app.py`」當伺服器，每次 Lambda 被觸發時，`awscli` 會把那次的 HTTP 事件丟進 Flask `app`，就等於「幫你打了一次 Flask route」。

所以架構會變成：

Discord Bot → 呼叫一個 HTTP URL (API Gateway) → API Gateway 觸發 Lambda → Lambda 把 event 丟進 Flask `app` → 跑 `/generate-image` → 回傳結果 → Discord 回覆訊息。

第一步：幫 Lambda 接一個 API Gateway

因為你本地沒 AWS Access Key (voclabs 不給 IAM User)，

最簡單安全的方法是把 Lambda 用 API Gateway 變成一個 HTTP API，Discord bot 用 `requests` 去打這個網址就好。

建議你用 **API Gateway 的 REST API + Lambda Proxy integration** (`awscli` 預設吃這種 event)。

1. 建 REST API

1. 進 AWS Console → **API Gateway**
2. 點左邊 **APIs** → 上面按 **Create API**
3. 選：
 - **REST API** (不是 HTTP API)
 - **Build**
4. 填：
 - API name：例如 `finalproject-api`
 - Endpoint type：Regional
 - 其他先預設 → **Create API**

2. 建 resource + method

1. 左邊樹狀選單選你的 API → 展開 `/`
2. 按 **Actions** → **Create Resource**
 - Resource Name：`generate-image`
 - Resource Path：`/generate-image`
 - **Create Resource**
3. 選剛建立的 `/generate-image` → **Actions** → **Create Method**
 - 下拉選擇 `POST` → ✓

4. 在 POST 設定頁：

- Integration type : **Lambda Function**
- ☒ 勾 **Use Lambda Proxy integration**
- Lambda Region : 選你 Lambda 的 region
- Lambda Function : 選你的那個 `lambda_function`
- Save (如果跳出權限視窗，按 OK 讓它自動加 invoke 權限)

3. Deploy API

1. Actions → Deploy API

- Deployment stage : New Stage
- Stage name : `prod`

2. 部署後會看到一個 **Invoke URL**，例如：

text

 複製程式碼

```
https://abcd1234.execute-api.us-east-1.amazonaws.com/prod
```

那你的 `/generate-image` 完整 URL 就是：

text

 複製程式碼

```
https://abcd1234.execute-api.us-east-1.amazonaws.com/prod/generate-image
```

這個就是等一下 Discord bot 要打的 endpoint。

第二步：Discord Bot 端呼叫 Lambda (HTTP POST)

假設你用的是 `discord.py`，下面是一個最簡單版本（同步 requests，方便看）：

python

 複製程式碼

```
# discordbot.py
import os
import requests
import discord
from discord.ext import commands

DISCORD_TOKEN = os.getenv("DISCORD_TOKEN")

# 把這個換成剛剛 API Gateway 的 URL
LAMBDA_URL = "https://abcd1234.execute-api.us-east-1.amazonaws.com/prod/generate-
```

```

intents = discord.Intents.default()
bot = commands.Bot(command_prefix="!", intents=intents)

@bot.command(name="genimg")
async def genimg(ctx, *, prompt: str):
    await ctx.trigger_typing()

    payload = {
        "prompt": prompt,
        "user_id": str(ctx.author.id)
    }

    try:
        resp = requests.post(LAMBDA_URL, json=payload, timeout=90)
        data = resp.json()

        if not data.get("success"):
            await ctx.reply(f"生成失敗：{data.get('error')}", details={data.get('details')})
            return

        # 依照你 generate_image 回傳的欄位調整
        # 如果你之後改成回傳 download_url (S3 presigned URL) :
        image_url = data.get("download_url") or data.get("local_url")

        if image_url:
            await ctx.reply(f"生成完成：{image_url}")
        else:
            await ctx.reply("生成成功但沒有收到圖片連結 QQ")

    except Exception as e:
        await ctx.reply(f"呼叫 Lambda 失敗：{e}")

bot.run(DISCORD_TOKEN)

```

之後你把 Hugging Face token 問題修好，

這個指令 `!genimg 一隻可愛的貓`

就會：Discord → API GW → Lambda → Flask `/generate-image` → 回傳圖片 URL → Bot 貼到頻道。

第三步：你現在的程式流程長什麼樣？

1. Discord 使用者在頻道打：

text

 複製程式碼

!genimg 一隻可愛的貓咪

2. discordbot.py 收到後：

- 對 LAMBDA_URL 發 HTTP POST，body 是：

json

 複製程式碼

```
{ "prompt": "一隻可愛的貓咪", "user_id": "123456789" }
```

3. API Gateway 收到後：

- 把整個 HTTP request 轉成 Lambda event (有 httpMethod, path, body 等)
- 丟給 lambda_function.lambda_handler

4. lambda_handler 裡：

python

 複製程式碼

```
return awsgi.response(app, event, context)
```

awscli 把 event 轉成 WSGI，丟進 Flask 的 app。

剛好 route /generate-image、method POST → 執行 generate_image()。

5. generate_image() 跑 Hugging Face → 存圖到 S3 → 回 JSON 給 API GW → API GW 回給 Discord bot。

第四步：你 不需要 手動「啟動 app.py」

在 Lambda 架構裡：

- 不要再 app.run()，那是本地開 Flask server 用的。
- 你現在 app.py 只要有：

python

 複製程式碼

```
app = Flask(__name__)
@app.route("/generate-image", methods=["POST"])
...

```

就夠了；每次 Lambda handler 被呼叫時，awscli 都會把本次請求丟進 app 處理，可以想成「框架幫你進出 HTTP 層」。

你接下來可以做的順序：

1. 先用 Postman / curl 打 API Gateway 的 `/generate-image`，確認 Lambda 跑得起來。
2. 確定 body、回傳 JSON 都 OK 後，把 `LAMBDA_URL` 填進 Discord bot。
3. 在 Discord 測 `!genimg xxx`，看有沒有收到 Lambda 的錯誤訊息或成功訊息。

如果你願意貼一下你的「API Gateway 的 Invoke URL」還有目前 `generate_image` 的回傳 JSON 格式，我可以直接幫你把 Discord bot 那段 `genimg` 指令的程式改成完全對齊你現在的 API。



↳ 建 REST API

security policy要選什麼

先說結論：

你這個情境 (Discord Bot 直接呼叫 API Gateway/Lambda) 安全性先簡單就好 → security / authorization 一律選「NONE / Open (不要驗證)」就可以了。

你大概看到的選項長這樣

1. REST API 建好後，對 `/generate-image` 建 POST method 時

在 Method Request 裡會有：

- Authorization
→ 請選： `NONE` ☒
- API Key Required
→ 勾： `false` (不要勾) ☒

這樣你的 endpoint 就是 公開的 HTTP POST，Discord bot 用 `requests.post(url, json=...)` 就能打。

2. 如果你是看到 Lambda Function URL 的 Security (Auth type)

有時候你可能走的是 Lambda 的 Function URL，那裡會問：

- Auth type
 - `AWS_IAM`

- **NONE**

這裡你要選：**NONE** 

(不然你的 Discord bot 還要會簽 AWS IAM 的 SigV4，很麻煩，而且你現在 voclabs 也沒有 IAM key)

為什麼選 NONE 是合理的？

- 這是課堂/專題用，先以「能通」為主
- 你沒有 IAM User / Cognito 可以用 (voclabs 把 IAM 管理鎖住)
- Discord bot 只是從你控制的程式打出來

你可以在程式裡 **自己加一個簡單的密鑰** 來擋亂打一點，例如：

往 Lambda 打時加一個 header：

python

 複製程式碼

```
headers = {"X-API-SECRET": "some-secret-value"}
```

然後在 Flask 的 `generate_image()` 最前面檢查：

python

 複製程式碼

```
from flask import abort

@app.route("/generate-image", methods=["POST"])
def generate_image():
    if request.headers.get("X-API-SECRET") != os.getenv("INTERNAL_SECRET"):
        abort(403)
    ...
```

Lambda 的環境變數再多加一個 `INTERNAL_SECRET`，這樣就算 endpoint 是公開的，一般人也猜不到你的 secret，不會亂用。

總結你現在要選的

- API Gateway REST API / Method 的 Authorization：**NONE**
- API Key Required：不要勾
- (如果有) Lambda Function URL Auth type：**NONE**

你先照這樣 deploy 一版，
只要你拿到的 Invoke URL 是：

text

 複製程式碼

```
https://xxxx.execute-api.us-east-1.amazonaws.com/prod/generate-image
```

貼到你 `discordbot.py` 的 `LAMBDA_URL` 就可以實際打通一整條路。
如果哪一步畫面跟你看到的稍微不一樣，你可以截圖給我，我幫你指出該選哪一個。

     ...

```
import discord
from discord.ext import commands
import logging
from dotenv import load_dotenv
import os
import io
import aiohttp
import requests

load_dotenv()
DC_token = os.getenv('DISCORD_TOKEN')
FLASK_API_URL = os.getenv('FLASK_API_URL',
'http://localhost:5000')

handler = logging.FileHandler(filename='discord.log',
encoding='utf-8', mode='w')
intent = discord.Intents.default()
intent.message_content = True
intent.members = True

bot = commands.Bot(command_prefix='/',
intents=intent)

@bot.event
async def on_ready():
    print(f"We are ready to go in, {bot.user.name}")

@bot.command()
async def hello(ctx):
```

```
await ctx.send(f"Hello {ctx.author.mention}!")

@bot.command()
async def create_photo(ctx, *, prompt: str = "未輸入 prompt"):
    try:
        payload = {"prompt": prompt, "user_id":
str(ctx.author.id)}
        response =
requests.post("http://localhost:5000/generate-image",
json=payload, timeout=60)
        data = response.json()

        if not data.get("success"):
            err = data.get("error") or "未知錯誤"
            details = data.get("details")
            msg = f"❌ Flask 回傳錯誤: {err}"
            if details:
                msg += f"\n 📄 詳細: {details}"
            await ctx.send(msg)
            return

        file_url = data.get("local_url")
        if not file_url:
            await ctx.send("❌ Flask 沒有回傳檔案 URL")
            return

        file_resp = requests.get(file_url)
        if file_resp.status_code == 200:
            with io.BytesIO(file_resp.content) as f:
                await ctx.send(file=discord.File(f,
filename="generated.png"))
            else:
                await ctx.send("❌ 下載圖片失敗")

    except Exception as e:
        await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

# /create_audio 指令
@bot.command()
async def create_audio(ctx, *, prompt: str = "未輸入
```

```
prompt"):
    # await ctx.send("這是生成音樂的功能")
    try:
        payload = {"prompt": prompt}
        response =
requests.post("http://localhost:5000/generate-audio",
json=payload, timeout=10)
        data = response.json()

        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤:
{data.get('reply', '未知錯誤')}")
            return

        # 2 取得 local_url
        file_url = data.get("local_url")
        if not file_url:
            await ctx.send("❌ Flask 沒有回傳檔案 URL")
            return

        # 3 從 URL 下載檔案內容
        file_resp = requests.get(file_url)
        file_content = file_resp.text # 文字檔使用 text

        # 4 將內容回 Discord
        await ctx.send(f"✅ 從 Flask 取得檔案內
容:\n{file_content}")

    except Exception as e:
        await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

@bot.command()
async def history(ctx):
    try:
        # 自動帶入 Discord 使用者 ID
        response = requests.get(f"
{FLASK_API_URL}/history?user_id={ctx.author.id}",
timeout=10)
        data = response.json()

        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤:
{data.get('error', '未知錯誤')}")
```

```

return

records = data.get("records", [])
if not records:
    await ctx.send(f"👤 使用者 {ctx.author.id} 沒有任何紀錄。")
    return

# 組合輸出
msg_lines = [f"{r['id']}. {r['prompt']} → {r['file_url']}"]
for r in records:
    msg = "\n".join(msg_lines)

# 分段輸出避免超過 Discord 限制
if len(msg) > 1900:
    chunks = [msg[i:i+1900] for i in range(0, len(msg), 1900)]
    for chunk in chunks:
        await ctx.send(f"📄 你的紀錄:\n{chunk}")
    else:
        await ctx.send(f"📄 你的紀錄:\n{msg}")

except Exception as e:
    await ctx.send(f"❌ 查詢 DB 失敗: {e}")

bot.run(DC_token, log_handler=handler, log_level=logging.DEBUG)
這是bot.py

```

已思考 28s >

讚，這支 `bot.py` 很完整 👍

現在它是「打本地 Flask (`http://localhost:5000`)」，我們要改成「打 API Gateway → 觸發 Lambda → 跑 `app.py`」。

我直接幫你改成「呼叫 Lambda API 的版本」，再解釋你要改哪些地方。

1. 環境變數改成存「API Gateway URL」

現在：

python

複製程式碼

```
FLASK_API_URL = os.getenv('FLASK_API_URL', 'http://localhost:5000')
```

我們直接把這個當「後端 API base URL」用就好，只是把 value 改成 **API Gateway 的 base URL**，例如：

env

複製程式碼

```
FLASK_API_URL=https://abcd1234.execute-api.us-east-1.amazonaws.com/prod
```

(這個在你 API Gateway Deploy 之後會看到)

2. 把所有 `http://localhost:5000` 改成用 `FLASK_API_URL`

原本 `create_photo` 裡：

python

複製程式碼

```
response = requests.post("http://localhost:5000/generate-image", json=payload, ti
```

改成：

python

複製程式碼

```
response = requests.post(f"{FLASK_API_URL}/generate-image", json=payload, timeout
```

`create_audio`：

python

複製程式碼

```
response = requests.post("http://localhost:5000/generate-audio", json=payload, ti
```

改成：

python

複製程式碼

```
response = requests.post(f"{FLASK_API_URL}/generate-audio", json=payload, timeout
```

history :

```
python
```

複製程式碼

```
response = requests.get(f"{FLASK_API_URL}/history?user_id={ctx.author.id}", timeo
```

這個你已經用 `FLASK_API_URL` 了，這邊不用動 🙌

(只要 Lambda 那邊有實作 `/history` 並接上 API Gateway)

3. 回傳格式要對齊 Lambda (S3) 那邊

目前你的 Lambda `/generate-image` 在 `app.py` 的最後是：

```
python
```

複製程式碼

```
return jsonify({
    "success": True,
    # "local_url": local_url,
    "reply": f"Flask 收到圖片prompt: {prompt}"
})
```

但 Discord bot 還在抓：

```
python
```

複製程式碼

```
file_url = data.get("local_url")
```

建議你把 Lambda 的回傳改成這樣 (在 `generate_image` 的最後)：

```
python
```

複製程式碼

```
return jsonify({
    "success": True,
    "download_url": presigned_url, # 🙌 這是 S3 presigned URL
    "reply": f"Flask 收到圖片prompt: {prompt}"
})
```

然後在 `bot.py` 用 `download_url`：

python

 複製程式碼

```
file_url = data.get("download_url")
```

這樣整條鏈就對齊了：

Lambda 產圖 → 存 S3 → 回一個下載網址給 bot → bot 去抓圖回傳 Discord。

4. 幫你整合好的 bot.py 範例

這版假設：

- 你已經在 .env 設：FLASK_API_URL=https://xxxx.execute-api.../prod
- Lambda /generate-image 回傳 success + download_url

python

 複製程式碼

```
import discord
from discord.ext import commands
import logging
from dotenv import load_dotenv
import os
import io
import requests

load_dotenv()
DC_token = os.getenv('DISCORD_TOKEN')

# 這裡放 API Gateway 的 base URL (在 .env 設 FLASK_API_URL)
FLASK_API_URL = os.getenv('FLASK_API_URL', 'http://localhost:5000')

handler = logging.FileHandler(filename='discord.log', encoding='utf-8', mode='w')
intent = discord.Intents.default()
intent.message_content = True
intent.members = True

bot = commands.Bot(command_prefix='/', intents=intent)

@bot.event
async def on_ready():
    print(f"We are ready to go in, {bot.user.name}")

@bot.command()
async def hello(ctx):
```



```
await ctx.send(f"Hello {ctx.author.mention}!")

@bot.command()
async def create_photo(ctx, *, prompt: str = "未輸入prompt"):
    try:
        payload = {"prompt": prompt, "user_id": str(ctx.author.id)}

        # ✅ 改成打 Lambda API (API Gateway)
        response = requests.post(f"{FLASK_API_URL}/generate-image", json=payload,
                                data = response.json())

        if not data.get("success"):
            err = data.get("error") or "未知錯誤"
            details = data.get("details")
            msg = f"❌ 後端回傳錯誤: {err}"
            if details:
                msg += f"\n📄 詳細: {details}"
            await ctx.send(msg)
            return

        # ✅ 後端改成回傳 download_url (S3 presigned URL)
        file_url = data.get("download_url") or data.get("local_url")
        if not file_url:
            await ctx.send("❌ 後端沒有回傳圖片 URL")
            return

        file_resp = requests.get(file_url)
        if file_resp.status_code == 200:
            with io.BytesIO(file_resp.content) as f:
                await ctx.send(file=discord.File(f, filename="generated.png"))
        else:
            await ctx.send("❌ 下載圖片失敗")

    except Exception as e:
        await ctx.send(f"❌ 呼叫後端失敗: {e}")

@bot.command()
async def create_audio(ctx, *, prompt: str = "未輸入prompt"):
    try:
        payload = {"prompt": prompt}
```

```

#  改成呼叫 Lambda 版的 /generate-audio (之後你在 Lambda 實作好)
response = requests.post(f"{FLASK_API_URL}/generate-audio", json=payload,
data = response.json()

if not data.get("success"):
    await ctx.send(f" 使用者 {ctx.author.id} 沒有任何紀錄。")
            return

        msg_lines = [f"{r['id']}. {r['prompt']} → {r['file_url']}" for r in records]
        msg = "\n".join(msg_lines)

        if len(msg) > 1900:
            chunks = [msg[i:i+1900] for i in range(0, len(msg), 1900)]
            for chunk in chunks:
                await ctx.send(f" 你的紀錄:\n{chunk}")

```

```

else:
    await ctx.send(f"📄 你的紀錄:\n{msg}")

except Exception as e:
    await ctx.send(f"❌ 查詢紀錄失敗: {e}")

bot.run(DC_token, log_handler=handler, log_level=logging.DEBUG)

```

5. 你現在整條流程會變成

1. Discord 中打 `/create_photo` 一隻可愛的貓
2. bot.py → POST 到 `https://xxxx.execute-api.../prod/generate-image`
3. API Gateway → 觸發 Lambda → awscli → Flask 的 `/generate-image` → 呼叫 HF → 存 S3 → 回 `download_url`
4. Discord bot 用 `requests.get(download_url)` 把圖片抓回來 → 貼到頻道

你現在要做的實作順序可以是：

1. 先確認 API Gateway `/generate-image` 用 Postman/curl 打得到成功/錯誤 JSON
2. 把 `.env` 的 `FLASK_API_URL` 換成 API Gateway base URL
3. 把 Lambda 的 `generate_image` 改成回傳 `download_url`
4. 用這版 `bot.py` 在 Discord 測 `/create_photo`

過程中如果 401 / S3 error / 其他錯誤再出現，把錯誤訊息貼給我，我可以幫你從「哪一層出錯」開始定位。

📄 📄 📄 📄 📄 ...

```

from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import sqlite3
import requests
from PIL import Image
import awscli

```

```
import boto3

# from db import init_db, insert_record
# init_db()
load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"

app = Flask(__name__)

s3 = boto3.client("s3") # 建立 s3 client

# 本地儲存目錄
# LOCAL_STORAGE = './generated_files'
# os.makedirs(f'{LOCAL_STORAGE}/images',
exist_ok=True)
# os.makedirs(f'{LOCAL_STORAGE}/audios',
exist_ok=True)

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少
prompt"}), 400

        headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
        payload = {"inputs": prompt}

        response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)
```

```
if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

content_type = response.headers.get('Content-
Type', '')

if 'application/json' in content_type:
    error_data = response.json()
    return jsonify({
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# 嘗試解析圖片
try:
    image =
Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

# 儲存到本地
# timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S
")
# filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
# local_path = f"
{LOCAL_STORAGE}/images/{filename}"
```

```
# image.save(local_path, format="PNG")

# 本地 URL
# local_url =
f"http://localhost:5000/generated_files/images/{filename}"

# 存到 S3
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = filename

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content,    # 直接用原始 bytes
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500
```

```
# 寫入 SQLite
# insert_record(user_id, prompt, local_url)
"""

return jsonify({
    "success": True,
    # "local_url": local_url,
    "reply": f"Flask 收到圖片 prompt: {prompt}"
})
"""

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    "reply": f"Flask 收到圖片 prompt: {prompt}"
})

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500

"""

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")

        filename = "ta.txt"
        # 模擬生成圖片，先回傳本地 txt URL
        # local_url =
        f"http://localhost:5000/generated_files/test.txt"
        local_url =
        f"http://localhost:5000/generated_files/audios/{filenam
e}"

        return jsonify({
            "success": True,
```

```
"local_url": local_url,
"reply": f"Flask 收到音樂prompt: {prompt}"
})

except Exception as e:
    return jsonify({"success":False, "error":str(e)}),500
# 呼叫索取生成結果(圖片/音樂)
@app.route('/generated_files/<file_type>/<filename>',
methods=['GET'])
def serve_file(file_type, filename):

    # file_type: 'images' 或 'audios'
    # filename: 檔案名稱

    # 驗證子資料夾
    if file_type not in ["images", "audios"]:
        return jsonify({"success": False, "error": "不支援的檔案類型"}), 400

    directory = os.path.join(LOCAL_STORAGE, file_type)
    return send_from_directory(directory, filename)

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
        query 取得 user_id
        conn = sqlite3.connect("bot_records.db")
        c = conn.cursor()

        if user_id:
            c.execute("SELECT id, user_id, timestamp,
prompt, file_url FROM records WHERE user_id=?
ORDER BY id DESC", (user_id,))
        else:
            c.execute("SELECT id, user_id, timestamp,
prompt, file_url FROM records ORDER BY id DESC")

        rows = c.fetchall()
        conn.close()

        return jsonify({
            "success": True,
```



```

        "records": [
            {"id": r[0], "user_id": r[1], "timestamp": r[2],
            "prompt": r[3], "file_url": r[4]}
            for r in rows
        ]
    })
except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500
"""

```

```

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)

```

以上是雲端的app.py

```

import discord
from discord.ext import commands
import logging
from dotenv import load_dotenv
import os
import io
import aiohttp
import requests

load_dotenv()
DISCORD_TOKEN
="MTQzNTI3MDk4NDc4NDU0Nzg0Mg.Glnp2b.JOi1Ke
8hUGHyH_W0tp2q-D0WCtFiDglyUCZi3U"
DC_token = os.getenv('DISCORD_TOKEN')
# FLASK_API_URL = os.getenv('FLASK_API_URL',
'http://localhost:5000')
FLASK_API_URL = "https://xpapysqd2i.execute-api.us-
east-1.amazonaws.com/prod"

handler = logging.FileHandler(filename='discord.log',
encoding='utf-8', mode='w')
intent = discord.Intents.default()
intent.message_content = True
intent.members = True

bot = commands.Bot(command_prefix='/',
intents=intent)

```

```
@bot.event
async def on_ready():
    print(f"We are ready to go in, {bot.user.name}")

@bot.command()
async def hello(ctx):
    await ctx.send(f"Hello {ctx.author.mention}!")

@bot.command()
async def create_photo(ctx, *, prompt: str = "未輸入 prompt"):
    try:
        payload = {"prompt": prompt, "user_id":
str(ctx.author.id)}
        # response =
requests.post("http://localhost:5000/generate-image",
json=payload, timeout=60)
        response = requests.post(f"
{FLASK_API_URL}/generate-image", json=payload,
timeout=60)
        data = response.json()

        if not data.get("success"):
            err = data.get("error") or "未知錯誤"
            details = data.get("details")
            msg = f"❌ Flask 回傳錯誤: {err}"
            if details:
                msg += f"\n 📄 詳細: {details}"
            await ctx.send(msg)
            return

        """"
        file_url = data.get("local_url")
        if not file_url:
            await ctx.send("❌ Flask 沒有回傳檔案 URL")
            return

        file_resp = requests.get(file_url)
        if file_resp.status_code == 200:
            with io.BytesIO(file_resp.content) as f:
                await ctx.send(file=discord.File(f,
```

```
filename="generated.png"))
    else:
        await ctx.send("❌ 下載圖片失敗")
        """"

except Exception as e:
    await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

""""

# /create_audio 指令
@bot.command()
async def create_audio(ctx, *, prompt: str = "未輸入 prompt"):
    # await ctx.send("這是生成音樂的功能")
    try:
        payload = {"prompt": prompt}
        response =
requests.post("http://localhost:5000/generate-audio",
json=payload, timeout=10)
        data = response.json()

        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤:
{data.get('reply', '未知錯誤')}")
            return

        # 2 取得 local_url
        file_url = data.get("local_url")
        if not file_url:
            await ctx.send("❌ Flask 沒有回傳檔案 URL")
            return

        # 3 從 URL 下載檔案內容
        file_resp = requests.get(file_url)
        file_content = file_resp.text # 文字檔使用 text

        # 4 將內容回 Discord
        await ctx.send(f"✅ 從 Flask 取得檔案內
容:\n{file_content}")

    except Exception as e:
        await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

@bot.command()
```

```

async def history(ctx):
    try:
        # 自動帶入 Discord 使用者 ID
        response = requests.get(f"
{FLASK_API_URL}/history?user_id={ctx.author.id}",
        timeout=10)
        data = response.json()

        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤:
{data.get('error', '未知錯誤')}")
            return

        records = data.get("records", [])
        if not records:
            await ctx.send(f"👤 使用者 {ctx.author.id} 沒有任
何紀錄。")
            return

        # 組合輸出
        msg_lines = [f"{r['id']}. {r['prompt']} → {r['file_url']}"]
        for r in records:
            msg = "\n".join(msg_lines)

            # 分段輸出避免超過 Discord 限制
            if len(msg) > 1900:
                chunks = [msg[i:i+1900] for i in range(0,
len(msg), 1900)]
                for chunk in chunks:
                    await ctx.send(f"📄 你的紀錄:\n{chunk}")
            else:
                await ctx.send(f"📄 你的紀錄:\n{msg}")

    except Exception as e:
        await ctx.send(f"❌ 查詢 DB 失敗: {e}")
"""

bot.run(DC_token, log_handler=handler,
log_level=logging.DEBUG)

```

以上是本地bot.py
幫我檢查是否有問題

已思考 40s >

先說結論：

整體架構是對的：Discord → API Gateway → Lambda(awscli) → app.py → S3

程式只剩幾個「要修 / 要補」的點，尤其是：你剛剛直接把 Discord token 貼出來，這個一定要重設。

我分成三塊講：🔒 安全、☁️ app.py、🤖 bot.py。

🔒 0. 先處理超重要的安全問題 (Discord Token)

你現在 `bot.py` 有這行：

python

複製程式碼

```
DISCORD_TOKEN ="你的那串 token"
DC_token = os.getenv( 'DISCORD_TOKEN' )
...
bot.run(DC_token, ...)
```

問題有兩個：

1. 你已經把真正的 Discord Bot Token 貼出來了

- 這個 token 等同於「Bot 的密碼」，任何人拿到就可以控制你的 Bot。
- 建議你立刻去 Discord Developer Portal：
 - 找到你的 Bot → Reset Token
 - 換一個新的，舊的就作廢了。

2. 程式其實完全沒用到 `DISCORD_TOKEN` 這個常數

- `bot.run()` 用的是 `DC_token` (讀 `.env`)
- 所以那一行硬寫 token 完全是多餘，而且是嚴重外洩風險。

✅ 建議改成這樣 (乾淨版)：

python

複製程式碼

```
load_dotenv()
DC_token = os.getenv( 'DISCORD_TOKEN' )

if not DC_token:
    raise RuntimeError("環境變數 DISCORD_TOKEN 沒有設定")
```

```
bot.run(DC_token, log_handler=handler, log_level=logging.DEBUG)
```

然後在 `.env` 裡放：

env

複製程式碼

```
DISCORD_TOKEN=新生成的那串 token
FLASK_API_URL=https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod
```

☁ 1. 雲端的 app.py 檢查

你貼的 `app.py` 目前核心路由是這樣（我只抓關鍵）：

python

複製程式碼

```
@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片prompt")
        user_id = data.get("user_id", "unknown")
        ...
        response = requests.post(HuggingFace_URL, headers=headers, json=payload,
        ...
        # 存到 S3
        timestamp = ...
        filename = ...
        s3_key = filename

        s3.put_object(...)
        presigned_url = s3.generate_presigned_url(...)

        return jsonify({
            "success": True,
            "download_url": presigned_url,
            "reply": f"Flask 收到圖片prompt: {prompt}"
        })
    except Exception as e:
        return jsonify({
            "success": False,
```

```
"error": str(e)
}), 500
```

✅ 這些是 OK 的

- HuggingFace 呼叫流程正常 (401 是 Token 問題，不是程式語法問題)
- 沒有再寫本地檔案，直接把 `response.content` 丟 S3 👍
- 有產生 `presigned_url`，回傳 `download_url`，方便 Discord bot 用來抓圖 👍
- `S3_BUCKET_NAME` 用環境變數讀，很合理

🔧 建議小修一下的地方 (不是一定要改，但比較穩)

1. `data = request.json` → 建議保底：

```
python
```

[複製程式碼](#)

```
data = request.json or {}
```

避免 body 不是合法 JSON 時，`data` 變 `None`，`data.get(...)` 直接爆。

2. 401 時的錯誤 `details` 現在是 `response.text`，會再變成字串包字串
如果你想讓 Discord 那邊看到比較乾淨，可以用我之前給你的版本：

```
python
```

[複製程式碼](#)

```
try:
    details = response.json()
except Exception:
    details = {"raw": response.text}

return jsonify({
    "success": False,
    "error": f"Hugging Face API 錯誤: {response.status_code}",
    "details": details
}), 500
```

3. `if __name__ == '__main__': app.run(...)`

- 在 Lambda 裡不會被執行，保留也沒關係
- 你要在本地跑 Flask server 測試時會用得到，所以可以留著

整體來說，`app.py` 現在是合理可以部署的狀態，只要 HuggingFace Token 正確、S3 權限 OK，就能正常產圖存 S3 並回 `download_url`。

🤖 2. 本地 bot.py 檢查 & 修正

你現在 `create_photo` 是這樣（我簡化一下）：

python

📋 複製程式碼

```
@bot.command()
async def create_photo(ctx, *, prompt: str = "未輸入prompt"):
    try:
        payload = {"prompt": prompt, "user_id": str(ctx.author.id)}
        response = requests.post(f"{FLASK_API_URL}/generate-image", json=payload,
                                data = response.json())

        if not data.get("success"):
            ...
            return

        """
        file_url = data.get("local_url")
        ...
        """

    except Exception as e:
        await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")
```

✅ 這些是對的

- `payload` 帶 `prompt` + `user_id` → 對應 `app.py` 的 `generate_image` ✅
- URL 用 `FLASK_API_URL` + `/generate-image` → 對接 API Gateway ✅
- 錯誤訊息處理 OK

❌ 少做了一件事：成功時沒有去抓 `download_url`，也沒有把圖片傳回 Discord

你已經讓 `app.py` 回傳 `download_url`，但在 `bot` 這邊你把「下載圖」的那段整個註解掉了，而且還是用舊的 `local_url`。

應該要改成：用 `download_url` 去抓圖，然後回傳。

可以改成這樣：

python

📋 複製程式碼

```
@bot.command()
async def create_photo(ctx, *, prompt: str = "未輸入prompt"):
    try:
```



```

payload = {"prompt": prompt, "user_id": str(ctx.author.id)}
response = requests.post(f"{FLASK_API_URL}/generate-image", json=payload,
data = response.json()

if not data.get("success"):
    err = data.get("error") or "未知錯誤"
    details = data.get("details")
    msg = f"❌ 後端回傳錯誤: {err}"
    if details:
        msg += f"\n📄 詳細: {details}"
    await ctx.send(msg)
    return

# ✅ 從後端拿 S3 的 presigned URL
file_url = data.get("download_url")
if not file_url:
    await ctx.send("❌ 後端沒有回傳圖片 URL")
    return

# ✅ 真的去抓圖片
file_resp = requests.get(file_url)
if file_resp.status_code == 200:
    with io.BytesIO(file_resp.content) as f:
        await ctx.send(file=discord.File(f, filename="generated.png"))
else:
    await ctx.send(f"❌ 下載圖片失敗 (HTTP {file_resp.status_code}) ")

except Exception as e:
    await ctx.send(f"❌ 呼叫後端失敗: {e}")

```

其它小細節

- `aiohttp` 沒用到，可以先刪掉 `import` (不影響功能，只是乾淨一點)
- `FLASK_API_URL` 建議從 `.env` 讀就好，現在你已經寫死成常數，也可以用，但 `.env` 比較好換環境：

```
python
```

📄 複製程式碼

```
FLASK_API_URL = os.getenv('FLASK_API_URL', 'http://localhost:5000')
```

✓ 總結：現在整體有沒有大問題？

- 雲端 `app.py`：邏輯 OK，可以正常接 API Gateway + Lambda + S3 ✓
- 雲端 `lambda_function.py`（你之前貼過）：`awsgi.response(app, event, context)` 結構正確 ✓
- 本地 `bot.py`：
 - Discord Token 一定要重設，改用環境變數讀！**
 - `create_photo` 要改成使用 `download_url` 抓圖並傳回 Discord（上面給了完整版本）✓

你照上面兩個檔案小修完之後，流程就會變成：

- 使用者在 Discord 打
`/create_photo` 一隻可愛的貓
- bot → POST 到 API Gateway `/prod/generate-image`
- API GW → Lambda → awsgi → Flask `/generate-image`
- Flask 呼叫 HuggingFace → 存 S3 → 回 `download_url`
- bot 用 `download_url` 把圖抓回來 → 發在 Discord 頻道 🖼️

如果你改完後在 Discord 下 `/create_photo` 有任何錯誤訊息（例如 401、S3 AccessDenied），把錯誤文字貼給我，我可以直接幫你看是哪一層出的問題。

📄 🍌 🗨️ ⬆️ ↺️ ...

```

root@6a9b43efd6f4:/app/backend/src# python bot.py
Traceback (most recent call last):
  File "/app/backend/src/bot.py", line 126, in <module>
    bot.run(DC_token, log_handler=handler,
    log_level=logging.DEBUG)
  File "/usr/local/lib/python3.12/site-
packages/discord/client.py", line 929, in run
    asyncio.run(runner())
  File "/usr/local/lib/python3.12/asyncio/runners.py",
line 195, in run
    return runner.run(main)
           ^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/asyncio/runners.py",
line 118, in run
    return self._loop.run_until_complete(task)

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File
  
```

```
"/usr/local/lib/python3.12/asyncio/base_events.py",
line 691, in run_until_complete
    return future.result()
           ^^^^^^^^^^^^^^^^^
File "/usr/local/lib/python3.12/site-
packages/discord/client.py", line 918, in runner
    await self.start(token, reconnect=reconnect)
File "/usr/local/lib/python3.12/site-
packages/discord/client.py", line 846, in start
    await self.login(token)
File "/usr/local/lib/python3.12/site-
packages/discord/client.py", line 672, in login
    raise TypeError(f'expected token to be a str, received
{token.__class__.__name__} instead')
TypeError: expected token to be a str, received
NoneType instead
root@6a9b43efd6f4:/app/backend/src# python bot.py
Traceback (most recent call last):
  File "/app/backend/src/bot.py", line 127, in <module>
    bot.run(DC_token, log_handler=handler,
log_level=logging.DEBUG)
  File "/usr/local/lib/python3.12/site-
packages/discord/client.py", line 929, in run
    asyncio.run(runner())
  File "/usr/local/lib/python3.12/asyncio/runners.py",
line 195, in run
    return runner.run(main)
           ^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/asyncio/runners.py",
line 118, in run
    return self._loop.run_until_complete(task)

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File
"/usr/local/lib/python3.12/asyncio/base_events.py",
line 691, in run_until_complete
    return future.result()
           ^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/site-
packages/discord/client.py", line 918, in runner
    await self.start(token, reconnect=reconnect)
  File "/usr/local/lib/python3.12/site-
packages/discord/client.py", line 846, in start
```

```
await self.login(token)
File "/usr/local/lib/python3.12/site-packages/discord/client.py", line 672, in login
    raise TypeError(f'expected token to be a str, received {token.__class__.__name__} instead')
TypeError: expected token to be a str, received NoneType instead
```

很好，這個錯誤現在非常清楚：

```
TypeError: expected token to be a str, received NoneType instead
```

→ 代表 `bot.run(DC_token, ...)` 裡的 `DC_token = None`

→ 也就是 `os.getenv('DISCORD_TOKEN')` 根本沒拿到值。

1. 問題點在哪？

你現在（照你之前改的）應該是寫類似：

python

 複製程式碼

```
load_dotenv()
DC_token = os.getenv('DISCORD_TOKEN')
bot.run(DC_token, log_handler=handler, log_level=logging.DEBUG)
```

但在 Docker 容器裡執行時：

- 找不到環境變數 `DISCORD_TOKEN`
- 或 `.env` 檔沒有被讀到 / 沒被 mount 進容器
 - 所以 `DC_token` 變成 `None`
 - discord.py 報：expected token to be a str, got NoneType

2. 先加一個檢查（避免之後又一樣摔）

在 `bot.py` 下面這段加上防呆：

python

 複製程式碼

```
load_dotenv()
DC_token = os.getenv('DISCORD_TOKEN')
```

```
if not DC_token:
    raise RuntimeError("環境變數 DISCORD_TOKEN 沒有設定，請在 .env 或 docker 環境變數
```

這樣至少你會在一開始就看到「沒有設定 DISCORD_TOKEN」，而不是看到一大串 stack trace。

3. 在 Docker 裡要怎麼給 DISCORD_TOKEN ?

方法 A：在容器裡確認有沒有 .env / 內容對不對

在你現在的路徑：

```
bash

root@6a9b43efd6f4:/app/backend/src# ls
root@6a9b43efd6f4:/app/backend/src# cat .env
```

複製程式碼

確認這裡的 `.env` 檔裡有這一行（新的 token，已經在 Discord Developer Portal 重設過的那一串）：

```
env

DISCORD_TOKEN=你的新 Bot Token
FLASK_API_URL=https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod
```

複製程式碼

注意：你之前貼出來的舊 token 要在 portal 裡 **重置 (Regenerate)**，那串已經不能再用了。

`load_dotenv()` 會去讀「目前工作目錄」底下的 `.env`，所以你要確定 `.env` 就在 `/app/backend/src/.env`。

方法 B：用環境變數 (docker run / docker-compose)

如果你是用 `docker run`：

```
bash

docker run -e DISCORD_TOKEN=你的新token -e FLASK_API_URL=https://xpapysqd2i.execute
```

複製程式碼

如果是 `docker-compose.yml`，可以加：

yaml

 複製程式碼

```
services:
  backend:
    environment:
      - DISCORD_TOKEN=你的新token
      - FLASK_API_URL=https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod
```

這樣 `os.getenv('DISCORD_TOKEN')` 在容器裡就會拿到值。

4. 建議你現在照這樣做：

1. 到 **Discord Developer Portal** 把 **Bot Token** 重設 (舊的已經外流，別再用)
2. 在你本機專案的 `backend/src/.env` 寫入：

env

 複製程式碼

```
DISCORD_TOKEN=新token
FLASK_API_URL=https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod
```

3. 確保這個 `.env` 有被 copy / mount 到 Docker 的 `/app/backend/src` (可以進容器 `cat /app/backend/src/.env` 確認)
4. 在 `bot.py` 加上剛剛那個檢查：

python

 複製程式碼

```
load_dotenv()
DC_token = os.getenv('DISCORD_TOKEN')

if not DC_token:
    raise RuntimeError("環境變數 DISCORD_TOKEN 沒有設定")
```

5. 再跑一次：

bash

 複製程式碼

```
root@6a9b43efd6f4:/app/backend/src# python bot.py
```

如果這次有新錯誤 (例如連不到 API Gateway、HuggingFace 401、S3 問題等等)，把新的錯誤訊息貼給我，我可以幫你看下一步是哪一層出問題。

     ...

```
import json
from db import insert_record # 確保 Lambda 層或 zip
包含 db.py

def lambda_handler(event, context):
    # 模擬輸入資料
    user_id = event.get("user_id", "test-user")
    prompt = event.get("prompt", "測試圖片")
    file_url = event.get("file_url",
"https://example.com/test.png")

    try:
        result = insert_record(
            user_id=user_id,
            prompt=prompt,
            file_url=file_url,
            record_type="image",
            status="success",
            extra_meta={
                "model": "stable-diffusion-xl-base-1.0",
                "resolution": "1024x1024"
            }
        )
        return {
            "statusCode": 200,
            "body": json.dumps({
                "message": "✅ 插入成功",
                "record": result
            })
        }
    except Exception as e:
        return {
            "statusCode": 500,
            "body": json.dumps({
                "message": "❌ 插入失敗",
                "error": str(e)
            })
        }
}
```

給我test event

已思考 9s >

你這個 `lambda_handler` 是直接從 `event` 拿欄位：

python

複製程式碼

```
user_id = event.get("user_id", "test-user")
prompt = event.get("prompt", "測試圖片")
file_url = event.get("file_url", "https://example.com/test.png")
```

所以 **Test Event** 就是一個很單純的 JSON 物件，長這樣就可以了：

json

複製程式碼

```
{
  "user_id": "112502029",
  "prompt": "一隻在宇宙裡飄浮的貓咪",
  "file_url": "https://example.com/test-image.png"
}
```

在 Lambda Console：

1. 點 Test → Configure test event / Edit
2. 把上面這段 JSON 貼進去
3. 存起來再按 Test

如果你想測真正的檔案網址，就把 `file_url` 改成你實際存 S3 的 URL 或你系統裡用的網址字串就可以，例如：

json

複製程式碼

```
{
  "user_id": "112502029",
  "prompt": "final project 測試",
  "file_url": "https://你的-bucket.s3.amazonaws.com/images/test.png"
}
```

📄 🍴 🗣️ 📶 🔄 ...

存東西進dynamoDB後，要怎麼看資料位置

已思考 15s >

簡單講：

在 DynamoDB 裡，「資料的位置」就是：Region + Table 名字 + Partition Key / Sort Key 的那一列 Item。

要看剛剛 Lambda 存進去的資料，最方便就是用 AWS Console 進去看 Item。

我一步一步教你怎麼點。

1 先到正確的 Region & DynamoDB 服務

1. 右上角看一下 **Region** (你 Lambda 在哪一區，就要選同一區)

例如：`N. Virginia (us-east-1)`。

2. 左上角服務搜尋列打：`DynamoDB`
3. 點進 **DynamoDB** 服務主畫面。

2 找到你的 Table

1. 左邊選單點 **Tables** (資料表)
2. 中間清單會列出你所有的 table
 - 找你在程式裡用的那個名字 (例如你在 `insert_record` 裡有寫 `TABLE_NAME = "xxx"`)
3. 點進那個 Table 名字。

3 看「Items」裡的資料列

依照新版/舊版 Console 可能用詞有點不一樣，大致是：

1. 在 table 頁面上，找類似：
 - **Explore table items** 按鈕，或
 - 上方 Tab：**Items**
2. 點進去後，就會看到一個清單，每一列就是一個 **Item** (就像一筆 row)。

如果你剛剛 Lambda 用 Test event 寫進 DynamoDB，就會看到類似：

json

 複製程式碼

```
{
  "user_id": "test-user",
  "prompt": "測試圖片",
  "file_url": "https://example.com/test.png",
  "record_type": "image",
```

```

    "status": "success",
    "extra_meta": {
      "model": "stable-diffusion-xl-base-1.0",
      "resolution": "1024x1024"
    }
  }
}

```

具體欄位要看你 `insert_record(...)` 裡怎麼建 item。

4 如果項目很多，怎麼找到「某一筆」？

看你 Table 的 **Partition key / Sort key** 設計：

- 如果 Partition key 是 `user_id`：
 - 點「Query」→ 指定 `user_id = "112502029"`，就會看到該使用者的全部紀錄。
- 如果 Partition key 是 `id`（自動生成 UUID）：
 - 你可以先用「Scan」全部列出，再在畫面上搜尋關鍵字（例如 prompt 內容）。

你也可以用最笨但最直接的方式：

- 直接按 **Scan**（不加條件）。
- 看最上面幾筆是不是剛剛插入的那組資料。

5 如果完全看不到資料，優先檢查這幾件事

1. Region 是否選對？

Lambda 和 DynamoDB Table 必須在同一個 Region，你也要在那個 Region 開 Console。

2. Table 名稱是否跟程式的一樣？

你可以打開 `db.py` 裡的 `TABLE_NAME` 看一下實際在寫哪一個表。

3. Lambda 寫入有沒有成功？

你可以看 Lambda 的 Test 回傳：

- `statusCode = 200` 並且說「插入成功」→ 有機會在 table 裡看到
- `statusCode = 500` → 去 CloudWatch Logs 看錯誤訊息（可能是沒權限 or table 名打錯）

如果你願意把 `db.py` 裡跟 DynamoDB 有關的那幾行（例如 table 名、partition key）貼出來，我可以直接跟你說：

👉 「你要在 Console 的哪個 table、用哪個欄位去 Query，就可以看到你剛剛那筆測試資料。」



s3 bucket
用 lambda 存取

已思考 16s >

可以，用 Lambda 存取 S3 大概分成兩件事：

1. Lambda 要有權限 (IAM Role) 去碰你的 S3 bucket
2. 你的 Lambda 程式裡用 `boto3` 呼叫 S3 API (上傳 / 下載 / 列表)

你之前圖片上傳那段其實已經是「Lambda 存取 S3」的標準做法了，我幫你整理成一個通用版本，之後你要存別的檔也可以照抄。

1 Lambda 要有 S3 權限 (Execution Role)

在 Lambda 的 **Configuration** → **Permissions** 看到一個執行角色 (例如 `LabRole` 之類)，

那個 Role 的 Policy 要給 S3 權限，例如只給某個 bucket：

json

複製程式碼

```
{
  "Effect": "Allow",
  "Action": [
    "s3:PutObject",
    "s3:GetObject",
    "s3:ListBucket"
  ],
  "Resource": [
    "arn:aws:s3:::你的-bucket-名字",
    "arn:aws:s3:::你的-bucket-名字/*"
  ]
}
```

voclabs 環境可能是老師事先設好，你看不到 policy 細節，但重點概念是：
Lambda 是用「自己的 Role」去存取 S3，不用你手動寫 Access Key。

2 Lambda 裡用 boto3 存取 S3 (你可以當作模板)

先在 Lambda 的環境變數設定：

- `S3_BUCKET_NAME` = 你的 bucket 名字

(1) 上傳檔案到 S3 (put_object)

python

複製程式碼

```
import os
import boto3

s3 = boto3.client("s3")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")

def lambda_handler(event, context):
    # 假設我們要存一段文字當成 .txt 檔
    content = "Hello S3 from Lambda!"
    key = "test-folder/hello.txt" # 存在 bucket 裡的路徑

    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=key,
        Body=content.encode("utf-8"),
        ContentType="text/plain"
    )

    return {
        "statusCode": 200,
        "body": f"已上傳到 s3://{S3_BUCKET_NAME}/{key}"
    }
```

你現在圖片那段其實就是把 `Body` 換成 `response.content` (圖片 bytes)，已經在用這一招：

python

複製程式碼

```
s3.put_object(
    Bucket=S3_BUCKET_NAME,
```

```
    Key=s3_key,  
    Body=response.content,  
    ContentType="image/png"  
)
```

(2) 從 S3 讀檔 (get_object)

python

 複製程式碼

```
def lambda_handler(event, context):  
    key = event.get("key", "test-folder/hello.txt")  
  
    obj = s3.get_object(Bucket=S3_BUCKET_NAME, Key=key)  
    data = obj["Body"].read().decode("utf-8")    # 如果是文字檔  
  
    return {  
        "statusCode": 200,  
        "body": f"讀到內容：{data}"  
    }
```

如果是圖片檔，你就不要 decode，直接拿 `Body().read()` 的 bytes 去做後續處理。

(3) 列出某個 bucket / 資料夾裡的檔案 (list_objects_v2)

python

 複製程式碼

```
def lambda_handler(event, context):  
    prefix = event.get("prefix", "images/")    # 看哪個子資料夾  
  
    resp = s3.list_objects_v2(Bucket=S3_BUCKET_NAME, Prefix=prefix)  
  
    keys = []  
    for item in resp.get("Contents", []):  
        keys.append(item["Key"])  
  
    return {  
        "statusCode": 200,  
        "body": {  
            "prefix": prefix,  
            "files": keys  
        }  
    }
```

```
}  
}
```

3 如果想從 S3 觸發 Lambda (反過來)

剛剛講的是「Lambda 主動去存取 S3。」

你如果要變成「S3 有新檔案 → 自動觸發 Lambda」，做法是：

1. 到 S3 → 你的 bucket → Properties → Event notifications
2. 建一個 Event：
 - Event type：All object create events (物件建立就觸發)
 - Prefix (可選)：例如 uploads/，只監聽這個資料夾
 - Destination：選你的 Lambda function

這樣每次有檔案 put_object 進來 (例如你從前端/別處上傳)，S3 會把 event 丟給 Lambda；

在 Lambda 裡 event 會長得像：

json

複製程式碼

```
{  
  "Records": [  
    {  
      "s3": {  
        "bucket": {  
          "name": "你的-bucket"  
        },  
        "object": {  
          "key": "images/xxxx.png"  
        }  
      }  
    }  
  ]  
}
```

你就可以在 lambda_handler 裡讀出 key，再用 get_object 做下一步處理。

4 跟你現在專題的關係 (小整理)

你目前的架構是：

- Lambda (awscli + Flask) → 呼叫 HuggingFace 產圖 →
`s3.put_object(...)` 把圖存進 S3 → `presigned_url` 回給 Discord bot

這已經是 **Lambda 存取 S3 的標準實作** 了。

如果你接下來要：

- 把「歷史記錄」的圖片 URL 也存 DynamoDB / SQLite
- 或要做「列出某個 user 的所有圖片」

就會再用到 `list_objects_v2` 或資料庫去記錄 `s3_key` / `download_url` 。

如果你跟我說你的 bucket 名跟「下一步想做的事」（例如：列出所有圖片、刪檔案），我可以直接幫你寫對應的 Lambda 範例 function 給你貼進去用。



```
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import sqlite3
import requests
from PIL import Image
import awscli
import boto3
# from db import insert_record, get_history

# from db import init_db, insert_record
# init_db()
load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"

app = Flask(__name__)

s3 = boto3.client("s3") # 建立 s3 client
# dynamodb = boto3.resource("dynamodb")
```

```
# table = dynamodb.Table("UserRecords")

# 本地儲存目錄
# LOCAL_STORAGE = './generated_files'
# os.makedirs(f'{LOCAL_STORAGE}/images',
# exist_ok=True)
# os.makedirs(f'{LOCAL_STORAGE}/audios',
# exist_ok=True)

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json or {}
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少
prompt"}), 400

        headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
        payload = {"inputs": prompt}

        response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)

        if response.status_code != 200:
            return jsonify({
                "success": False,
                "error": f"Hugging Face API 錯誤:
{response.status_code}",
                "details": response.text
            }, 500)

        content_type = response.headers.get('Content-
Type', '')

        if 'application/json' in content_type:
            error_data = response.json()
            return jsonify({
                "success": False,
```



```
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }}, 500

    if 'image' not in content_type:
        return jsonify({
            "success": False,
            "error": f"非預期的回傳格式: {content_type}"
        }), 500

    # 嘗試解析圖片
    try:
        image =
        Image.open(io.BytesIO(response.content))
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"圖片解析失敗: {str(e)}"
        }), 500

    # 儲存到本地
    # timestamp =
    datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    # filename = f"
    {timestamp}_{uuid.uuid4().hex[:8]}.png"
    # local_path = f"
    {LOCAL_STORAGE}/images/{filename}"
    # image.save(local_path, format="PNG")

    # 本地 URL
    # local_url =
    f"http://localhost:5000/generated_files/images/{filename}"

    # 存到 S3
    timestamp =
    datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"
    {timestamp}_{uuid.uuid4().hex[:8]}.png"
    s3_key = filename
```

```
try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content,    # 直接用原始 bytes
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

"""

# 建立紀錄
record_id = str(uuid.uuid4())
created_at =
datetime.datetime.utcnow().isoformat()

table.put_item(
    Item={
        "user_id": user_id,
        "record_id": record_id,
        "prompt": prompt,
        "file_url": presigned_url,
        "created_at": created_at
    }
)
"""

# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=,
        record_type="image",
        status="success",
    )
    return {
        "statusCode": 200,
        "body": json.dumps({
            "message": "✅ 插入成功",
```

```
        "record": result
    })
}
except Exception as e:
    return {
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    }

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

# 寫入 SQLite
# insert_record(user_id, prompt, local_url)
"""
return jsonify({
    "success": True,
    # "local_url": local_url,
    "reply": f"Flask 收到圖片prompt: {prompt}"
})
"""

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
presigned URL
    "reply": f"Flask 收到圖片prompt: {prompt}"
})
```

```
    })

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片prompt")

        filename = "ta.txt"
        # 模擬生成圖片，先回傳本地 txt URL
        # local_url =
        f"http://localhost:5000/generated_files/test.txt"
        local_url =
        f"http://localhost:5000/generated_files/audios/{filename}"

        return jsonify({
            "success": True,
            "local_url": local_url,
            "reply": f"Flask 收到音樂prompt: {prompt}"
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

"""
# 呼叫索取生成結果(圖片/音樂)
@app.route('/generated_files/<file_type>/<filename>',
methods=['GET'])
def serve_file(file_type, filename):

    # file_type: 'images' 或 'audios'
    # filename: 檔案名稱
```

```
# 驗證子資料夾
if file_type not in ["images", "audios"]:
    return jsonify({"success": False, "error": "不支援的檔案類型"}), 400

directory = os.path.join(LOCAL_STORAGE, file_type)
return send_from_directory(directory, filename)
"""

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL query 取得 user_id
        conn = sqlite3.connect("bot_records.db")
        c = conn.cursor()

        if user_id:
            c.execute("SELECT id, user_id, timestamp, prompt, file_url FROM records WHERE user_id=? ORDER BY id DESC", (user_id,))
        else:
            c.execute("SELECT id, user_id, timestamp, prompt, file_url FROM records ORDER BY id DESC")

        rows = c.fetchall()
        conn.close()

        return jsonify({
            "success": True,
            "records": [
                {"id": r[0], "user_id": r[1], "timestamp": r[2],
                "prompt": r[3], "file_url": r[4]}
                for r in rows
            ]
        })
    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)
file_url要填甚麼
```

已思考 21s >

超好，現在你已經走到「圖片丟到 S3 之後，還要把資訊存進資料庫」這一步了 🍌

你問的 `file_url` 要填什麼，其實就是在決定：「我要在 DB 裡記下哪一種圖片位置」。

先看你現在的程式哪裡在填 `file_url`

你貼的這段裡有這一塊（而且現在還是壞的）：

python

複製程式碼

```
# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=,
        record_type="image",
        status="success",
    )
    return {
        "statusCode": 200,
        "body": json.dumps({
            "message": "✅ 插入成功",
            "record": result
        })
    }
except Exception as e:
    return {
        "statusCode": 500,
        "body": json.dumps({
            "message": "❌ 插入失敗",
            "error": str(e)
        })
    }
}
```

這裡有幾個問題：

1. `file_url=` 什麼都沒填 → 語法直接錯
2. 這個 function 是 Flask 的 route，不應該 return Lambda 那種 `{"statusCode": ...}` 結構，這邊要維持 `return jsonify(...)` 才對

3. 你產生 `presigned_url` 的程式碼寫在「後面」，所以就算你要用 `presigned_url`，也拿不到

決定：`file_url` 裡到底要存什麼？

這裡有兩個常見選擇，你看你專題要哪一種：

方案 A：存「可直接下載的網址」（`presigned_url`）

- 優點：DB 直接拿來給前端 / Discord 用
- 缺點：`presigned URL` 有時效（你現在是 3600 秒）

寫法：

```
python
```

[複製程式碼](#)

```
file_url = presigned_url
```

這個很適合你現在的課堂專題，因為主要是 demo。

方案 B：存「永久的 S3 位置」（`bucket + key`）

- 優點：這個不會過期，以後你要再產 `presigned URL` 都可以
- 實際存的可能是：
 - `s3://bucket-name/key` 這種「邏輯位置」，或
 - `https://bucket-name.s3.amazonaws.com/key` 這種 HTTP URL

寫法（例如）：

```
python
```

[複製程式碼](#)

```
file_url = f"s3://{S3_BUCKET_NAME}/{s3_key}"  
# 或  
file_url = f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
```

比較正規的做法是：DB 存 `key + bucket`，讀取時再動態產 `presigned URL`。
但你現在作業比較趕，我會建議先用 方案 A（存 `presigned_url`）就好。

幫你整理一份「順序正確、可以存到 DynamoDB+回傳給 bot 的 `generate_image` 範例」

假設你的 `insert_record` 長相大概是：

python

 複製程式碼

```
# db.py
def insert_record(user_id, prompt, file_url, record_type, status, extra_meta=None
    ...
    return item # 或者回傳你存進去的那筆資料
```

那 `generate_image` 可以改成這樣（只改中後段）：

python

 複製程式碼

```
from db import insert_record
import json # 如果你真的要用到 json.dumps (但 Flask 這裡通常不需要)

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json or {}
        prompt = data.get("prompt", "沒有收到圖片prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少 prompt"}), 400

        headers = {"Authorization": f"Bearer {HuggingFace_Token}"}
        payload = {"inputs": prompt}

        response = requests.post(HuggingFace_URL, headers=headers, json=payload,

        if response.status_code != 200:
            return jsonify({
                "success": False,
                "error": f"Hugging Face API 錯誤: {response.status_code}",
                "details": response.text
            }), 500

        content_type = response.headers.get('Content-Type', '')

        if 'application/json' in content_type:
            error_data = response.json()
            return jsonify({
```



```
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# (可選) 確認真的能開圖
try:
    Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

# 1 先存到 S3
timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = filename

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content,
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

# 2 再產 presigned URL
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
```

```

        "Bucket": S3_BUCKET_NAME,
        "Key": s3_key
    },
    ExpiresIn=3600
)
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

# 3 存進 DynamoDB (或你自己的 db.insert_record)
try:
    record = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=presigned_url,  # ✓ 這裡就填你要存在 DB 的位置
        record_type="image",
        status="success",
    )
except Exception as e:
    # 存 DB 失敗，但圖片已經在 S3 了 → 這邊你可以選擇要不要直接回 error
    return jsonify({
        "success": False,
        "error": f"DB 插入失敗: {str(e)}"
    }), 500

# 4 回給 Discord bot 用
return jsonify({
    "success": True,
    "download_url": presigned_url,  # 給 bot 抓圖用
    "reply": f"Flask 收到圖片prompt: {prompt}",
    "record": record  # (可選) 順便把剛存的紀錄回傳
})

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500

```

重點：

- "file_url" : 你可以填 presigned_url (現在最直覺) "
- "順序 : 先 put_object → 再產 presigned_url → 再 insert_record → 最後 jsonify 回前端"

你現在可以選擇 :

- 如果只是要先跑起來 , 就讓 file_url = presigned_url , 之後再想要不要改成 s3://...
- 如果你說明一下你 insert_record 現在的實作 (DynamoDB 還是 SQLite , 欄位長什麼樣) , 我可以直接幫你串成 generate_image + insert_record + history 串成一整組一致的版本 , 這樣 Discord /history 也可以看 DynamoDB 裡的紀錄。



```
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import sqlite3
import requests
from PIL import Image
import awsgi
import boto3
from db import insert_record, get_history

load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"

app = Flask(__name__)

s3 = boto3.client("s3") # 建立 s3 client
# dynamodb = boto3.resource("dynamodb")
# table = dynamodb.Table("UserRecords")

# 本地儲存目錄
```

```
# LOCAL_STORAGE = './generated_files'
# os.makedirs(f'{LOCAL_STORAGE}/images',
exist_ok=True)
# os.makedirs(f'{LOCAL_STORAGE}/audios',
exist_ok=True)

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json or {}
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少
prompt"}), 400

        headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
        payload = {"inputs": prompt}

        response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)

        if response.status_code != 200:
            return jsonify({
                "success": False,
                "error": f"Hugging Face API 錯誤:
{response.status_code}",
                "details": response.text
            }), 500

        content_type = response.headers.get('Content-
Type', '')

        if 'application/json' in content_type:
            error_data = response.json()
            return jsonify({
                "success": False,
                "error": "Hugging Face 回傳錯誤",
                "details": error_data
            }), 500
```

```
if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# 嘗試解析圖片
try:
    image =
Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

# 儲存到本地
# timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
# filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
# local_path = f"
{LOCAL_STORAGE}/images/{filename}"
# image.save(local_path, format="PNG")

# 本地 URL
# local_url =
f"http://localhost:5000/generated_files/images/{filename}"

# 存到 S3
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = filename

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
```

```
        Key=s3_key,
        Body=response.content,    # 直接用原始 bytes
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

"""
# 建立紀錄
record_id = str(uuid.uuid4())
created_at =
datetime.datetime.utcnow().isoformat()

table.put_item(
    Item={
        "user_id": user_id,
        "record_id": record_id,
        "prompt": prompt,
        "file_url": presigned_url,
        "created_at": created_at
    }
)
"""

# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=f"s3://{S3_BUCKET_NAME}/{s3_key}",
        record_type="image",
        status="success",
    )
    return jsonify({
        "statusCode": 200,
        "body": json.dumps({
            "message": "✅ 插入成功",
            "record": result
        })
    })
})
```

```
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

# 寫入 SQLite
# insert_record(user_id, prompt, local_url)
"""
return jsonify({
    "success": True,
    # "local_url": local_url,
    "reply": f"Flask 收到圖片prompt: {prompt}"
})
"""

return jsonify({
    "success": True,
    "download_url": presigned_url, # 🖱 這是 S3
presigned URL
    "reply": f"Flask 收到圖片prompt: {prompt}"
})
```

```
except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片prompt")

        filename = "ta.txt"
        # 模擬生成圖片，先回傳本地 txt URL
        # local_url =
        f"http://localhost:5000/generated_files/test.txt"
        local_url =
        f"http://localhost:5000/generated_files/audios/{filename}"

        return jsonify({
            "success": True,
            "local_url": local_url,
            "reply": f"Flask 收到音樂prompt: {prompt}"
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

"""
# 呼叫索取生成結果(圖片/音樂)
@app.route('/generated_files/<file_type>/<filename>',
methods=['GET'])
def serve_file(file_type, filename):

    # file_type: 'images' 或 'audios'
    # filename: 檔案名稱

    # 驗證子資料夾
    if file_type not in ["images", "audios"]:
        return jsonify({"success": False, "error": "不支援的檔
```


案類型"}), 400

```
directory = os.path.join(LOCAL_STORAGE, file_type)
return send_from_directory(directory, filename)
"""
```

```
@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
        query 取得 user_id
        conn = sqlite3.connect("bot_records.db")
        c = conn.cursor()

        if user_id:
            c.execute("SELECT id, user_id, timestamp,
prompt, file_url FROM records WHERE user_id=?
ORDER BY id DESC", (user_id,))
        else:
            c.execute("SELECT id, user_id, timestamp,
prompt, file_url FROM records ORDER BY id DESC")

        rows = c.fetchall()
        conn.close()

        return jsonify({
            "success": True,
            "records": [
                {"id": r[0], "user_id": r[1], "timestamp": r[2],
"prompt": r[3], "file_url": r[4]}
                for r in rows
            ]
        })
    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)

import os
import uuid
import datetime
```

```
import boto3
from boto3.dynamodb.conditions import Key
from dotenv import load_dotenv

# 載入環境變數
load_dotenv()
AWS_REGION = os.getenv("AWS_DEFAULT_REGION",
"us-east-1")
TABLE_NAME = os.getenv("DYNAMODB_TABLE",
"UserRecords")

# 建立 DynamoDB 資源
dynamodb = boto3.resource("dynamodb",
region_name=AWS_REGION)
table = dynamodb.Table(TABLE_NAME)

def insert_record(user_id: str, prompt: str, file_url: str,
record_type: str = "image", status: str = "success",
extra_meta: dict = None):
    """
    新增一筆紀錄到 DynamoDB
    """
    record_id = str(uuid.uuid4())
    created_at = datetime.datetime.utcnow().isoformat()

    item = {
        "user_id": user_id,
        "record_id": record_id,
        "prompt": prompt,
        "file_url": file_url,
        "created_at": created_at,
        "type": record_type,
        "status": status,
    }

    if extra_meta:
        item["extra_meta"] = extra_meta

    table.put_item(Item=item)
    return item

    """

def get_history(user_id: str):
```

查詢某個使用者的所有紀錄

```
resp = table.query(

KeyConditionExpression=Key("user_id").eq(user_id)
)
return resp.get("Items", [])

def get_record(user_id: str, record_id: str):

    # 查詢單筆紀錄

    resp = table.get_item(
        Key={
            "user_id": user_id,
            "record_id": record_id
        }
    )
    return resp.get("Item")
"""
為甚麼無法正常插入dynamodb
```

已思考 55s >

現在這兩支檔案放在一起看，問題大概有三類：

- 1 你的 `generate-image` 回傳邏輯有「流程錯誤」
- 2 `insert_record` 有可能沒被正確呼叫 / 權限不夠
- 3 Flask 回傳用了 Lambda 的 `statusCode/body` 格式（怪怪的）

我先直接幫你把 **正確版流程畫好**，再說為什麼你現在「看起來像是沒插進 DynamoDB」。

先說結論：程式邏輯現在會卡在這裡

現在這一段：

```
python
```

 複製程式碼

```
# 存進 DynamoDB
try:
```

```
        result = insert_record(
            user_id=user_id,
            prompt=prompt,
            file_url=f"s3://{S3_BUCKET_NAME}/{s3_key}",
            record_type="image",
            status="success",
        )
    return jsonify({
        "statusCode": 200,
        "body": json.dumps({
            "message": "✅ 插入成功",
            "record": result
        })
    })
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "❌ 插入失敗",
            "error": str(e)
        })
    })
})
```

有幾個問題：

1. 這裡用了 `json.dumps`，但上面沒有 `import json`

→ 第一次執行到這裡會噴 `NameError: name 'json' is not defined`，

→ 被外層 `except Exception as e:` 接住，整個 route 回傳 `"success": False, "error": "name 'json' is not defined"`。

→ 你如果有看到這種錯，就幾乎可以確定 insert 有沒有成功要去 Console 看。

注意：即使這裡噴錯，`insert_record(...)` 已經有被呼叫過，如果它本身沒丟例外，**DynamoDB** 那筆資料是寫進去了，不會 rollback。

2. 這段回傳的 JSON 長相是：

```
json

{
  "statusCode": 200,
  "body": "{...json 字串...}"
}
```

 複製程式碼

這是 Lambda 最外層 handler 用的格式，

在 Flask route 裡應該直接回 `jsonify({...})`，不要自己再包一層 `statusCode / body`。

3. 你在「存進 DynamoDB」這個區塊就 return 掉了，
所以下面的：

python

複製程式碼

```
presigned_url = s3.generate_presigned_url(...)
return jsonify({
    "success": True,
    "download_url": presigned_url,
    ...
})
```

根本永遠不會被執行到（因為上面已經 return 了）。

正確流程應該長這樣（改好版本給你）

✅ 建議版 `generate_image`（直接拿去貼）

python

複製程式碼

```
from flask import Flask, request, jsonify
from dotenv import load_dotenv
import os, io, uuid, datetime
import requests
from PIL import Image
import boto3
from db import insert_record

load_dotenv()
HuggingFace_Token = os.getenv("HUGGINGFACE_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-inference/models/stabilityai/

app = Flask(__name__)
s3 = boto3.client("s3")

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
```

```
data = request.json or {}
prompt = data.get("prompt", "沒有收到圖片prompt")
user_id = data.get("user_id", "unknown")

if not prompt:
    return jsonify({"success": False, "error": "缺少 prompt"}), 400

# 1 呼叫 Hugging Face 產圖
headers = {"Authorization": f"Bearer {HuggingFace_Token}"}
payload = {"inputs": prompt}

response = requests.post(HuggingFace_URL, headers=headers, json=payload,

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤: {response.status_code}",
        "details": response.text
    }), 500

content_type = response.headers.get('Content-Type', '')

if "image" not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# (可選) 檢查圖片能不能打開
try:
    Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

# 2 存到 S3
timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = filename

try:
```

```
s3.put_object(
    Bucket=S3_BUCKET_NAME,
    Key=s3_key,
    Body=response.content,
    ContentType="image/png"
)
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

# 3 產 presigned URL (給 Discord 用)
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

# 4 存進 DynamoDB (file_url 你選擇要存 s3 路徑或 presigned URL)
try:
    record = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=f"s3://{S3_BUCKET_NAME}/{s3_key}", # 或改成 presigned_url
        record_type="image",
        status="success",
    )
except Exception as e:
    # 如果你希望「DB 失敗但圖片還是可以用」，也可以只印 log 不 return 500
    return jsonify({
        "success": False,
        "error": f"DynamoDB 插入失敗: {str(e)}"
    }), 500
```

```
# 5 最終回給 Discord bot
return jsonify({
    "success": True,
    "download_url": presigned_url,
    "reply": f"Flask 收到圖片prompt: {prompt}",
    "record": record
})

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500
```

重點：

- 不再在 Flask 裡自己包 `statusCode` / `body`，直接 `jsonify` 就好
- 先 S3 → 再 `presigned_url` → 再 `insert_record` → 最後 `return`
- `file_url` 我幫你設成 `s3://bucket/key`，之後 `/history` 要顯示真實圖片可以再用 `generate_presigned_url` 轉出來

DynamoDB 端要能「正常插入」，還要確認這幾件事

你 `db.py` 寫得滿 OK 的，我只幫你檢查關鍵點：

python

 複製程式碼

```
AWS_REGION = os.getenv("AWS_DEFAULT_REGION", "us-east-1")
TABLE_NAME = os.getenv("DYNAMODB_TABLE", "UserRecords")

dynamodb = boto3.resource("dynamodb", region_name=AWS_REGION)
table = dynamodb.Table(TABLE_NAME)
```

插入：

python

 複製程式碼

```
table.put_item(Item=item)
```

要能正常寫入，至少要符合：

1. Region 要對得上：

- 你的 DynamoDB Table 建在哪個 Region，`AWS_DEFAULT_REGION` 就要那個
- Lambda 也要在同一個 Region

2. Table 名字要對：

- Console 裡的 Table 名稱要叫 `UserRecords` (或你在 env `DYNAMODB_TABLE` 設的值)

3. Table 的 Key schema 要跟你寫入的一致：

你現在的 item 長這樣：

python

複製程式碼

```
item = {  
    "user_id": user_id,  
    "record_id": record_id,  
    ...  
}
```

那 Table 的 key schema 應該是：

- Partition key： `user_id` (String)
- Sort key (range key)： `record_id` (String)

如果 table 只有 partition key (沒有 record_id)，那 `put_item` 不會錯，但透過 query 時就要用別種方式；

不過通常會設成上述這種複合 key。

4. Lambda Execution Role 要有 dynamodb:PutItem 權限

- 最少要允許：

json

複製程式碼

```
{  
  "Action": [  
    "dynamodb:PutItem"  
  ],  
  "Resource": "arn:aws:dynamodb:REGION:ACCOUNT_ID:table/UserRecords"  
}
```

- voclabs 環境可能老師已經幫你加好，如果沒有，就會在 CloudWatch Logs 看到類似 `AccessDeniedException` 的錯誤。

下一步你可以怎麼確認「到底有沒有插入成功」

1. 在 `insert_record` 的 `except` 那裡，先照我上面的程式一樣回傳錯誤字串：

之後前端/Discord 收到的就會包含 DynamoDB 實際的錯誤原因 (例如 Table 不存

在、權限不足)。

2. 到 AWS Console → DynamoDB → Tables → 選 `UserRecords` → Explore items / Items

看有沒有新資料：

json

複製程式碼

```
{
  "user_id": "你的 Discord ID",
  "record_id": "xxxx-uuid",
  "prompt": "...",
  "file_url": "s3://你的bucket/xxxx.png",
  "created_at": "...",
  "type": "image",
  "status": "success"
}
```

如果你測一次 `/create_photo` 後，拿到的回應是 `success: False, error: "DynamoDB 插入失敗: ..."`，把那段錯誤訊息貼給我，我可以直接幫你看是 **權限**、**Region**、還是 **Table 名 / Schema** 的問題。

📄 👍 💬 ↗️ ↺️ ...

```
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import sqlite3
import requests
from PIL import Image
import awsgi
import boto3
from db import insert_record, get_history
import json

load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
```

```
inference/models/stabilityai/stable-diffusion-xl-base-1.0"
```

```
app = Flask(__name__)
```

```
s3 = boto3.client("s3") # 建立 s3 client  
# dynamodb = boto3.resource("dynamodb")  
# table = dynamodb.Table("UserRecords")
```

```
# 本地儲存目錄  
# LOCAL_STORAGE = './generated_files'  
# os.makedirs(f'{LOCAL_STORAGE}/images',  
exist_ok=True)  
# os.makedirs(f'{LOCAL_STORAGE}/audios',  
exist_ok=True)
```

```
@app.route("/generate-image", methods=["POST"])  
def generate_image():
```

```
    try:  
        data = request.json or {}  
        prompt = data.get("prompt", "沒有收到圖片  
prompt")  
        user_id = data.get("user_id", "unknown")  
  
        if not prompt:  
            return jsonify({"success": False, "error": "缺少  
prompt"}), 400
```

```
        headers = {"Authorization": f"Bearer  
{HuggingFace_Token}"}  
        payload = {"inputs": prompt}
```

```
        response = requests.post(HuggingFace_URL,  
headers=headers, json=payload, timeout=60)
```

```
        if response.status_code != 200:  
            return jsonify({  
                "success": False,  
                "error": f"Hugging Face API 錯誤:  
{response.status_code}",  
                "details": response.text  
            }), 500
```

```
content_type = response.headers.get('Content-Type', '')

if 'application/json' in content_type:
    error_data = response.json()
    return jsonify({
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# 嘗試解析圖片
try:
    image =
Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

# 儲存到本地
# timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
# filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
# local_path = f"
{LOCAL_STORAGE}/images/{filename}"
# image.save(local_path, format="PNG")

# 本地 URL
# local_url =
f"http://localhost:5000/generated_files/images/{filename}"

# 存到 S3
```

```
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")

filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = filename

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content,    # 直接用原始 bytes
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

"""
# 建立紀錄
record_id = str(uuid.uuid4())
created_at =
datetime.datetime.utcnow().isoformat()

table.put_item(
    Item={
        "user_id": user_id,
        "record_id": record_id,
        "prompt": prompt,
        "file_url": presigned_url,
        "created_at": created_at
    }
)
"""

# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=f"s3://{S3_BUCKET_NAME}/{s3_key}",
```

```
        record_type="image",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "❌ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

# 寫入 SQLite
# insert_record(user_id, prompt, local_url)
"""
return jsonify({
    "success": True,
    # "local_url": local_url,
    "reply": f"Flask 收到圖片prompt: {prompt}"
})
"""

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    presigned URL
    "reply": f"Flask 收到圖片prompt: {prompt}"
})
```

```
    })

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片prompt")

        filename = "ta.txt"
        # 模擬生成圖片，先回傳本地 txt URL
        # local_url =
        f"http://localhost:5000/generated_files/test.txt"
        local_url =
        f"http://localhost:5000/generated_files/audios/{filename}"

        return jsonify({
            "success": True,
            "local_url": local_url,
            "reply": f"Flask 收到音樂prompt: {prompt}"
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

"""
# 呼叫索取生成結果(圖片/音樂)
@app.route('/generated_files/<file_type>/<filename>',
methods=['GET'])
def serve_file(file_type, filename):

    # file_type: 'images' 或 'audios'
    # filename: 檔案名稱
```

```
# 驗證子資料夾
if file_type not in ["images", "audios"]:
    return jsonify({"success": False, "error": "不支援的檔案類型"}), 400

directory = os.path.join(LOCAL_STORAGE, file_type)
return send_from_directory(directory, filename)
"""

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL query 取得 user_id
        conn = sqlite3.connect("bot_records.db")
        c = conn.cursor()

        if user_id:
            c.execute("SELECT id, user_id, timestamp, prompt, file_url FROM records WHERE user_id=? ORDER BY id DESC", (user_id,))
        else:
            c.execute("SELECT id, user_id, timestamp, prompt, file_url FROM records ORDER BY id DESC")

        rows = c.fetchall()
        conn.close()

        return jsonify({
            "success": True,
            "records": [
                {"id": r[0], "user_id": r[1], "timestamp": r[2],
                 "prompt": r[3], "file_url": r[4]}
                for r in rows
            ]
        })
    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)
```



```
import os
import uuid
import datetime
import boto3
from boto3.dynamodb.conditions import Key
from dotenv import load_dotenv

# 載入環境變數
load_dotenv()
AWS_REGION = os.getenv("AWS_DEFAULT_REGION",
                        "us-east-1")
TABLE_NAME = os.getenv("DYNAMODB_TABLE",
                        "UserRecords")

# 建立 DynamoDB 資源
dynamodb = boto3.resource("dynamodb",
                           region_name=AWS_REGION)
table = dynamodb.Table(TABLE_NAME)

def insert_record(user_id: str, prompt: str, file_url: str,
                  record_type: str = "image", status: str = "success",
                  extra_meta: dict = None):
    """
    新增一筆紀錄到 DynamoDB
    """
    record_id = str(uuid.uuid4())
    created_at = datetime.datetime.utcnow().isoformat()

    item = {
        "user_id": user_id,
        "record_id": record_id,
        "prompt": prompt,
        "file_url": file_url,
        "created_at": created_at,
        "type": record_type,
        "status": status,
    }

    if extra_meta:
        item["extra_meta"] = extra_meta

    table.put_item(Item=item)
    return item
```

```
"""
def get_history(user_id: str):

    查詢某個使用者的所有紀錄

    resp = table.query(

        KeyConditionExpression=Key("user_id").eq(user_id)
    )
    return resp.get("Items", [])

def get_record(user_id: str, record_id: str):

    # 查詢單筆紀錄

    resp = table.get_item(
        Key={
            "user_id": user_id,
            "record_id": record_id
        }
    )
    return resp.get("Item")
"""
```

還是無法正常插入，也無法成功生成圖片

已思考 41s >

關鍵點有兩個，而且在你現在這份程式裡都看得到跡象：

1. 圖片沒生成成功（多半還是 Hugging Face 那段出錯）
2. insert_record 有機會根本沒跑到，或在 DynamoDB 那邊噴錯

我先把你貼的程式整理成「不會卡流程、比較正常的一版」，再跟你說哪裡最容易出問題，你照著檢查一下就能知道是哪一層壞掉。

✅ 建議你先把 `generate_image` 改成這一版

直接覆蓋你現在的 `/generate-image` 即可（保留其它 route 不動）：

python

複製程式碼

```
@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json or {}
        prompt = data.get("prompt", "沒有收到圖片prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少 prompt"}), 400

        #  呼叫 Hugging Face 生圖
        headers = {"Authorization": f"Bearer {HuggingFace_Token}"}
        payload = {"inputs": prompt}

        response = requests.post(HuggingFace_URL, headers=headers, json=payload,

        if response.status_code != 200:
            # 這裡就能看到 HF 真正錯誤 (例如 401)
            return jsonify({
                "success": False,
                "step": "huggingface",
                "error": f"Hugging Face API 錯誤: {response.status_code}",
                "details": response.text
            }), 500

        content_type = response.headers.get('Content-Type', '')

        if "image" not in content_type:
            return jsonify({
                "success": False,
                "step": "huggingface",
                "error": f"非預期的回傳格式: {content_type}",
                "details": response.text[:200] # 避免太長
            }), 500

        # (可選) 確認圖片能打開
        try:
            Image.open(io.BytesIO(response.content))
        except Exception as e:
            return jsonify({
                "success": False,
                "step": "image_open",
                "error": f"圖片解析失敗: {str(e)}"
```

```
    })), 500

# 2 存到 S3
timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = filename

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content,
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "step": "s3_put_object",
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

# 3 產 presigned URL (給 Discord 用)
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600
    )
except Exception as e:
    return jsonify({
        "success": False,
        "step": "s3_presign",
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

# 4 存進 DynamoDB (用 s3:// 路徑記錄)
try:
    record = insert_record(
        user_id=user_id,
        prompt=prompt,
```

```

        file_url=f"s3://{S3_BUCKET_NAME}/{s3_key}",
        record_type="image",
        status="success"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "step": "dynamodb_insert",
        "error": f"插入 DynamoDB 失敗: {str(e)}"
    }), 500

# 5 回傳給 Discord bot
return jsonify({
    "success": True,
    "download_url": presigned_url,
    "reply": f"Flask 收到圖片prompt: {prompt}",
    "record": record
})

except Exception as e:
    return jsonify({
        "success": False,
        "step": "unknown",
        "error": str(e)
    }), 500

```

這版跟你原本的差別：

- 不再在 Flask 裡用 `{"statusCode": ..., "body": ...}` 這種 Lambda 格式
- 每一段都加了 `step` 字段：`huggingface` / `s3_put_object` / `s3_presign` / `dynamodb_insert`
→ 你一看回傳 JSON 就知道卡在哪一步
- 先 `presigned_url` → 再 `insert_record` → 最後一次性 `jsonify` 回前端，不會「中途 return 跳掉下面程式」

為什麼「圖片沒生出來」的可能原因（你可以自己對）

看 `generate_image` 裡前半段：

python

 複製程式碼

```
response = requests.post(HuggingFace_URL, headers=headers, json=payload, timeout=

if response.status_code != 200:
    ...
```

最常看到的是：

- `status_code = 401` , `response.text = {"error": "Invalid username or password."}`
→ HUGGINGFACE_TOKEN 錯 / 沒權限
- `status_code = 403`
→ Token 存在，但帳號沒用這個 model 的權限 / 需要綁信用卡
- `status_code = 5xx`
→ Hugging Face 那邊服務本身有問題

你改成我上面的版本後，再測一次 `/generate-image` (不管是 Lambda Test 還是 Discord bot 打 API Gateway)，

看回傳 JSON 裡 `step` 和 `error` 就知道是 HuggingFace 出錯還是 S3 出錯。

🔍 為什麼「DynamoDB 插不進去」的常見原因

`insert_record` 本身寫得沒問題，最大的幾個可能：

1. Region 不對

- `AWS_DEFAULT_REGION` 沒設 → 你用預設 `us-east-1`
- 但你的 DynamoDB Table 可能建在別的 Region (例如 `ap-northeast-1`)
- 解法：
 - 確認 Table 的 Region
 - 把 Lambda 的環境變數 `AWS_DEFAULT_REGION` 設成一樣
 - 或在 `db.py` 把 `region_name` 改成實際 Region

2. Table 名稱不對

- 你現在預設 `TABLE_NAME = "UserRecords"`
- Console 裡 Table 也要叫 `UserRecords` (大小寫要一模一樣)

3. Key schema 不吻合

- 你插入的 item 有：

```
python
```

複製程式碼

```
"user_id": user_id,
"record_id": record_id,
```

- Table 至少要有 Partition key : `user_id` , Sort key : `record_id` (都是 String)
- 如果你 Table 只有一個 pk 叫其他名字 , 就不會照你想的方式查

4. Lambda Role 沒有 `dynamodb:PutItem` 權限

- 會在錯誤訊息裡看到 `AccessDeniedException`
- 這類錯誤現在會被我上面改過的 `step: "dynamodb_insert"` 和 `error` 字串直接丟回給前端

✓ 你接下來可以這樣測

1. 部署好這版 `app.py` 到 Lambda (重新 zip 上傳、Handler 還是走 `awscli` 那個)
2. 用 Lambda 的 Test event 或 API Gateway + Postman 打 :

json

複製程式碼

```
{
  "prompt": "a cute cat",
  "user_id": "112502029"
}
```

3. 看回傳 JSON :

- 如果是 :

json

複製程式碼

```
{
  "success": false,
  "step": "huggingface",
  "error": "Hugging Face API 錯誤: 401",
  "details": "..."
}
```

→ 處理 HuggingFace Token / 權限

- 如果是 :

json

複製程式碼

```
{
  "success": false,
  "step": "s3_put_object",
  "error": "上傳 S3 失敗: ..."
}
```

→ 檢查 S3 bucket 名稱、Lambda Role S3 權限

- 如果是：

json

複製程式碼

```
{
  "success": false,
  "step": "dynamodb_insert",
  "error": "插入 DynamoDB 失敗: ResourceNotFoundException: Requested resou
}
```

→ 檢查 Region / Table 名稱 / Key schema / 權限

- 如果是：

json

複製程式碼

```
{
  "success": true,
  "download_url": "...",
  "record": {...}
}
```

→ 圖片生成 + S3 上傳 + DynamoDB 插入全部成功 ✓

接下來 Discord bot 用 `download_url` 抓圖就會成功。

你先照我這版 `generate_image` 改掉原本那段，

再跑一次（透過 Lambda Test / API Gateway / Discord 任何一個入口），

看回來的 JSON 裡 `step` 是哪個，就知道「到底卡在 HuggingFace / S3 / DynamoDB 的哪一層」，

這樣你就不會只看到「無法成功」卻不知道是哪裡爆掉。

📄 🍷 🗨 📶 🔄 ...

s3://{S3_BUCKET_NAME}/{s3_key}
改成超連結

已思考 17s >

懂你的意思了～

你現在在 DynamoDB 裡存的是這樣一串字串：

python

 複製程式碼

```
f"s3://{S3_BUCKET_NAME}/{s3_key}"
```

這種 `s3://` 開頭只是「S3 內部路徑」，不是瀏覽器可以點的超連結。

如果想變成可以點的超連結，要改成 `https://...` 的 URL。

1 把 `s3://...` 改成 HTTPS 連結

在 `generate_image()` 裡這段：

python

 複製程式碼

```
result = insert_record(  
    user_id=user_id,  
    prompt=prompt,  
    file_url=f"s3://{S3_BUCKET_NAME}/{s3_key}",  
    record_type="image",  
    status="success"  
)
```

可以改成 S3 的 HTTP URL：

python

 複製程式碼

```
http_url = f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"  
  
result = insert_record(  
    user_id=user_id,  
    prompt=prompt,  
    file_url=http_url,  
    record_type="image",  
    status="success"  
)
```

這樣存進 DynamoDB 的 `file_url` 會長這樣：

text

 複製程式碼

```
https://你的-bucket-name.s3.amazonaws.com/20251126xxxx_abcd1234.png
```

在很多地方（像瀏覽器、Discord、某些後台系統）顯示出來就會是**可點擊的超連結**。

⚠ 前提：

這個連結要能直接開圖，物件要是 **public readable**，

否則點了還是會 403。

如果 bucket 沒開 public，http URL 只是「標記位置」，不能直接用。

2 也可以直接存「presigned URL」（馬上可點，但會過期）

你現在已經有產生 presigned URL：

python

複製程式碼

```
presigned_url = s3.generate_presigned_url(
    ClientMethod="get_object",
    Params={
        "Bucket": S3_BUCKET_NAME,
        "Key": s3_key
    },
    ExpiresIn=3600
)
```

如果你只是要「短時間內點得到」（作業 demo 用），

也可以直接把 **這個當成 file_url** 存進去：

python

複製程式碼

```
result = insert_record(
    user_id=user_id,
    prompt=prompt,
    file_url=presigned_url,  # 👉 直接存可用的下載連結
    record_type="image",
    status="success"
)
```

- 優點：存進去就馬上是可點、可下載的超連結
- 缺點：過了 3600 秒連結就失效

3 推薦你目前的做法

如果只是期末專題 demo、歷史紀錄要點一下就能看圖，我會建議：

- 回傳給 Discord bot 用的是 `presigned_url` (你現在已經這樣做 )
- DynamoDB 裡的 `file_url` 也先存 `presigned_url`
→ 用起來最直覺，不用再處理權限問題

也就是在 `generate_image()` 裡改成：

python

 複製程式碼

```
# 產生 presigned_url 之後：
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600
    )
except Exception as e:
    ...

# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=presigned_url,  #  這裡改成超連結
        record_type="image",
        status="success"
    )
except Exception as e:
    ...
```

之後你在 `/history` 或別的地方把 `file_url` 印出來，

Discord / 瀏覽器看到這個 `https://...`，就會變成可以直接點的超連結。

你可以先選一種 (`https://bucket.s3.amazonaws.com/key` 或 `presigned_url`) 改一條線跑跑看，

改完如果回傳的 JSON 結構有任何問題 (或點了還是 403) , 把錯誤訊息或回傳內容貼給我 , 我再幫你看是哪一段卡住。

📄 👍 💬 ↗️ ↺️ ...

s3要讓知道連結的人都能存取，怎麼設定

已思考 13s >

很好，現在就是在問：我要怎麼讓 S3 的檔案只要知道網址就能直接開？

其實有兩種做法，你現在的專題已經在用其中一種了：

1. ✅ 安全做法：用 presigned URL (已經在你程式裡)
2. ⚠️ 比較開、風險高的做法：把 S3 設成公開讀取 (任何人拿到 HTTPS 連結都能看)

我幫你分開講。

方案 1：維持現在，用「presigned URL」就好 (推薦)

你現在 Lambda 裡已經有：

python

📄 複製程式碼

```
presigned_url = s3.generate_presigned_url(  
    ClientMethod="get_object",  
    Params={  
        "Bucket": S3_BUCKET_NAME,  
        "Key": s3_key  
    },  
    ExpiresIn=3600 # 有效時間秒數  
)
```

這個網址特性是：

- 只有拿到這串連結的人可以存取 ✅
- 不用把 bucket 設成 public ✅
- 有時間限制 (例如 3600 秒後失效) ✅
- 連結長得像：

text

📄 複製程式碼

```
https://你的-bucket.s3.amazonaws.com/xxx.png?X-Amz-Algorithm=AWS4-HMAC-SHA256
```

如果你只要「知道連結的人可以看」，而且你的系統自己發給使用者，
這個其實就是 AWS 官方推薦的做法，不需要再改 S3 public 設定。

方案 2：把 S3 檔案設成「所有人都能用 HTTPS 連結直接開」（公開讀取）

如果你是想要這種：

- 用一個「乾淨」的網址，比如：

```
https://my-bucket.s3.amazonaws.com/20251126_abcd.png
```

- 任何人只要有這個網址，就可以在瀏覽器直接打開，不需要簽名、不會過期

那你要 真的把 bucket / objects 設成 public-read。步驟如下（以單一 bucket 為例）：

步驟 1：關閉 bucket 的「Block public access」

1. 進 AWS Console → S3 → 點你的 bucket
2. 點上方 tab：Permissions
3. 找到 Block public access (bucket settings)
4. 按 Edit
5. 把 Block all public access 相關勾勾都取消
6. 勾選底下「我了解會讓這個 bucket 變成 public」的確認框，再 Save

這步是「允許這個 bucket 有 public 存取的可能」，還沒真正公開。

步驟 2：加一條允許任何人讀取物件的 Bucket Policy

在同一個 Permissions 頁面往下滑，找到 Bucket policy → Edit

貼上類似這個（把 你的bucket名 改成你的）：

json

複製程式碼

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowPublicReadAllObjects",
```

```
{
  "Effect": "Allow",
  "Principal": "*",
  "Action": [
    "s3:GetObject"
  ],
  "Resource": [
    "arn:aws:s3:::你的bucket名/*"
  ]
}
```

存檔後，這個 bucket 底下所有物件的 **GET**（下載 / 查看）都對全世界開放。
只要知道連結，就能開。

步驟 3：確認 URL 可以直接開

1. 上傳一個測試檔案（或用你 Lambda 存好的檔案）
2. 找到那個 object → 點進去 → 你會看到 **Object URL**，類似：

text

 複製程式碼

`https://你的bucket名.s3.amazonaws.com/20251126123456_abcd1234.png`

3. 把這個 URL 貼到瀏覽器 / 貼給別人，看能不能不用登入 AWS 就開啟

安全提醒（真的很重要）

- 這樣設定後，任何人只要知道 URL 就能存取檔案（包括網路爬蟲、誤貼出去的連結）
- 如果你的圖片可能有敏感資料（例如使用者頭像、對話截圖），不建議直接開 public
- 有些學校/課程會要求不要把 S3 bucket 設成 public（會寫在作業要求或 policy 裡）

所以我會建議你：

- 作業 / demo 用：可以用 public bucket，簡單看結果 🍌
- 實際專案 / 上線：回到 **presigned URL** 的做法，
你現在的 Lambda 早就支援了，其實不需要讓 bucket 全開。

結合你現在的程式怎麼做最好？

你目前：

- Lambda 會把圖片存到 S3 
- 會產生 `presigned_url` 回給 Discord bot 
- `insert_record` 的 `file_url` 你打算存什麼？

如果你把 bucket 改 public-read，可以這樣存：

python

 複製程式碼

```
public_url = f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"

result = insert_record(
    user_id=user_id,
    prompt=prompt,
    file_url=public_url,  #  這就是一個可點可直接開的「超連結」
    record_type="image",
    status="success"
)
```

然後 `/history` 或之後前端顯示的時候，就可以直接顯示這個 `file_url`，
對方點下去就會在瀏覽器打開圖片。

你可以先決定：

 「我要繼續用 presigned URL (安全、但會過期)」

還是

 「為了展示方便，把 bucket 改 public-read，存 public URL」

如果你跟我說「bucket 名」和「你現在希望 history 裡看到的樣子」（例如要不要變成 `<超連結>`），我可以幫你順便把 `history` 的回傳設計好，讓你在 Discord `/history` 就看到可點的網址。

     ...

```
import os
import uuid
import datetime
import boto3
from boto3.dynamodb.conditions import Key
from dotenv import load_dotenv
```

```
# 載入環境變數
load_dotenv()
AWS_REGION = os.getenv("AWS_DEFAULT_REGION",
"us-east-1")
TABLE_NAME = os.getenv("DYNAMODB_TABLE",
"UserRecords")

# 建立 DynamoDB 資源
dynamodb = boto3.resource("dynamodb",
region_name=AWS_REGION)
table = dynamodb.Table(TABLE_NAME)

def insert_record(user_id: str, prompt: str, file_url: str,
record_type: str = "image", status: str = "success",
extra_meta: dict = None):
    """
    新增一筆紀錄到 DynamoDB
    """
    record_id = str(uuid.uuid4())
    created_at = datetime.datetime.utcnow().isoformat()

    item = {
        "user_id": user_id,
        "record_id": record_id,
        "prompt": prompt,
        "file_url": file_url,
        "created_at": created_at,
        "type": record_type,
        "status": status,
    }

    if extra_meta:
        item["extra_meta"] = extra_meta

    table.put_item(Item=item)
    return item

def get_history(user_id: str):

    # 查詢某個使用者的所有紀錄

    resp = table.query()
```



```

KeyConditionExpression=Key("user_id").eq(user_id)
    )
    return resp.get("Items", [])

def get_record(user_id: str, record_id: str):

    # 查詢單筆紀錄

    resp = table.get_item(
        Key={
            "user_id": user_id,
            "record_id": record_id
        }
    )
    return resp.get("Item")
get_history會回傳甚麼

```

`get_history(user_id)` 這個函式，會回傳一個「list of dict」，也就是：

- 型態：`List[Dict[str, Any]]`
- 內容：DynamoDB 裡 該 `user_id` 的所有紀錄，每一筆是一個 item (字典)

具體說，它做了什麼？

程式碼：

python

 複製程式碼

```

def get_history(user_id: str):
    # 查詢某個使用者的所有紀錄
    resp = table.query(
        KeyConditionExpression=Key("user_id").eq(user_id)
    )
    return resp.get("Items", [])

```

- `table.query(...)`：用 **Partition Key = user_id** 去查這個 table
- DynamoDB 回傳的 `resp` 是一個 dict，裡面有很多欄位，其中：
 - `resp["Items"]` 是一個 list
- `return resp.get("Items", [])`：
 - 如果有資料：回傳 `[item1, item2, ...]`

→ 如果沒有任何紀錄：回傳 `[]` (空 list)

每一個 item 長什麼樣？

因為你在 `insert_record` 裡是這樣存的：

python

 複製程式碼

```
item = {
    "user_id": user_id,
    "record_id": record_id,
    "prompt": prompt,
    "file_url": file_url,
    "created_at": created_at,
    "type": record_type,
    "status": status,
}
```

所以 `get_history("某個 user_id")` 得到的會大概像：

python

 複製程式碼

```
[
    {
        "user_id": "112502029",
        "record_id": "9c5b9b55-1b4f-4a0a-bfa2-123456789abc",
        "prompt": "一隻在宇宙漂浮的貓",
        "file_url": "https://你的-bucket.s3.amazonaws.com/20251126_xxxx.png",
        "created_at": "2025-11-26T16:30:12.345678",
        "type": "image",
        "status": "success"
    },
    {
        "user_id": "112502029",
        "record_id": "0a1b2c3d-....",
        "prompt": "一座雪山上的小木屋",
        "file_url": "...",
        "created_at": "...",
        "type": "image",
        "status": "success"
    },
]
```

```
...  
]
```

給你一個實際用法範例

如果你之後在 Flask 或 Discord bot 裡要用，可以這樣：

python

 複製程式碼

```
from db import get_history  
  
user_id = "112502029"  
items = get_history(user_id)  
  
for item in items:  
    print(item["prompt"], "→", item["file_url"])
```

或者在 `/history` API 直接 `jsonify({"records": items})`，Discord 那邊就可以拿到每一筆的 `prompt` 和 `file_url` 來排版顯示。

     ...

我要他只回傳fileurl跟prompt

可以～你現在的 `get_history` 是把整個 item 都丟回去，我幫你改成只回傳 `prompt` 跟 `file_url` 兩個欄位就好。

改好的 `get_history`

直接把原本的 `get_history` 換成這個版本：

python

 複製程式碼

```
def get_history(user_id: str):  
    """  
    查詢某個使用者的所有紀錄，只回傳 prompt 和 file_url  
    """  
    resp = table.query(  
        KeyConditionExpression=Key("user_id").eq(user_id)  
    )
```

```

items = resp.get("Items", [])

# 只保留 prompt 和 file_url
simple_items = [
    {
        "prompt": item.get("prompt"),
        "file_url": item.get("file_url"),
    }
    for item in items
]

return simple_items

```

這樣你在 Flask 或 Discord 裡呼叫：

python

 複製程式碼

```
records = get_history(user_id)
```

拿到的就會是像這樣的 list：

python

 複製程式碼

```

[
    {"prompt": "一隻在宇宙漂浮的貓", "file_url": "https://..."},
    {"prompt": "在雪山上的小木屋", "file_url": "https://..."},
    ...
]

```

如果你之後想再加 `created_at` 或 `record_id`，就再多塞一兩個欄位進那個 dict 就好 😊

     ...

```

from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import sqlite3
import requests
from PIL import Image
import awsgi
import boto3

```

```
from db import insert_record, get_history
import json

load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"

app = Flask(__name__)

s3 = boto3.client("s3") # 建立 s3 client
# dynamodb = boto3.resource("dynamodb")
# table = dynamodb.Table("UserRecords")

# 本地儲存目錄
# LOCAL_STORAGE = './generated_files'
# os.makedirs(f'{LOCAL_STORAGE}/images',
exist_ok=True)
# os.makedirs(f'{LOCAL_STORAGE}/audios',
exist_ok=True)

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json or {}
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少
prompt"}), 400

        headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
        payload = {"inputs": prompt}

        response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)
```

```
if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

content_type = response.headers.get('Content-
Type', '')

if 'application/json' in content_type:
    error_data = response.json()
    return jsonify({
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# 嘗試解析圖片
try:
    image =
Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

# 儲存到本地
# timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S
")
# filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
# local_path = f"
```

```
{LOCAL_STORAGE}/images/{filename}"
    # image.save(local_path, format="PNG")

    # 本地 URL
    # local_url =
    f"http://localhost:5000/generated_files/images/{filename}"

    # 存到 S3
    timestamp =
    datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"
    {timestamp}_{uuid.uuid4().hex[:8]}.png"
    s3_key = filename

    try:
        s3.put_object(
            Bucket=S3_BUCKET_NAME,
            Key=s3_key,
            Body=response.content,    # 直接用原始 bytes
            ContentType="image/png"
        )
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"上傳 S3 失敗: {str(e)}"
        }), 500

    public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
    local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"
    # 存進 DynamoDB
    try:
        result = insert_record(
            user_id=user_id,
            prompt=prompt,
            file_url=public_url,
            record_type="image",
            status="success"
        )
    except Exception as e:
```

```
        return jsonify({
            "statusCode": 500,
            "body": json.dumps({
                "message": "❌ 插入失敗",
                "error": str(e)
            })
        })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

# 寫入 SQLite
# insert_record(user_id, prompt, local_url)
"""
return jsonify({
    "success": True,
    # "local_url": local_url,
    "reply": f"Flask 收到圖片prompt: {prompt}"
})
"""

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    "reply": f"Flask 收到圖片prompt: {prompt}"
})

except Exception as e:
    return jsonify({
```



```
        "success": False,
        "error": str(e)
    }, 500

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片prompt")

        filename = "ta.txt"
        # 模擬生成圖片，先回傳本地 txt URL
        # local_url =
        f"http://localhost:5000/generated_files/test.txt"
        local_url =
        f"http://localhost:5000/generated_files/audios/{filename}"

        return jsonify({
            "success": True,
            "local_url": local_url,
            "reply": f"Flask 收到音樂prompt: {prompt}"
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

"""
# 呼叫索取生成結果(圖片/音樂)
@app.route('/generated_files/<file_type>/<filename>',
methods=['GET'])
def serve_file(file_type, filename):

    # file_type: 'images' 或 'audios'
    # filename: 檔案名稱

    # 驗證子資料夾
    if file_type not in ["images", "audios"]:
        return jsonify({"success": False, "error": "不支援的檔案類型"}), 400
```

```

        directory = os.path.join(LOCAL_STORAGE, file_type)
        return send_from_directory(directory, filename)
    """

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
        query 取得 user_id
        conn = sqlite3.connect("bot_records.db")
        c = conn.cursor()

        if user_id:
            c.execute("SELECT id, user_id, timestamp,
prompt, file_url FROM records WHERE user_id=?
ORDER BY id DESC", (user_id,))
        else:
            c.execute("SELECT id, user_id, timestamp,
prompt, file_url FROM records ORDER BY id DESC")

        rows = c.fetchall()
        conn.close()

        return jsonify({
            "success": True,
            "records": [
                {"id": r[0], "user_id": r[1], "timestamp": r[2],
"prompt": r[3], "file_url": r[4]}
                for r in rows
            ]
        })
    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)

```

改寫history的部分

好，現在你前面都已經改成了用 **DynamoDB** 的 `insert_record` / `get_history`，
`/history` 就沒必要再用本機的 `sqlite` 了，我幫你整段換成用 **DynamoDB** 的版本。

1 先把 `sqlite` 相關的東西拿掉

在檔案最上面這行可以刪掉：

```
python
```

[複製程式碼](#)

```
import sqlite3
```

`/history` 那段整個改掉，不要再開 `bot_records.db`。

2 新版 `/history` (用 `DynamoDB` 的 `get_history`)

因為你剛剛說 `get_history` 要只回傳 `prompt` 跟 `file_url`，
那我們就直接把它包成 API 回給前端 / Discord。

把原本的 `/history` 整段換成這個：

```
python
```

[複製程式碼](#)

```
@app.route("/history", methods=["GET"])
def history():
    try:
        # 從 query string 拿 user_id，例如 /history?user_id=123456789
        user_id = request.args.get("user_id")

        if not user_id:
            return jsonify({
                "success": False,
                "error": "缺少 user_id"
            }), 400

        # 呼叫 DynamoDB 的 get_history (只回傳 prompt + file_url)
        records = get_history(user_id)

        return jsonify({
            "success": True,
            "records": records  # 形式: [{ "prompt": "...", "file_url": "..."}],
        })

    except Exception as e:
        return jsonify({
            "success": False,
```

```
"error": str(e)
}), 500
```

這樣：

- 請求：

```
GET https://你的-api-url/history?user_id=112502029
```

- 回應格式會是：

json

 複製程式碼

```
{
  "success": true,
  "records": [
    {
      "prompt": "一隻在宇宙漂浮的貓",
      "file_url": "https://你的bucket.s3.amazonaws.com/20251126_xxxx.png"
    },
    {
      "prompt": "一隻在打電動的狗",
      "file_url": "https://你的bucket.s3.amazonaws.com/20251126_yyyy.png"
    }
  ]
}
```

3 如果要在 Discord `/history` 用

(給你一個小參考，之後要加回 Discord 指令時可以用)

python

 複製程式碼

```
@bot.command()
async def history(ctx):
    try:
        user_id = str(ctx.author.id)
        resp = requests.get(f"{FLASK_API_URL}/history", params={"user_id": user_id})
        data = resp.json()

        if not data.get("success"):
            await ctx.send(f"❌ 後端錯誤: {data.get('error', '未知錯誤')}")
            return
```

```

records = data.get("records", [])
if not records:
    await ctx.send("📄 你目前沒有任何生成紀錄。")
    return

# 簡單組字串
lines = []
for idx, r in enumerate(records, start=1):
    lines.append(f"{idx}. {r.get('prompt')} → {r.get('file_url')}")

msg = "\n".join(lines[:10]) # 避免太長，先顯示前 10 筆
await ctx.send(f"📖 你的歷史紀錄：\n{msg}")

except Exception as e:
    await ctx.send(f"❌ 查詢歷史失敗：{e}")

```

你現在可以先：

1. 換掉 `/history` 這段
2. 在 Lambda / API Gateway 上部署新的 ZIP
3. 用瀏覽器 or Postman 打一下：

text

複製程式碼

GET https://你的-api-url/history?user_id=某個實際有資料的 user_id

看看回傳是不是 `success: true`，`records` 裡有 `prompt` + `file_url`。

如果有出現錯誤訊息（像是 DynamoDB 相關的），把那個錯誤字串貼給我，我再幫你看是哪一段卡住。

📄 📖 📄 📄 📄 ...

```

import discord
from discord.ext import commands
import logging
from dotenv import load_dotenv
import os
import io
import aiohttp

```

```
import requests

load_dotenv()
DC_token
="MTQzNTI3MDk4NDc4NDU0Nzg0Mg.Glnp2b.JOi1Ke
8hUGHyH_W0tp2q-D0WCtFiDglyUCZi3U"
# DC_token = os.getenv('DISCORD_TOKEN')
# FLASK_API_URL = os.getenv('FLASK_API_URL',
'http://localhost:5000')
FLASK_API_URL = "https://xpapysqd2i.execute-api.us-
east-1.amazonaws.com/prod"

handler = logging.FileHandler(filename='discord.log',
encoding='utf-8', mode='w')
intent = discord.Intents.default()
intent.message_content = True
intent.members = True

bot = commands.Bot(command_prefix='/',
intents=intent)

@bot.event
async def on_ready():
    print(f"We are ready to go in, {bot.user.name}")

@bot.command()
async def hello(ctx):
    await ctx.send(f"Hello {ctx.author.mention}!")

@bot.command()
async def create_photo(ctx, *, prompt: str = "未輸入
prompt"):
    try:
        payload = {"prompt": prompt, "user_id":
str(ctx.author.id)}
        # response =
requests.post("http://localhost:5000/generate-image",
json=payload, timeout=60)
        response = requests.post(f"
{FLASK_API_URL}/generate-image", json=payload,
timeout=60)
        data = response.json()
```

```

"""
if not data.get("success"):
    err = data.get("error") or "未知錯誤"
    details = data.get("details")
    msg = f"❌ Flask 回傳錯誤: {err}"
    if details:
        msg += f"\n 📄 詳細: {details}"
    await ctx.send(msg)
    return
"""

if not data.get("success"):
    await ctx.send(f"❌ Flask 回傳錯誤:
{data.get('error', '未知錯誤')}")
    return

file_url = data.get("download_url")
if not file_url:
    await ctx.send("❌ Flask 沒有回傳檔案 URL")
    return

file_resp = requests.get(file_url)
if file_resp.status_code == 200:
    with io.BytesIO(file_resp.content) as f:
        await ctx.send(file=discord.File(f,
filename="generated.png"))
    else:
        await ctx.send("❌ 下載圖片失敗")

except Exception as e:
    await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")

"""

# /create_audio 指令
@bot.command()
async def create_audio(ctx, *, prompt: str = "未輸入
prompt"):
    # await ctx.send("這是生成音樂的功能")
    try:
        payload = {"prompt": prompt}
        response =
requests.post("http://localhost:5000/generate-audio",
json=payload, timeout=10)
        data = response.json()

```

```

    if not data.get("success"):
        await ctx.send(f"❌ Flask 回傳錯誤: {data.get('reply', '未知錯誤')}")
        return

    # 2 取得 local_url
    file_url = data.get("local_url")
    if not file_url:
        await ctx.send("❌ Flask 沒有回傳檔案 URL")
        return

    # 3 從 URL 下載檔案內容
    file_resp = requests.get(file_url)
    file_content = file_resp.text # 文字檔使用 text

    # 4 將內容回 Discord
    await ctx.send(f"✅ 從 Flask 取得檔案內容:\n{file_content}")

except Exception as e:
    await ctx.send(f"❌ 呼叫 Flask 失敗: {e}")
"""

@bot.command()
async def history(ctx):
    try:
        # 自動帶入 Discord 使用者 ID
        response = requests.get(f"{FLASK_API_URL}/history?user_id={ctx.author.id}",
                                timeout=10)
        data = response.json()

        if not data.get("success"):
            await ctx.send(f"❌ Flask 回傳錯誤: {data.get('error', '未知錯誤')}")
            return

        records = data.get("records", [])
        if not records:
            await ctx.send(f"🗨️ 使用者 {ctx.author.id} 沒有任何紀錄。")
            return

```



```
# 組合輸出
msg_lines = [f'{r["id"]}. {r["prompt"]} → {r["file_url"]}'
for r in records]
msg = "\n".join(msg_lines)

# 分段輸出避免超過 Discord 限制
if len(msg) > 1900:
    chunks = [msg[i:i+1900] for i in range(0,
len(msg), 1900)]
    for chunk in chunks:
        await ctx.send(f"📄 你的紀錄:\n{chunk}")
else:
    await ctx.send(f"📄 你的紀錄:\n{msg}")

except Exception as e:
    await ctx.send(f"❌ 查詢 DB 失敗: {e}")

bot.run(DC_token, log_handler=handler,
log_level=logging.DEBUG)
本地bot.py · history有沒有需要改
```

已思考 22s >

有，要改一點點，因為你後端 `/history` 換成 DynamoDB 版之後，回傳的 `records` 裡已經沒有 `id` 欄位了，只剩下（照我們前面設計）：

json

複製程式碼

```
{ "prompt": "...", "file_url": "..." }
```

但你現在 bot 這段還在用 `r['id']`：

python

複製程式碼

```
msg_lines = [f'{r["id"]}. {r["prompt"]} → {r["file_url"]}' for r in records]
```

這樣一跑就會 `KeyError: 'id'`。

✅ 建議修改後的 `/history` 指令

直接把原本的 `@bot.command() async def history(ctx):` 整段換成這個就好：

python

```

@bot.command()
async def history(ctx):
    try:
        # 自動帶入 Discord 使用者 ID
        params = {"user_id": str(ctx.author.id)}
        response = requests.get(f"{FLASK_API_URL}/history", params=params, timeout=10)
        data = response.json()

        if not data.get("success"):
            await ctx.send(f"❌ 後端回傳錯誤: {data.get('error', '未知錯誤')}")
            return

        records = data.get("records", [])
        if not records:
            await ctx.send("📄 你目前沒有任何生成紀錄。")
            return

        # records 形式: [{ "prompt": "...", "file_url": "..." }, ...]
        lines = []
        for idx, r in enumerate(records, start=1):
            prompt = r.get("prompt", "(無 prompt)")
            url = r.get("file_url", "(無網址)")
            lines.append(f"{idx}. {prompt} → {url}")

        msg = "\n".join(lines)

        # 分段輸出避免超過 Discord 限制
        if len(msg) > 1900:
            chunks = [msg[i:i+1900] for i in range(0, len(msg), 1900)]
            for chunk in chunks:
                await ctx.send(f"📄 你的紀錄: \n{chunk}")
        else:
            await ctx.send(f"📄 你的紀錄: \n{msg}")

    except Exception as e:
        await ctx.send(f"❌ 查詢歷史紀錄失敗: {e}")

```

這樣就跟我們剛剛改好的 Flask `/history` (回傳 `records: [{prompt, file_url}, ...]`) 對得上了。

順便提醒一下 (很重要 ⚠)

你現在在程式裡把 Discord bot token 直接寫死：

```
python
```

[複製程式碼](#)

```
DC_token = "MTQzNTI3MDk4NDc4NDU0Nzg0Mg.GInp2b.JOi1Ke8hUGHyH_W0tp2q-D0WCtFiDglyUCZi
```

這種 **真的 token** 一旦有貼到任何地方 (GitHub、作業上傳、截圖等)，

建議你立刻去 Discord Developer Portal：

1. 進你的 Application → Bot
2. Regenerate Token (重置 token)
3. 把新 token 放回 `.env`，程式改回：

```
python
```

[複製程式碼](#)

```
DC_token = os.getenv('DISCORD_TOKEN')
```

這樣比較安全，也比較不會被人亂用你的 bot。

[📄](#) [👍](#) [💬](#) [📌](#) [🔄](#) [...](#)

做一個test event

已思考 21s >

可以，幫你做兩個常用的 Lambda Test Event 範本：

1 測 **POST** `/generate-image`

2 測 **GET** `/history`

前提：你的 Lambda handler 是這樣 (awscli + Flask)：

```
python
```

[複製程式碼](#)

```
import awsgi
from app import app

def lambda_handler(event, context):
    return awsgi.response(app, event, context)
```

而 API Gateway 是 REST API，stage 是 `prod`（你的網址是 `.../prod`）。

1 Test Event：呼叫 `/generate-image`（POST）

在 Lambda Console → Test → Create new event

Event name 可以叫：`test_generate_image`

把內容改成下面這段 JSON：

json

複製程式碼

```
{
  "resource": "/generate-image",
  "path": "/prod/generate-image",
  "httpMethod": "POST",
  "headers": {
    "Content-Type": "application/json"
  },
  "multiValueHeaders": {
    "Content-Type": [
      "application/json"
    ]
  },
  "queryStringParameters": null,
  "multiValueQueryStringParameters": null,
  "pathParameters": null,
  "stageVariables": null,
  "requestContext": {
    "path": "/prod/generate-image",
    "stage": "prod",
    "identity": {
      "sourceIp": "127.0.0.1",
      "userAgent": "lambda-test-event"
    }
  },
  "body": "{\"prompt\":\"a cute cat in space\",\"user_id\":\"112502029\"}",
  "isBase64Encoded": false
}
```

重點：

- `httpMethod` 一定要是 `"POST"` → awsgi 會用這個

- `path / requestContext.path` 對應到你 Flask 裡的 `@app.route("/generate-image", methods=["POST"])`
- `body` 是字串形式的 JSON (外層要加 escape)

如果你的程式沒問題，執行後會看到類似：

json

 複製程式碼

```
{
  "statusCode": "200",
  "headers": { ... },
  "body": "{\"success\": true, \"download_url\": \"https://...\", ...}"
}
```

2 Test Event：呼叫 `/history` (GET)

再新增一個 event，名稱例如：`test_history`，內容：

json

 複製程式碼

```
{
  "resource": "/history",
  "path": "/prod/history",
  "httpMethod": "GET",
  "headers": {
    "Content-Type": "application/json"
  },
  "multiValueHeaders": {
    "Content-Type": [
      "application/json"
    ]
  },
  "queryStringParameters": {
    "user_id": "112502029"
  },
  "multiValueQueryStringParameters": {
    "user_id": [
      "112502029"
    ]
  },
  "pathParameters": null,
  "stageVariables": null,
}
```

```
"requestContext": {
  "path": "/prod/history",
  "stage": "prod",
  "identity": {
    "sourceIp": "127.0.0.1",
    "userAgent": "lambda-test-event"
  }
},
"body": null,
"isBase64Encoded": false
}
```

這會被 awsgi 轉成一次 Flask 的 GET `/history?user_id=112502029` 呼叫，
你的 Flask `/history` 路由就會拿到對應的 `user_id`，再去 DynamoDB 查資料。

如果你測 `test_generate_image` 出來是 `success: false`，
或 `step: ...` 的錯誤訊息（之前我們有加過），把那一段錯誤貼給我，我可以幫你直接看
是哪一層（HuggingFace / S3 / DynamoDB）出問題。

📄 🍌 🗨️ 🔼 ↺️ ...

Status: Succeeded

Test Event Name: test_3

Response:

```
{
  "statusCode": "404",
  "headers": {
    "Content-Type": "text/html; charset=utf-8",
    "Content-Length": "207"
  },
  "isBase64Encoded": false,
  "body": "<!doctype html>\n<html\nlang=en>\n<title>404 Not Found</title>\n<h1>Not\nFound</h1>\n<p>The requested URL was not found\non the server. If you entered the URL manually please\ncheck your spelling and try again.</p>\n"
```

The area below shows the last 4 KB of the execution log.

Function Logs:

START RequestId: 91822246-df3b-4945-8301-d6fe2d08bb9d Version: \$LATEST
END RequestId: 91822246-df3b-4945-8301-d6fe2d08bb9d
REPORT RequestId: 91822246-df3b-4945-8301-d6fe2d08bb9d Duration: 10.29 ms Billed Duration: 11 ms Memory Size: 128 MB Max Memory Used: 101 MB

Request ID: 91822246-df3b-4945-8301-d6fe2d08bb9d
這樣有正確收到紀錄嗎

已思考 17s >

短答：沒有，這次 Lambda 完全沒進到 `generate_image`，所以也不會寫進 DynamoDB。現在只是 Flask 回你一個標準的 404 HTML 頁面而已。

為什麼會 404？

關鍵在這個欄位（你現在的 test event 裡大概長這樣）：

json

複製程式碼

```
"path": "/prod/generate-image",
```

但你的 Flask route 是這樣寫的：

python

複製程式碼

```
@app.route("/generate-image", methods=["POST"])  
def generate_image():  
    ...
```

`awscli` 會把 event 裡的 `event["path"]` 當成 `PATH_INFO`，

所以 Flask 實際看到的網址是：

text

複製程式碼

```
/prod/generate-image
```

結果路由只有 `/generate-image`，當然就 404 了，因此：

✅ Lambda 有跑到

❌ 但 Flask route 沒 match → `generate_image()` 完全沒被呼叫 → 也沒有 S3 / DynamoDB 操作

修正：給你一個正確的 test event (呼叫 `/generate-image`)

在 Lambda 的 Test event 裡，把整個 payload 換成這個版本 (注意 `"path"`)：

json

複製程式碼

```
{
  "resource": "/generate-image",
  "path": "/generate-image",
  "httpMethod": "POST",
  "headers": {
    "Content-Type": "application/json"
  },
  "multiValueHeaders": {
    "Content-Type": [
      "application/json"
    ]
  },
  "queryStringParameters": null,
  "multiValueQueryStringParameters": null,
  "pathParameters": null,
  "stageVariables": null,
  "requestContext": {
    "path": "/generate-image",
    "stage": "prod",
    "identity": {
      "sourceIp": "127.0.0.1",
      "userAgent": "lambda-test-event"
    }
  },
  "body": "{\"prompt\":\"a cute cat in space\",\"user_id\":\"112502029\"}",
  "isBase64Encoded": false
}
```


差異就是：

- `path`：從 `/prod/generate-image` 改成 `/generate-image`
- `requestContext.path` 也跟著改成 `/generate-image`（這個對 `awscli` 其實不一定必要，但一起改最乾淨）

這樣：

- 如果 Hugging Face / S3 / DynamoDB 都正常 → 會回 `statusCode: "200"`，body 裡有 `success: true`、`download_url`
- 如果卡在某一步（例如 HF 401、S3 權限、DynamoDB 權限），會是 `success: false`，你程式裡的錯誤訊息會顯示出來

怎麼確認「有沒有真的寫進 DynamoDB」？

只要這次不是 404，而是有走到 `insert_record(...)`：

1. 在 Console → DynamoDB → Tables → `UserRecords` → **Explore

📄 🍴 🔍 ⬆️ ↺ ...

我是要測試history

啊原來你剛剛那個 404 是在測 `/history`，那更簡單 🙌

先回答你的問題：

這樣有正確收到紀錄嗎？

沒有。

`statusCode: 404` 是 Flask 回的「路由找不到」，

代表 Lambda → `awscli` 有跑，但完全沒進到 `/history` 這個 function，
所以更不可能去查 DynamoDB。

正確的 `/history` Test Event 範例

給你一個可以直接貼到 Lambda 的 test event（重點是 `path` 要是 `/history`，不是 `/prod/history`）：

json

📄 複製程式碼

```
{
  "resource": "/history",
  "path": "/history",
  "httpMethod": "GET",
  "headers": {
    "Content-Type": "application/json"
  },
  "multiValueHeaders": {
    "Content-Type": [
      "application/json"
    ]
  },
  "queryStringParameters": {
    "user_id": "112502029"
  },
  "multiValueQueryStringParameters": {
    "user_id": [
      "112502029"
    ]
  },
  "pathParameters": null,
  "stageVariables": null,
  "requestContext": {
    "path": "/prod/history",
    "stage": "prod",
    "identity": {
      "sourceIp": "127.0.0.1",
      "userAgent": "lambda-test-event"
    }
  },
  "body": null,
  "isBase64Encoded": false
}
```

說明一下幾個重點欄位：

- `"path": "/history"` 🖱️ 這個一定要跟 Flask 的路由一樣

```
python
```

📄 複製程式碼

```
@app.route("/history", methods=["GET"])
```

- `"httpMethod": "GET"` 🖱️ 因為你的是 GET

- `"queryStringParameters": { "user_id": "112502029" }`

對應到 Flask 裡的：

python

 複製程式碼

```
user_id = request.args.get("user_id")
```

`requestContext.path` 裡面的 `/prod/history` 是 API Gateway 那邊的 stage 路徑，
awscli 主要看的是 `path` 這個欄位，所以 `path` 必須是 `/history` 才會 match 到你的 route。

執行之後會看到什麼？

- 如果 DynamoDB 有資料，而且 `get_history(user_id)` 正常：

json

 複製程式碼

```
{
  "statusCode": "200",
  "body": "{\"success\": true, \"records\": [...]}"
}
```

- 如果你還沒插任何紀錄，會是：

json

 複製程式碼

```
{
  "success": true,
  "records": []
}
```

- 如果炸在 DynamoDB (例如 Table 名或 Region 錯) ：
你會看到：

json

 複製程式碼

```
{
  "success": false,
  "error": "某某 Exception 內容..."
}
```

你可以先用這個 `test_history` 跑一次，把 Response 裡的 `body` 打開看，
如果有錯誤字串 (例如 `ResourceNotFoundException` / `AccessDeniedException`)，直接貼

給我，我可以幫你看是 Table 設定還是權限的問題。



↪ 正確的 `/history` Test Event 範例

測試是沒錯，但實際在discord打/history，卻無法收到 prompt跟url，而是回傳 **✗** Flask 回傳錯誤: 未知錯誤

已思考 58s >

好，現在狀況我幫你整理一下：

- **✓** Lambda 的 `/history` 用 Test Event 測，是 OK 的（代表 Lambda + Flask + DynamoDB 都沒問題）
- **✗** 但在 Discord 打 `/history` 時，bot 收到的 HTTP 回應內容，
`data.get("success")` 是 `False` 或根本沒有 `success` 這個 key，所以進到這行：

python

複製程式碼

```
await ctx.send(f"✗ Flask 回傳錯誤: {data.get('error', '未知錯誤')}")
```

這超典型是 API Gateway 那一層沒設好 `/history` 這個路由，

導致 Discord 打 `https://.../prod/history?user_id=...` 時，APIGW 回的是像：

json

複製程式碼

```
{"message": "Missing Authentication Token"}
```

而不是你 Flask 回的：

json

複製程式碼

```
{"success": true, "records": [...]}
```

Lambda Test 直接叫 Lambda，不經過 API Gateway，所以當然是好的。

Discord 是打 HTTP URL，要經過 API Gateway，那邊如果沒有 `/history` 這個 method，就會錯。

一步一步解：為什麼「測試對」，但「Discord 不對」

1. Lambda Test :

你自己餵 event :

json

複製程式碼

```
{
  "path": "/history",
  "httpMethod": "GET",
  ...
}
```

→ 直接送到 Lambda handler → awsgi → Flask /history → DynamoDB → 正常回傳。

2. Discord :

bot 寫的是 :

python

複製程式碼

```
response = requests.get(f"{FLASK_API_URL}/history?user_id={ctx.author.id}", t
```

也就是打 :

text

複製程式碼

```
GET https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/history?user_
```

這時候 :

- 先經過 API Gateway
- 如果 APIGW 裡 沒有 /history 這個 Resource + Method (至少 GET)
→ APIGW 自己回一個 JSON : {"message":"Missing Authentication Token"}
- bot 收到後 response.json() 變成 :

python

複製程式碼

```
data = {"message": "Missing Authentication Token"}
```

- data.get("success") 是 None → 判定為失敗
- data.get("error", "未知錯誤") 取不到 error key → 用預設 "未知錯誤"
- 所以你看 :

❌ Flask 回傳錯誤: 未知錯誤

其實錯的是 **API Gateway** 沒把 `/history` 這條路接到 **Lambda**，不是 **Flask**。

要怎麼修：在 API Gateway 設定 `/history` 這個 endpoint

到 AWS Console → API Gateway → 找到你那個 REST API (對應到 `.../prod` 那個)：

1. 在左邊的 **Resources** 區塊，看看下面是不是只有 `/generate-image`，沒有 `/history`。
2. 如果沒有 `/history`：
 - 對 `/` (root) 按右鍵 → **Create Resource**
 - Resource Name: `history`
 - Resource Path: `/history`
3. 創好 `/history` 之後，選它 → **Create Method**
 - 選 **GET**
 - Integration type : **Lambda Function**
 - Use Lambda Proxy integration : 要勾 (跟 `/generate-image` 一樣)
 - Lambda Function : 選你這個 Flask + awsgi 的 Lambda
4. 設好後，點上面 **Actions** → **Deploy API**
 - 選 Stage : `prod`

這樣 `https://.../prod/history` 才會真的打進你的 Lambda，
然後 awsgi 才會把它轉給 Flask 的 `/history` route。

順便建議：在 bot 這邊加一點 debug (暫時)

你可以暫時把 Discord `/history` 指令改一下，印出後端回了什麼：

python

 複製程式碼

```
@bot.command()
async def history(ctx):
    try:
        response = requests.get(f"{FLASK_API_URL}/history?user_id={ctx.author.id}")

        # 暫時 debug : 先把 raw text 印出來看看
        # (可以先自己在終端 print，再決定要不要回到 Discord)
        print("history status:", response.status_code)
        print("history raw:", response.text)

        data = response.json()
```

```

if not data.get("success"):
    await ctx.send(f"❌ 後端回傳錯誤: {data.get('error', data)}")
    return

records = data.get("records", [])
if not records:
    await ctx.send(f"👤 使用者 {ctx.author.id} 沒有任何紀錄。")
    return

lines = []
for idx, r in enumerate(records, start=1):
    lines.append(f"{idx}. {r.get('prompt')} → {r.get('file_url')}")
msg = "\n".join(lines)

await ctx.send(f"📋 你的紀錄: \n{msg}")

except Exception as e:
    await ctx.send(f"❌ 查詢歷史紀錄失敗: {e}")

```

如果你在終端看到 `{"message": "Missing Authentication Token"}`，就 100% 確定是 API Gateway 那邊沒接好。

總結你現在要做的：

1. 確認 API Gateway 有 `/history` GET method，有連到正確的 Lambda，並重新 Deploy。
2. 再回 Discord 打 `/history` 看是不是就變成：
 - 有紀錄 → 列出 `prompt → file_url`
 - 沒紀錄 → 顯示「你目前沒有任何生成紀錄」

如果你在 APIGW 裡看到的 Resources 不確定怎麼設，你可以把那一頁的結構（有哪些 path）打字給我，我幫你看應該怎麼連。



為什麼history回傳的時候，會生成圖

好問題，這裡先講結論：

照你現在貼的程式結構來看，`/history` 這個 API 本身「不會」去生圖，唯一會生圖的是 `/generate-image`。

所以你看「打 `/history` 結果有產生圖片」，多半是下面其中一種情況：

1. 先確認後端 `/history` 裡到底有沒有生圖的程式碼

你現在的 `app.py`（精簡一下）：

python

複製程式碼

```
@app.route("/generate-image", methods=["POST"])
def generate_image():
    ...
    # 呼叫 HuggingFace 生圖
    response = requests.post(HuggingFace_URL, headers=headers, json=payload, time
    ...
    # 存 S3
    s3.put_object(...)
    ...
    # 存 DynamoDB insert_record(...)
    ...
    return jsonify({
        "success": True,
        "download_url": presigned_url,
        "reply": f"Flask 收到圖片prompt: {prompt}"
    })
```

這裡才有：

- 呼叫 HuggingFace
- 把圖存到 S3
- 存 DynamoDB

而你的 `/history` 現在應該是這樣（你要改成這一版）：

python

複製程式碼

```
@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id")
```



```

    if not user_id:
        return jsonify({
            "success": False,
            "error": "缺少 user_id"
        }), 400

    records = get_history(user_id) # 只會去 DynamoDB 撈資料

    return jsonify({
        "success": True,
        "records": records
    })

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500

```

👉 這個 `/history` 完全沒有 HuggingFace、沒有 S3 `put_object`，理論上不可能觸發「新生成一張圖」。

你可以在兩個 route 最前面加 `print` 來確認到底誰被呼叫：

python

 複製程式碼

```

@app.route("/generate-image", methods=["POST"])
def generate_image():
    print("[DEBUG] hit /generate-image")
    ...

@app.route("/history", methods=["GET"])
def history():
    print("[DEBUG] hit /history", request.args.get("user_id"))
    ...

```

然後去 CloudWatch 看 log。

你在 Discord 打 `/history` 的那一瞬間，log 裡到底印的是哪一行。

2. 很常見的情況：Discord 指令 & API Gateway 路徑搞混

你的 Discord `/history` 現在是這樣：

python

 複製程式碼

```
@bot.command()
async def history(ctx):
    try:
        response = requests.get(f"{FLASK_API_URL}/history?user_id={ctx.author.id}")
        data = response.json()
        ...
```

只會用 GET 去打 `/history`，不會幫你發圖。

只有 `/create_photo` 指令才會：

python

 複製程式碼

```
@bot.command()
async def create_photo(ctx, *, prompt: str = "未輸入prompt"):
    ...
    response = requests.post(f"{FLASK_API_URL}/generate-image", json=payload, tim
    ...
    file_resp = requests.get(file_url)
    if file_resp.status_code == 200:
        with io.BytesIO(file_resp.content) as f:
            await ctx.send(file=discord.File(f, filename="generated.png"))
```

👉 所以「在 Discord 看到圖片被貼出來」一定是 `/create_photo` 那個指令，
`/history` 正常情況只會傳一段文字訊息。

如果你說的是：

- 呼叫 `/history` 的時候，S3 裡多了一張新圖
- 或 HuggingFace 的使用量增加了一次

那就很可能是：

API Gateway 的路由設定有問題

例如：

- 你只建了一條 Resource：`/generate-image`
- `/history` 這個 path 沒有真正接到 Lambda

- 結果你在 APIGW 裡用了 proxy 或手動 mapping，把 `/history` 也錯誤地轉成 `/generate-image` 傳給 Lambda

這種狀況會變成：

- 用瀏覽器/Discord 打 `.../prod/history`
- API Gateway 收到 `/history`，但 Integration Request 寫死 path → `/generate-image`
- 傳給 Lambda 的 `event["path"]` 變成 `/generate-image`
- awsgi 看到 `/generate-image` → Flask `generate_image()` → 生圖 🚨

解法：

在 API Gateway 的 Console → 你的 API → **Resources** 看一下：

- `/generate-image` → 有 POST，Integration 是你的 Lambda
- `/history` → 要自己新增一條 GET，Integration 也是同一個 Lambda，但不要改 path override

如果你只設了一條 `/proxy+` 的 ANY + Lambda Proxy Integration，那就不用改 path，這時候 `event["path"]` 會是 `/prod/history`，awsgi 會傳給 Flask；如果你曾經手動改過 mapping template，要特別檢查。

3. 怎麼自己確認到底發生什麼事？

我建議你做兩個小 debug：

3-1. 在 `lambda_function.py` 印出 event

python

複製程式碼

```
import json
import awsgi
from app import app

def lambda_handler(event, context):
    print("[DEBUG] EVENT:", json.dumps(event))
    return awsgi.response(app, event, context)
```

然後：

1. 在 Discord 打 `/history`
2. 去 CloudWatch Logs 看剛剛那一筆 log 的 `EVENT` 裡面：
 - 看 `event["path"]` 寫什麼？

- 如果是 `/generate-image` → 代表 API Gateway 把路徑改掉了
- 正常要是 `/history` 或 `/prod/history`

3-2. 利用剛剛在 Flask route 加的 `print("[DEBUG] hit /...")`

這樣你就可以知道：

- `/history` 指令打進來時，實際執行的是 Flask 的 `/generate-image` 還是 `/history`

4. 總結你現在該檢查的東西

1. 先確認 `app.py` 裡 `/history` 沒有生圖的程式碼（照我上面那版改）
2. 在 `/generate-image` 和 `/history` 開頭加上 `print("[DEBUG] hit ...")`，重新部署
3. 在 `lambda_function.py` 印出 `event`，重新部署
4. 在 API Gateway Resources 裡確認：
 - `/generate-image` → POST → Lambda
 - `/history` → GET → Lambda
 - 沒有奇怪的 path override/mapping 把 `/history` 轉成 `/generate-image`
5. 再回 Discord 打 `/history`，看 CloudWatch：
 - `event["path"]` 是什麼？
 - 被執行的是哪一個 Flask route？

你可以先照這樣加 debug、測一次，把 CloudWatch 裡的 `EVENT` 和 `[DEBUG] hit ...` 那幾行 log（遮掉帳號 ID 就好）貼給我，我就可以直接幫你判斷：「是 API Gateway 路徑問題」還是「程式碼貼錯 route」。

📄 🍷 🔍 ↗️ ↻️ ...

我發現是只要輸入連結，discord會自動生成該連結的圖片

對，你發現的完全正確 🎯

那張「好像又生成出來」的圖片，其實不是你的程式在重生圖，
是 Discord 自己在幫「圖片連結」做預覽（embed / link preview）。

只要你在頻道裡貼出一個指向圖片的 URL，例如：

text

📄 複製程式碼

```
https://你的-bucket.s3.amazonaws.com/xxxx.png
```

Discord 會自動去抓那個連結的內容：

- 如果是圖片 → 顯示縮圖
- 如果是網頁 → 顯示標題、說明、縮圖

所以現在流程其實是：

1. 你 Lambda 真的只在 `/generate-image` 那次有 生圖 + 存 S3 
2. `/history` 只是把 DynamoDB 裡的 `file_url` (public S3 連結) 印出來
3. 你在 Discord 看到一條訊息有很多 `https://...png`
4. Discord 客戶端自己幫每個 URL 做「預覽卡片」，看起來就像一堆圖又被「生成」了，其實只是「被顯示出來」而已

如果你「不想要」Discord 自動預覽圖片，有幾種做法

方式 1：在程式裡抑制 embed (推薦)

discord.py `send` 有一個參數 `suppress_embeds=True`，

你可以在 history 指令那邊改成這樣：

python

 複製程式碼

```
@bot.command()
async def history(ctx):
    try:
        response = requests.get(f"{FLASK_API_URL}/history?user_id={ctx.author.id}")
        data = response.json()

        if not data.get("success"):
            await ctx.send(f"❌ 後端回傳錯誤: {data.get('error', '未知錯誤')}")
            return

        records = data.get("records", [])
        if not records:
            await ctx.send(f"🗿 使用者 {ctx.author.id} 沒有任何紀錄。")
            return

        lines = []
        for idx, r in enumerate(records, start=1):
            lines.append(f"{idx}. {r.get('prompt')} → {r.get('file_url')}")
        msg = "\n".join(lines)
```

```
# 關鍵：不要讓 Discord 展開連結成圖片
await ctx.send(f"📄 你的紀錄：\n{msg}", suppress_embeds=True)

except Exception as e:
    await ctx.send(f"❌ 查詢歷史紀錄失敗：{e}")
```

這樣就算訊息裡有 `https://...png`，Discord 也不會自動展開成圖片卡片。

注意：`suppress_embeds=True` 是 discord.py 新版才有的，如果出錯，跟我說一下錯誤訊息，我再幫你調整成另一種寫法（例如用 `<URL>` 的方式）。

方式 2：在文字上「手動阻止 embed」

如果你是人手動打訊息，可以用以下幾招：

- 用 `< >` 包住 URL：

text

複製程式碼

```
<https://你的-bucket.s3.amazonaws.com/xxx.png>
```

→ Discord 會把它當普通文字，不會展開預覽

- 或把連結放在 code block：

text

複製程式碼

```
`https://你的-bucket.s3.amazonaws.com/xxx.png`
```

→ 也不會有預覽卡片

方式 3：Discord 使用者設定（整體關掉預覽）

你自己也可以在 Discord 客戶端裡關掉「自動預覽」：

1. User Settings (左下角齒輪)
2. Text & Images / 文字與圖片
3. 把類似「顯示連結的網站預覽」的選項關掉

這樣你自己看訊息時就不會有連結預覽了（但別人還是會看到）。

總結一下

- ✓ Lambda / Flask `/generate-image` 才是「真正生圖」的地方
- ✓ `/history` 只是拿 DynamoDB 的 `file_url` 出來印
- 🖼️ 你看到「一貼 link 就出現圖片」，是 **Discord 幫你預覽 link**，不是你程式又跑去 HuggingFace 了
- 如果不想讓它預覽：
 - 在 bot `ctx.send(..., suppress_embeds=True)`
 - 或改訊息格式 (用 `<URL>` / code block)

你如果願意把現在 `/history` 指令的實際回傳訊息 (長怎樣) 貼一段給我看，我也可以順便幫你排版成「一行 prompt + 超連結」那種比較漂亮的格式 🍌

📄 🍌 🗨️ ⬆️ ↺️ ...

```
public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
```

圖片是存在 s3 bucket 的 folder 裡，url 怎麼改

已思考 26s >

對，你現在這行：

python

📄 複製程式碼

```
public_url = f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
```

是「檔案在 bucket 根目錄」時才會對，

如果你的圖是放在 S3 的某個 **folder** (其實是 **prefix**) 裡，就要把那個 folder 一起加進 `s3_key` 裡。

1 觀念先釐清：S3 的「folder」其實是 key 的一部分

在 S3 裡，如果你看到這種路徑：

- bucket : `my-bucket`
- folder : `images/`
- 檔名 : `20251126_abc.png`

實際上的 key 是：

```
text
```

 複製程式碼

```
images/20251126_abc.png
```

所以：

- 不是 bucket 有真的「資料夾物件」
- 而是物件 key 本身長得像 `images/...`、`images/cat/xxx.png`

2 正確寫法：在 `s3_key` 裡加上 folder

你現在的程式是：

```
python
```

 複製程式碼

```
timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = filename # 🙅 目前沒有資料夾

s3.put_object(
    Bucket=S3_BUCKET_NAME,
    Key=s3_key,
    Body=response.content,
    ContentType="image/png"
)

public_url = f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
```

如果你想存到 bucket 裡的 `images/` 這個 folder，請改成：

```
python
```

 複製程式碼

```
FOLDER_PREFIX = "images" # 你在 S3 看到的資料夾名稱

timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.png"

# ✅ key 裡包含資料夾
s3_key = f"{FOLDER_PREFIX}/{filename}"

# 上傳 S3
s3.put_object(
    Bucket=S3_BUCKET_NAME,
```



```

    Key=s3_key,
    Body=response.content,
    ContentType="image/png"
)

# 對外的 public 連結也用同一個 key
public_url = f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"

```

如果你的資料夾是多層，例如：`generated/images/`，那就：

```

python 複製程式碼

FOLDER_PREFIX = "generated/images"
s3_key = f"{FOLDER_PREFIX}/{filename}"
public_url = f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"

```

3 把同一個 `s3_key` 用在所有地方

之後建議你：

- `s3.put_object()` 用 `Key=s3_key`
- `public_url` 用 `.../{s3_key}`
- 存進 DynamoDB 的 `file_url` 用 `public_url`
- Discord `/history` 顯示的網址用 `file_url`

這樣你只要改一個 `FOLDER_PREFIX`，整條流水線（S3 檔案位置、URL、歷史紀錄）就會一起跟著對齊。

如果你願意把你現在 S3 上看到的實際路徑（例如：`my-bucket / ??? / 20251126_xxx.png`）打給我，我可以直接幫你寫出對應的 `s3_key` 和 `public_url` 寫法。

📄 🍴 🗣️ ⬆️ ↺ ...

```

@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    try:
        data = request.json
        suno_items = data["data"]["data"]

```

```
for item in suno_items:
    audio_url = item.get("source_audio_url")
    music_id = item["id"]

    # 下載音檔
    mp3 = requests.get(audio_url).content

    # 上傳 S3
    timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
    s3_key = f"Audios/{filename}"

    try:
        s3.put_object(
            Bucket=S3_BUCKET_NAME,
            Key=s3_key,
            Body=response.content,    # 直接用原始
bytes
            ContentType="audio/png"
        )
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"上傳 S3 失敗: {str(e)}"
        }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"
# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="image",
        status="success"
    )
except Exception as e:
```

```

return jsonify({
    "statusCode": 500,
    "body": json.dumps({
        "message": "✖ 插入失敗",
        "error": str(e)
    })
})
# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({"success": True})

except Exception as e:
    print("callback error:", e)
    return jsonify({"error": str(e)}), 500
s3要存的是這個音檔的url，s3 put object要怎麼寫

```

已思考 23s >

先說重點：

S3 不會「存 URL」本身，它是存「檔案內容」，然後你再用 `Bucket + Key` 組出這個檔案的 URL。

你現在已經有音檔的 bytes 在 `mp3` 變數裡了，所以 `put_object` 應該把 `Body` 設成 `mp3`，而不是 `response.content`，`ContentType` 也要改成音檔的 MIME type。

我幫你把整段比較重要的地方改正、標重點 📌

1. 正確的 `s3.put_object` 寫法 (音檔)

你原本這段有幾個問題：

```
python

mp3 = requests.get(audio_url).content

s3.put_object(
    Bucket=S3_BUCKET_NAME,
    Key=s3_key,
    Body=response.content,      # ❌ 這裡應該用 mp3
    ContentType="audio/png"    # ❌ 這是圖片的 MIME type
)
```

 複製程式碼

改成這樣：

```
python

mp3 = requests.get(audio_url).content # 這裡拿到音檔的 bytes

s3.put_object(
    Bucket=S3_BUCKET_NAME,
    Key=s3_key,                    # 例如 Audios/20251126_xxxx.mp3
    Body=mp3,                     # ✅ 上傳音檔 bytes
    ContentType="audio/mpeg"      # ✅ 一般 mp3 的 MIME type (或 audio/mp3)
)
```

 複製程式碼

這樣 S3 裡就會真的有一個 `Audios/...mp3` 的物件。

接著你要存到 DynamoDB 的「URL」可以這樣組：

```
python

public_url = f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
```

 複製程式碼

這個 `public_url` 才是你要存進 `insert_record` 的「音檔連結」。

2. 幫你整理一版比較完整、修正後的 `audio_callback`

我先針對 S3 + DynamoDB 部分修好 (`user_id` / `prompt` 要你依實際資料來源填)：

```
python
```

 複製程式碼

```
@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    try:
        data = request.json
        suno_items = data["data"]["data"]

        # TODO: 看你的 callback payload 裡到底有沒有 user_id / prompt
        # 這裡先用預設值，避免 NameError
        user_id = data.get("user_id", "unknown")
        prompt = data.get("prompt", "music from Suno")

        for item in suno_items:
            audio_url = item.get("source_audio_url")
            music_id = item["id"]

            if not audio_url:
                continue # 沒有 audio_url 就跳過

            # 下載音檔
            mp3 = requests.get(audio_url).content

            # 上傳 S3
            timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
            filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
            s3_key = f"Audios/{filename}" # 存在 Audios/ 資料夾裡

            try:
                s3.put_object(
                    Bucket=S3_BUCKET_NAME,
                    Key=s3_key,
                    Body=mp3, # ✅ 用音檔 bytes
                    ContentType="audio/mpeg" # ✅ mp3 MIME type
                )
            except Exception as e:
                return jsonify({
                    "success": False,
                    "error": f"上傳 S3 失敗: {str(e)}"
                }), 500

            # S3 上的公開 URL (如果 bucket 有設 public-read)
            public_url = f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
            local_url = f"s3://{S3_BUCKET_NAME}/{s3_key}"
```

```

# 存進 DynamoDB (這裡記得 record_type 用 audio 比較合理)
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="audio",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })

# 如果你還是想要 presigned URL (有時效性的下載連結)
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({"success": True})

except Exception as e:
    print("callback error:", e)
    return jsonify({"error": str(e)}), 500

```

這裡幾個地方你要特別注意：

1. `Body` 用 `mp3`，不是 `response.content`
`response` 這個變數在這個 `function` 裡根本沒定義，所以會 `NameError`。
2. `ContentType` 一定要改成音檔類型 `audio/mpeg`
「`audio/png`」這種不存在，`PNG` 是圖片。
3. `DynamoDB record_type` 應該用 `"audio"` 而不是 `"image"`
之後 `/history` 如果要分開顯示圖片 / 音樂，可以靠 `type` 判斷。
4. `user_id` 跟 `prompt` 在這個 `callback` 裡目前沒有來源
你現在的 `code` 直接用 `user_id / prompt` 會噴 `NameError`，
要嘛從 `data` 裡拿，要嘛給預設值（上面先幫你放 `data.get(...)`）。

3. 再說一次：S3 不是「存 URL」，是「存檔案」

流程是這樣：

1. 你從第三方（Suno）拿到 `audio_url`
2. 用 `requests.get(audio_url).content` 把實際的 `mp3` 檔案抓下來（`bytes`）
3. 用 `s3.put_object(Body=mp3, ...)` 把 `bytes` 存進 S3
4. 把 `bucket + key` 組成一個 HTTP 連結：
`https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}`
5. 這個「公開連結」才是你要存進 `DynamoDB` / 回給 `Discord` 的 URL

如果你等一下貼 Suno 真實 `callback payload` 給我（把敏感資訊遮掉），

我也可以幫你把 `user_id` 和 `prompt` 在這個 `callback` 裡接起來，一路存進 `DynamoDB`。

📄 🍌 🗨️ 📶 🔄 ...

給我 `audio_call` 的 `test event`

沒問題，我幫你做一個可以直接丟進 `Lambda Console` 的 `audio_callback` 測試事件 🍌

你的後端期望的 `body` 結構是：

python

📄 複製程式碼

```
data = request.json
suno_items = data["data"]["data"]
# 裡面的 item 要有 id、source_audio_url
```

所以 Test Event 可以這樣寫：

json

 複製程式碼

```
{
  "resource": "/audio-callback",
  "path": "/audio-callback",
  "httpMethod": "POST",
  "headers": {
    "Content-Type": "application/json"
  },
  "multiValueHeaders": {
    "Content-Type": [
      "application/json"
    ]
  },
  "queryStringParameters": null,
  "multiValueQueryStringParameters": null,
  "pathParameters": null,
  "stageVariables": null,
  "requestContext": {
    "path": "/audio-callback",
    "stage": "prod",
    "identity": {
      "sourceIp": "127.0.0.1",
      "userAgent": "lambda-test-event"
    }
  },
  "body": "{ \"user_id\": \"112502029\", \"prompt\": \"test music from suno\", \"isBase64Encoded\": false"
}
```

你要改的只有兩個地方

- `"user_id": "112502029"`：可以換成你實際想測的 user id
- `"source_audio_url": "https://www.example.com/test.mp3"`：
這裡要改成一個 **真的可以下載 mp3 的網址**（或你 Suno 回傳的 URL）

這樣丟進 Lambda 測試後：

- 會進到 `/audio-callback`
- 下載那個 `source_audio_url` 的 mp3
- 上傳到 S3 的 `Audios/xxxx.mp3`

- 呼叫 `insert_record(...)` 存進 DynamoDB

如果跑完之後 Lambda 回 `success: true` ,

你就可以去 S3 / DynamoDB Console 確認檔案 & 紀錄有沒有進去了。



```
{
  "code": 200,
  "data": {
    "callbackType": "complete",
    "data": [
      {
        "audio_url": "https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MGMtNDZlY2JiNzg2ZDAz.mp3",
        "createTime": 1763910096539,
        "duration": 107.92,
        "id": "258d08b5-42f6-4d1a-850c-46ecbb786d03",
        "image_url": "https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MGMtNDZlY2JiNzg2ZDAz.jpeg",
        "model_name": "chirp-v3-5",
        "prompt": "",
        "source_audio_url": "https://cdn1.suno.ai/258d08b5-42f6-4d1a-850c-46ecbb786d03.mp3",
        "source_image_url": "https://cdn2.suno.ai/image_258d08b5-42f6-4d1a-850c-46ecbb786d03.jpeg",
        "source_stream_audio_url": "https://cdn1.suno.ai/258d08b5-42f6-4d1a-850c-46ecbb786d03.mp3",
        "stream_audio_url": "https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MGMtNDZlY2JiNzg2ZDAz",
        "tags": "古典",
        "title": "宁静钢琴冥想"
      },
      {
        "audio_url": "https://musicfile.api.box/YWVjYzIxYzAtNDk1MC00MTVmLTg1ODUtODFmNzRjNDkxZGRj.mp3",
        "createTime": 1763910096539,
        "duration": 168.36,
        "id": "aecc91c0-4950-415f-8585-81f74c491ddc",
        "image_url": "https://musicfile.api.box/YWVjYzIxYzAtNDk1MC00MTVmLTg1ODUtODFmNzRjNDkxZGRj.jpeg",
        "model_name": "chirp-v3-5",
        "prompt": "",
        "source_audio_url": "https://cdn1.suno.ai/aecc91c0-4950-415f-8585-81f74c491ddc.mp3",
        "source_image_url": "https://cdn2.suno.ai/image_aecc91c0-4950-415f-8585-81f74c491ddc.jpeg",
        "source_stream_audio_url": "https://cdn1.suno.ai/aecc91c0-4950-415f-8585-81f74c491ddc.mp3",
        "stream_audio_url": "https://musicfile.api.box/YWVjYzIxYzAtNDk1MC00MTVmLTg1ODUtODFmNzRjNDkxZGRj",
        "tags": "古典",
        "title": "宁静钢琴冥想"
      }
    ]
  },
  "task_id": "2b00f76847c403fe9051b70c95bc27da"
}
```

```
"msg": "All generated successfully."}
這是suno回傳的json
```

已思考 1m 33s >

讚，你有把 Suno 真實 callback 貼出來，這樣好處理很多 🍌

你那段 Suno JSON 長這樣（整理一下換行而已）：

json

複製程式碼

```
{
  "code": 200,
  "data": {
    "callbackType": "complete",
    "data": [
      {
        "audio_url": "https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MGMtN",
        "createTime": 1763910096539,
        "duration": 107.92,
        "id": "258d08b5-42f6-4d1a-850c-46ecbb786d03",
        "image_url": "https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MGMtN",
        "model_name": "chirp-v3-5",
        "prompt": "",
        "source_audio_url": "https://cdn1.suno.ai/258d08b5-42f6-4d1a-850c-46ecbb7",
        "source_image_url": "https://cdn2.suno.ai/image_258d08b5-42f6-4d1a-850c-4",
        "source_stream_audio_url": "https://cdn1.suno.ai/258d08b5-42f6-4d1a-850c-",
        "stream_audio_url": "https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLT",
        "tags": "古典",
        "title": "宁静钢琴冥想"
      },
      {
        "audio_url": "https://musicfile.api.box/YWVjYzIxYzAtNDk1MC00MTVmLTg1ODUtO",
        "createTime": 1763910096539,
        "duration": 168.36,
        "id": "aecc91c0-4950-415f-8585-81f74c491ddc",
        "image_url": "https://musicfile.api.box/YWVjYzIxYzAtNDk1MC00MTVmLTg1ODUtO",
        "model_name": "chirp-v3-5",
        "prompt": "",
        "source_audio_url": "https://cdn1.suno.ai/aecc91c0-4950-415f-8585-81f74c4",
        "source_image_url": "https://cdn2.suno.ai/image_aecc91c0-4950-415f-8585-8",
        "source_stream_audio_url": "https://cdn1.suno.ai/aecc91c0-4950-415f-8585-
```

```
    "stream_audio_url": "https://musicfile.api.box/YWVjYzIxYzAtNDk1MC00MTVmLT",
    "tags": "古典",
    "title": "宁静钢琴冥想"
  }
],
"task_id": "2b00f76847c403fe9051b70c95bc27da"
},
"msg": "All generated successfully."
}
```

後端的程式是這樣讀的：

python

 複製程式碼

```
data = request.json
suno_items = data["data"]["data"]
```

✅ 對應完全正確（外層的 `"data"` 裡面有一個 `"data"` 陣列）。

✅ Lambda 的 audio_callback 測試事件（給你直接貼）

在 Lambda Console → Test → Create new event

Event JSON 可以像這樣（重點是 `path`、`httpMethod`、`body`）：

json

 複製程式碼

```
{
  "resource": "/audio-callback",
  "path": "/audio-callback",
  "httpMethod": "POST",
  "headers": {
    "Content-Type": "application/json"
  },
  "multiValueHeaders": {
    "Content-Type": [
      "application/json"
    ]
  },
  "queryStringParameters": null,
  "multiValueQueryStringParameters": null,
  "pathParameters": null,
  "stageVariables": null,
```

```

"requestContext": {
  "path": "/audio-callback",
  "stage": "prod",
  "identity": {
    "sourceIp": "127.0.0.1",
    "userAgent": "lambda-test-event"
  }
},
"body": "{\"code\":200,\"data\":{\"callbackType\":\"complete\", \"data\": [{\"aud
\"isBase64Encoded\": false
}

```

這裡幾個關鍵：

- `path: "/audio-callback"` 👉 要跟 Flask 路由一樣：

```
python
```

複製程式碼

```
@app.route('/audio-callback', methods=['POST'])
```

- `httpMethod: "POST"` 👉 callback 通常都是 POST
- `body` 是「字串形式」的 JSON，所以裡面所有 `"` 都變成 `\` (我已經幫你跳脫好了，你可以整段複製)

🔍 後端 audio_callback 邏輯再確認一下

你現在的後端邏輯大概是這樣：

```
python
```

複製程式碼

```

@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    data = request.json
    suno_items = data["data"]["data"] # 這裡就會拿到你貼的那兩個 item

    for item in suno_items:
        audio_url = item.get("source_audio_url")
        music_id = item["id"]
        mp3 = requests.get(audio_url).content

        timestamp = ...
        filename = ...

```

```
s3_key = f"Audios/{filename}"

s3.put_object(
    Bucket=S3_BUCKET_NAME,
    Key=s3_key,
    Body=mp3,
    ContentType="audio/mpeg"
)

public_url = f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"

insert_record(..., file_url=public_url, record_type="audio", ...)
```

👉 用我給你的 test event 跑：

- 如果成功：

Lambda 回應應該是：

json

📋 複製程式碼

```
{ "success": true }
```

然後：

- S3 的 Audios/ 底下會多兩個 mp3
- DynamoDB UserRecords (或你設定的 TABLE_NAME) 裡會多兩筆 type = audio 記錄
- 如果失敗 (例如：S3 權限、DynamoDB 權限)：

你會在 Lambda Response 裡看到：

json

📋 複製程式碼

```
{
  "success": false,
  "error": "某某 Exception ..."
}
```

把那段錯誤貼給我，我可以幫你看是哪一層 (S3 / DynamoDB / 程式碼) 卡住。

如果你想下一步讓 Discord bot 也能 /create_audio 然後拿到這些 Suno + S3 的 mp3 URL，我也可以幫你把那一條串起來 (包含 history 裡區分 image / audio)。

```
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import requests
from PIL import Image
import awsgi
import boto3
from db import insert_record, get_history
import json

load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"

app = Flask(__name__)

s3 = boto3.client("s3") # 建立 s3 client
# dynamodb = boto3.resource("dynamodb")
# table = dynamodb.Table("UserRecords")

# 本地儲存目錄
# LOCAL_STORAGE = './generated_files'
# os.makedirs(f'{LOCAL_STORAGE}/images',
exist_ok=True)
# os.makedirs(f'{LOCAL_STORAGE}/audios',
exist_ok=True)

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")
```

```
if not prompt:
    return jsonify({"success": False, "error": "缺少
prompt"}), 400

headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
payload = {"inputs": prompt}

response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

content_type = response.headers.get('Content-
Type', '')

if 'application/json' in content_type:
    error_data = response.json()
    return jsonify({
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# 嘗試解析圖片
try:
    image =
Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
```

```
"success": False,
"error": f"圖片解析失敗: {str(e)}"
}), 500

# 儲存到本地
# timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
# filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
# local_path = f"
{LOCAL_STORAGE}/images/{filename}"
# image.save(local_path, format="PNG")

# 本地 URL
# local_url =
f"http://localhost:5000/generated_files/images/{filename}"

# 存到 S3
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = f"Images/{filename}"

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content,    # 直接用原始 bytes
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
```



```
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"
# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="image",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

# 寫入 SQLite
# insert_record(user_id, prompt, local_url)
"""
return jsonify({
    "success": True,
    # "local_url": local_url,
    "reply": f"Flask 收到圖片prompt: {prompt}"
})
```

```
"""  
    return jsonify(  
        "success": True,  
        "download_url": presigned_url, # 👉 這是 S3  
presigned URL  
        "reply": f"Flask 收到圖片prompt: {prompt}"  
    )  
  
    except Exception as e:  
        return jsonify(  
            "success": False,  
            "error": str(e)  
        ), 500  
  
@app.route('/generate-audio', methods=['POST'])  
def generate_audio():  
    try:  
        print("=== /generate-audio 被呼叫 ===")  
  
        data = request.json  
        print("收到的 JSON:", data)  
  
        prompt = data.get('prompt')  
        # discord_channel_id =  
data.get('discord_channel_id')  
        # discord_user_id = data.get('discord_user_id')  
  
        if not prompt:  
            print("❌ 缺少 prompt")  
            return jsonify({"success": False, "error": "缺少  
prompt"}), 400  
  
        payload = {  
            "prompt": prompt,  
            "style": "古典",  
            "title": "AI Generated Music",  
            "customMode": True,  
            "instrumental": True,  
            "model": "V3_5",  
  
            # ★ 改成 API Gateway callback  
            "callBackUrl": "https://xpapysqd2i.execute-
```

```
api.us-east-1.amazonaws.com/prod/audio-callback"
}

print("Suno payload:", payload)

headers = {
    "Authorization": f"Bearer {SUNO_TOKEN}",
    "Content-Type": "application/json"
}

print("正在呼叫 Suno API...")
response = requests.post(SUNO_URL,
    json=payload, headers=headers, timeout=60)

print("Suno 回應狀態:", response.status_code)
print("Suno 回應內容:", response.text)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Suno API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

suno_resp = response.json()
print("解析後 Suno JSON:", suno_resp)

# ★ 抓 taskId ( Suno 回傳格式有時不同 )
suno_request_id = (
    suno_resp.get("id")
    or suno_resp.get("requestId")
    or suno_resp.get("data", {}).get("taskId")
)

if not suno_request_id:
    print("✗ Suno 沒有回傳 taskId")
    return jsonify({
        "success": False,
        "error": "Suno 未回傳 taskId"
    }), 500

# 回傳給 Discord bot，可以用來顯示“開始生成中”
```

```
        return jsonify({
            "success": True,
            "task_id": suno_request_id,
            "message": "音樂生成中"
        })

    except Exception as e:
        print("✖ Flask 在 /generate-audio 發生錯誤:", e)
        return jsonify({"success": False, "error": str(e)}), 500

@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    try:
        data = request.json
        suno_items = data["data"]["data"]

        for item in suno_items:
            audio_url = item.get("source_audio_url")
            music_id = item["id"]

            if not audio_url:
                print("✖ 沒有 audio_url")
                return jsonify({
                    "success": False,
                    "error": "Suno 沒有回傳 audio_url"
                }), 500

            # 下載音檔
            mp3 = requests.get(audio_url).content

            # 上傳 S3
            timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
            filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
            s3_key = f"Audios/{filename}"

            try:
                s3.put_object(
                    Bucket=S3_BUCKET_NAME,
                    Key=s3_key,
                    Body=mp3,    # 直接用原始 bytes
```

```
        ContentType="audio/mpeg"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"

# 存進 DynamoDB
try:
    result = insert_record(
        user_id="no user id",
        prompt="no prompt",
        file_url=public_url,
        record_type="audio",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
```

```

        "error": f"產生下載網址失敗: {str(e)}"
    }, 500

    return jsonify({
        "success": True,
        "download_url": presigned_url, # 👉 這是 S3
        "reply": f"Flask 收到音樂prompt: {prompt}"
    })

except Exception as e:
    print("callback error:", e)
    return jsonify({"error": str(e)}), 500

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
        query 取得 user_id

        records = get_history(user_id)

        return jsonify({
            "success": True,
            "records": records # 形式 : [{ "prompt": "...",
            "file_url": "... } , ...]
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)
    我要generate_audio的test event

```

已思考 18s >

好，直接給你可以丟進 Lambda 的 `generate-audio` 測試事件 JSON 👉

json

複製程式碼

```
{
  "resource": "/generate-audio",
  "path": "/generate-audio",
  "httpMethod": "POST",
  "headers": {
    "Content-Type": "application/json"
  },
  "multiValueHeaders": {
    "Content-Type": [
      "application/json"
    ]
  },
  "queryStringParameters": null,
  "multiValueQueryStringParameters": null,
  "pathParameters": null,
  "stageVariables": null,
  "requestContext": {
    "path": "/generate-audio",
    "stage": "prod",
    "identity": {
      "sourceIp": "127.0.0.1",
      "userAgent": "lambda-test-event"
    }
  },
  "body": "{ \"prompt\": \"test music from lambda\" }",
  "isBase64Encoded": false
}
```

這個 event 打進 awsgi → Flask 時，`request.json` 會是：

python

 複製程式碼

```
{"prompt": "test music from lambda"}
```

你的 `/generate-audio` 就會：

1. 收到 `prompt`
2. 組 Suno payload (帶 callback URL)
3. 呼叫 Suno API
4. 回傳 `{"success": True, "task_id": ..., "message": "音樂生成中"}` (如果 `SUNO_TOKEN` 和 `SUNO_URL` 都正確)

如果你之後有加上 `discord_user_id` 、 `discord_channel_id` 之類的欄位，只要在 `body` 裡多放就好，例如：

json

 複製程式碼

```
"body": "{ \"prompt\": \"test music from lambda\", \"discord_user_id\": \"1234567\" }
```



```
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import requests
from PIL import Image
import awsgi
import boto3
from db import insert_record, get_history
import json

load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"
SUNO_URL =
"https://api.sunoapi.org/api/v1/generate"

app = Flask(__name__)

s3 = boto3.client("s3") # 建立 s3 client
# dynamodb = boto3.resource("dynamodb")
# table = dynamodb.Table("UserRecords")

# 本地儲存目錄
# LOCAL_STORAGE = './generated_files'
# os.makedirs(f'{LOCAL_STORAGE}/images',
```



```
exist_ok=True)
# os.makedirs(f'{LOCAL_STORAGE}/audios',
exist_ok=True)

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少
prompt"}), 400

        headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
        payload = {"inputs": prompt}

        response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)

        if response.status_code != 200:
            return jsonify({
                "success": False,
                "error": f"Hugging Face API 錯誤:
{response.status_code}",
                "details": response.text
            }, 500)

        content_type = response.headers.get('Content-
Type', '')

        if 'application/json' in content_type:
            error_data = response.json()
            return jsonify({
                "success": False,
                "error": "Hugging Face 回傳錯誤",
                "details": error_data
            }, 500)

        if 'image' not in content_type:
```

```
        return jsonify({
            "success": False,
            "error": f"非預期的回傳格式: {content_type}"
        }), 500

    # 嘗試解析圖片
    try:
        image =
Image.open(io.BytesIO(response.content))
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"圖片解析失敗: {str(e)}"
        }), 500

    # 儲存到本地
    # timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    # filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
    # local_path = f"
{LOCAL_STORAGE}/images/{filename}"
    # image.save(local_path, format="PNG")

    # 本地 URL
    # local_url =
f"http://localhost:5000/generated_files/images/{filename}"

    # 存到 S3
    timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
    s3_key = f"Images/{filename}"

    try:
        s3.put_object(
            Bucket=S3_BUCKET_NAME,
            Key=s3_key,
            Body=response.content,    # 直接用原始 bytes
```

```
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"
# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="image",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "❌ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
```

```
"error": f"產生下載網址失敗: {str(e)}"
    }, 500

# 寫入 SQLite
# insert_record(user_id, prompt, local_url)
"""

return jsonify({
    "success": True,
    # "local_url": local_url,
    "reply": f"Flask 收到圖片prompt: {prompt}"
})
"""

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    presigned URL
    "reply": f"Flask 收到圖片prompt: {prompt}"
})

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }, 500)

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")

        data = request.json
        print("收到的 JSON:", data)

        prompt = data.get('prompt')
        # discord_channel_id =
        data.get('discord_channel_id')
        # discord_user_id = data.get('discord_user_id')

        if not prompt:
            print("❌ 缺少 prompt")
            return jsonify({"success": False, "error": "缺少
prompt"}), 400
```

```
payload = {
    "prompt": prompt,
    "style": "古典",
    "title": "AI Generated Music",
    "customMode": True,
    "instrumental": True,
    "model": "V3_5",

    # ★ 改成 API Gateway callback
    "callBackUrl": "https://xpapysqd2i.execute-
api.us-east-1.amazonaws.com/prod/audio-callback"
}

print("Suno payload:", payload)

headers = {
    "Authorization": f"Bearer {Suno_Token}",
    "Content-Type": "application/json"
}

print("正在呼叫 Suno API...")
response = requests.post(SUNO_URL,
json=payload, headers=headers, timeout=60)

print("Suno 回應狀態:", response.status_code)
print("Suno 回應內容:", response.text)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Suno API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

suno_resp = response.json()
print("解析後 Suno JSON:", suno_resp)

# ★ 抓 taskId (Suno 回傳格式有時不同)
suno_request_id = (
    suno_resp.get("id")
    or suno_resp.get("requestId"))
```

```
        or suno_resp.get("data", {}).get("taskId")
    )

    if not suno_request_id:
        print("✖ Suno 沒有回傳 taskId")
        return jsonify({
            "success": False,
            "error": "Suno 未回傳 taskId"
        }, 500)

    # 回傳給 Discord bot，可以用來顯示“開始生成中”
    return jsonify({
        "success": True,
        "task_id": suno_request_id,
        "message": "音樂生成中"
    })

except Exception as e:
    print("✖ Flask 在 /generate-audio 發生錯誤:", e)
    return jsonify({"success": False, "error": str(e)}, 500)

@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    try:
        data = request.json
        suno_items = data["data"]["data"]

        for item in suno_items:
            audio_url = item.get("source_audio_url")
            music_id = item["id"]

            if not audio_url:
                print("✖ 沒有 audio_url")
                return jsonify({
                    "success": False,
                    "error": "Suno 沒有回傳 audio_url"
                }, 500)

            # 下載音檔
            mp3 = requests.get(audio_url).content

            # 上傳 S3
            timestamp =
```

```
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    )
    filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
    s3_key = f"Audios/{filename}"

    try:
        s3.put_object(
            Bucket=S3_BUCKET_NAME,
            Key=s3_key,
            Body=mp3,    # 直接用原始 bytes
            ContentType="audio/mpeg"
        )
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"上傳 S3 失敗: {str(e)}"
        }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"

# 存進 DynamoDB
try:
    result = insert_record(
        user_id="no user id",
        prompt="no prompt",
        file_url=public_url,
        record_type="audio",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "❌ 插入失敗",
            "error": str(e)
        })
    })
# 提供可存取的 url
try:
```

```
presigned_url = s3.generate_presigned_url(
    ClientMethod="get_object",
    Params={
        "Bucket": S3_BUCKET_NAME,
        "Key": s3_key
    },
    ExpiresIn=3600 # 有效時間秒數
)
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
presigned URL
    "reply": f"Flask 收到音樂prompt: {prompt}"
})

except Exception as e:
    print("callback error:", e)
    return jsonify({"error": str(e)}), 500

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
query 取得 user_id

        records = get_history(user_id)

        return jsonify({
            "success": True,
            "records": records # 形式 : [{ "prompt": "...",
"file_url": "..."}], ...]
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
```



```
print("🚀 Flask 伺服器啟動中...")
app.run(host='0.0.0.0', port=5000, debug=True)

import os
import uuid
import datetime
import boto3
from boto3.dynamodb.conditions import Key
from dotenv import load_dotenv

# 載入環境變數
load_dotenv()
AWS_REGION = os.getenv("AWS_DEFAULT_REGION",
                        "us-east-1")
TABLE_NAME = os.getenv("DYNAMODB_TABLE",
                        "UserRecords")

# 建立 DynamoDB 資源
dynamodb = boto3.resource("dynamodb",
                           region_name=AWS_REGION)
table = dynamodb.Table(TABLE_NAME)

def insert_record(user_id: str, prompt: str, file_url: str,
                  record_type: str = "image", status: str = "success",
                  extra_meta: dict = None):
    """
    新增一筆紀錄到 DynamoDB
    """
    record_id = str(uuid.uuid4())
    created_at = datetime.datetime.utcnow().isoformat()

    item = {
        "user_id": user_id,
        "record_id": record_id,
        "prompt": prompt,
        "file_url": file_url,
        "created_at": created_at,
        "type": record_type,
        "status": status,
    }

    if extra_meta:
        item["extra_meta"] = extra_meta
```

```
table.put_item(Item=item)
return item

def get_history(user_id: str):

    # 查詢某個使用者的所有紀錄
    resp = table.query(

KeyConditionExpression=Key("user_id").eq(user_id)
    )
    items = resp.get("Items", [])

    # 只回傳紀錄中的 prompt 跟 url
    simple_items = [
        {
            "prompt": item.get("prompt"),
            "file_url": item.get("file_url"),
            "type": item.get("type")
        }
        for item in items
    ]
    return simple_items

def get_record(user_id: str, record_id: str):

    # 查詢單筆紀錄
    resp = table.get_item(
        Key={
            "user_id": user_id,
            "record_id": record_id
        }
    )
    return resp.get("Item")
```

generate_audio有成功執行，但是沒有存入s3跟 dynamodb，不知道callback url有沒有正常被呼叫

已思考 2m 36s >

關鍵點是：

S3 跟 DynamoDB 的寫入全部都在 `/audio-callback` 這個 route 裡，不在 `/generate-audio`。

所以你看 `/generate-audio` 回傳 `success`，只能代表「有成功丟請求給 Suno」，不代表 callback 有真的打回來，也不代表 S3 / Dynamo 一定有動作。

我幫你拆成兩件事看：

1. 先確認 `/audio-callback` 有沒有被呼叫

你現在的 `audio_callback` 開頭是：

python

 複製程式碼

```
@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    try:
        data = request.json
        suno_items = data["data"]["data"]
        ...
```

我建議你在最上面加一行 log，這樣 CloudWatch 比較好找：

python

 複製程式碼

```
@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    try:
        print("=== 收到 Suno callback ===")
        print("callback body:", json.dumps(request.json, ensure_ascii=False))
        data = request.json
        suno_items = data["data"]["data"]
        ...
```

然後檢查兩個情況：

✓ 測試 1：手動 POST (確認 AWS 端沒問題)

用你剛剛那個 Suno sample JSON，自己 POST 一次到 API Gateway：

bash

 複製程式碼

```
curl -X POST "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/audio-c
-H "Content-Type: application/json" \
-d '{
  "code":200,
  "data":{
    "callbackType":"complete",
    "data":[
      {
        "audio_url":"https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MGMt
        "createTime":1763910096539,
        "duration":107.92,
        "id":"258d08b5-42f6-4d1a-850c-46ecbb786d03",
        "image_url":"https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MGMt
        "model_name":"chirp-v3-5",
        "prompt":"",
        "source_audio_url":"https://cdn1.suno.ai/258d08b5-42f6-4d1a-850c-46ecbb
        "source_image_url":"https://cdn2.suno.ai/image_258d08b5-42f6-4d1a-850c-
        "source_stream_audio_url":"https://cdn1.suno.ai/258d08b5-42f6-4d1a-850c
        "stream_audio_url":"https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhL
        "tags":"古典",
        "title":"宁静钢琴冥想"
      }
    ],
    "task_id":"2b00f76847c403fe9051b70c95bc27da"
  },
  "msg":"All generated successfully."
}'
```

預期結果：

- Lambda 日誌 (CloudWatch) 裡要看到你印的：
 - `=== 收到 Suno callback ===`
 - `callback body: {...}`
- S3 bucket 裡應該多一個 `Audios/xxxx.mp3`
- DynamoDB `UserRecords` 應該多一筆：
 - `user_id = "no user id"`
 - `type = "audio"`
 - `file_url = https://你的-bucket.s3.amazonaws.com/Audios/xxxx.mp3`

如果「手動 POST」這個測試就 **沒有** 寫進 S3 / Dynamo，代表是你 `audio_callback` 的程式或 AWS 設定有問題，我下面第 3 點再幫你看。

如果「手動 POST 有成功」，但實際 Suno 產生音樂卻沒有寫入，那就代表 **Suno 根本沒有打回這個 URL**（可能是 token / callbackUrl / 權限問題），這時候就要去：

- Suno 那邊看 request log / 錯誤訊息（例如 credits 不夠、callback URL 失敗等等）。

2. 確認 Suno 的 callbackUrl 寫對

你在 `/generate-audio` 裡：

python

複製程式碼

```
payload = {
    "prompt": prompt,
    "style": "古典",
    "title": "AI Generated Music",
    "customMode": True,
    "instrumental": True,
    "model": "V3_5",
    "callbackUrl": "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/a"
}
```

這幾點可以檢查：

1. 參數名要用 `callbackUrl`（大小寫）

依照 sunoapi.org 的文件，是 `callbackUrl`（你現在是對的）。

2. 這個 URL 要能在外網直接 POST 存取

- 你已經用 Discord bot 打 `https://.../generate-image` 了，代表 API Gateway 應該是「Public、無認證」的，這樣 Suno 才叫得到。
- 只要「手動用 curl POST /audio-callback」可以成功（上面測試 1），代表 URL 對 AWS 來說是 OK 的。

3. Suno 那邊要沒有報錯

你在 `/generate-audio` 有印：

python

複製程式碼

```
print("Suno 回應狀態:", response.status_code)
print("Suno 回應內容:", response.text)
```

如果 Suno 回來的是 `code != 200` 或是訊息裡有錯誤（如 credits 不夠），就根本不會排進生成佇列，自然也不會觸發 callback。

3. 檢查 `audio_callback` 這段程式本身

你現在的 S3 + Dynamo 邏輯基本上是 OK 的，只有幾個小地方要注意：

(1) `prompt` 在 `callback` 裡沒定義（雖然不影響寫入）

結尾這裡：

```
python 📄 複製程式碼  
  
return jsonify({  
    "success": True,  
    "download_url": presigned_url,    # 👉 這是 S3 presigned URL  
    "reply": f"Flask 收到音樂prompt: {prompt}"  
})
```

`prompt` 在 `audio_callback` 裡根本沒有定義，

這行會噴 `NameError`，然後被最外層的 `except` 抓到，印：

```
python 📄 複製程式碼  
  
print("callback error:", e)  
return jsonify({"error": str(e)}), 500
```

✅ 好消息：

這個錯誤發生在 `S3 put_object` 和 `insert_record` 之後，

所以即使這行爆炸，只要程式真的跑到這裡，S3 / Dynamo 的寫入依然會成功。

但為了乾淨，你可以改成：

```
python 📄 複製程式碼  
  
return jsonify({  
    "success": True,  
    "download_url": presigned_url  
})
```

(2) 如果 JSON 結構不如預期，會在很前面就爆掉

```
python 📄 複製程式碼  
  
suno_items = data["data"]["data"]
```

如果 Suno 實際 callback 回來的 JSON 和你 sample 有落差（例如 key 名不同），這裡會

`KeyError`，

然後直接被外層 `except` 抓走 → S3 & Dynamo 都不會執行。

所以：

- 你一定要先看 CloudWatch 裡 `callback body`：印出來的那份 JSON，確認實際長得是不是你想的那樣。
- 如果怕出錯，可以寫防呆一點，例如：

python

 複製程式碼

```
data = request.json or {}  
outer = data.get("data") or {}  
suno_items = outer.get("data") or []
```

4. 總結你現在要做的三步

1. 先手動測試 `/audio-callback`：

用我給你的 `curl` (或 Postman) 打一次，確認：

- Lambda 有 log：`=== /generate-audio 被呼叫 === / callback body: ...`
- S3 多一個 `Audios/...mp3`
- DynamoDB 多一筆 `type=audio` 的紀錄

2. 看 CloudWatch 有沒有 `callback error`：之類的錯誤字串

如果有，把那段錯誤貼出來，就知道是 JSON 結構問題、S3 權限，還是 `NameError`。

3. 確定 Suno 的回應真的 `code=200` 並且有開始工作

看 `/generate-audio` 的 log 中 Suno 回傳的 JSON，

確認裡面 `code: 200`、`data.taskId` 有值，這樣才會真的進入排程。

如果你願意，下一步你可以：

- 先跑一次「手動 POST callback」
- 然後跟我說：「S3 有 / 沒有新 mp3、Dynamo 有 / 沒有新紀錄、CloudWatch 裡看到哪些 log」

我就可以直接幫你 pinpoint 是哪一段在卡。

     ...

Status: Succeeded

Test Event Name: test_generate_audio

Response:

```
{
  "statusCode": "200",
  "headers": {
    "Content-Type": "application/json",
    "Content-Length": "105"
  },
  "isBase64Encoded": false,
  "body": "
{\"message\": \"\\u97f3\\u6a02\\u751f\\u6210\\u4e2d\\
\", \"success\": true, \"task_id\": \"c67d57b7a13cf9650a0b9
1efe08fbd81\\\"}\\n\"
}
```

The area below shows the last 4 KB of the execution log.

Function Logs:

START RequestId: 0879779f-92a6-4eed-98ca-fb3a3f0a62bc Version: \$LATEST

=== /generate-audio 被呼叫 ===

收到的 JSON: {'prompt': '西洋音樂'}

Suno payload: {'prompt': '西洋音樂', 'style': '古典', 'title': 'AI Generated Music', 'customMode': True, 'instrumental': True, 'model': 'V3_5', 'callbackUrl':

'https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/audio-callback'}

正在呼叫 Suno API...

Suno 回應狀態: 200

Suno 回應內容: {"code":200,"msg":"success","data":{"taskId":"c67d57b7a13cf9650a0b91efe08fbd81"}}

解析後 Suno JSON: {'code': 200, 'msg': 'success', 'data': {'taskId': 'c67d57b7a13cf9650a0b91efe08fbd81'}}

END RequestId: 0879779f-92a6-4eed-98ca-fb3a3f0a62bc

REPORT RequestId: 0879779f-92a6-4eed-98ca-fb3a3f0a62bc Duration: 444.34 ms Billed Duration: 1743 ms

Memory Size: 128 MB Max Memory Used:

102 MB Init Duration: 1298.04 ms

Request ID: 0879779f-92a6-4eed-98ca-fb3a3f0a62bc

已思考 38s >

很好 👍 這代表 `/generate-audio` 這條已經完全正常運作了，現在 Suno 這邊也確定有收到請求，還回給你一個 `taskId`：

json

複製程式碼

```
{"code":200,"msg":"success","data":{"taskId":"c67d57b7a13cf9650a0b91efe08fbd81"}}
```

先說結論：

這次測的是 `/generate-audio`，它只負責：

1. 收到你的 prompt (這裡是「西洋音樂」)
2. 呼叫 Suno API
3. 拿到 `taskId` 後回傳給你「音樂生成中」

👉 不會在這一步寫 S3 或 DynamoDB

S3 / DynamoDB 的邏輯全部都在 `/audio-callback` 這條裡，所以要驗證「有沒有真的存檔、寫進 DynamoDB」，要測的是 `/audio-callback`。

接下來你要做兩件事

1 手動測 `/audio-callback` (確認 S3 + Dynamo 真的 OK)

用你剛剛那個 Suno 的 JSON，直接打到 API Gateway 的 `/audio-callback`：

bash

複製程式碼

```
curl -X POST "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/audio-c" \
-H "Content-Type: application/json" \
-d '{
  "code":200,
  "data":{
    "callbackType":"complete",
    "data":[
      {
        "audio_url":"https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MGMt
```

```

        "createTime":1763910096539,
        "duration":107.92,
        "id":"258d08b5-42f6-4d1a-850c-46ecbb786d03",
        "image_url":"https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MGMt
        "model_name":"chirp-v3-5",
        "prompt":"",
        "source_audio_url":"https://cdn1.suno.ai/258d08b5-42f6-4d1a-850c-46ecbb
        "source_image_url":"https://cdn2.suno.ai/image_258d08b5-42f6-4d1a-850c-4
        "source_stream_audio_url":"https://cdn1.suno.ai/258d08b5-42f6-4d1a-850c-
        "stream_audio_url":"https://musicfile.api.box/MjU4ZDA4YjUtNDJmNi00ZDFhLTg1MGMt
        "tags":"古典",
        "title":"宁静钢琴冥想"
    }
],
    "task_id":"2b00f76847c403fe9051b70c95bc27da"
},
    "msg":"All generated successfully."
}'

```

你的 `audio_callback` 裡建議先加這兩行方便確認：

python

 複製程式碼

```

@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    try:
        print("=== 收到 Suno callback ===")
        print("callback body:", json.dumps(request.json, ensure_ascii=False))
        data = request.json
        ...

```

打完之後檢查：

1. CloudWatch Log 裡有沒有：

- `=== 收到 Suno callback ===`
- `callback body: {...}`

2. S3 bucket 裡有沒有多一個 `Audios/xxxx.mp3`

3. DynamoDB `UserRecords` 裡有沒有多一筆：

- `type = "audio"`
- `file_url = https://你的-bucket.s3.amazonaws.com/Audios/xxxx.mp3`

如果這三個都有 

→ 代表你的 `/audio-callback` + S3 + Dynamo 完全 OK，接下來只剩下 Suno 實際 callback 的問題。

2 修一個小 bug，避免 callback 出錯就整個 500

在 `audio_callback` 結尾這一段：

python

 複製程式碼

```
return jsonify({
    "success": True,
    "download_url": presigned_url,    # 🙌 這是 S3 presigned URL
    "reply": f"Flask 收到音樂prompt: {prompt}"
})
```

這裡 `prompt` 在這個 function 裡根本沒有定義，會噴 `NameError`，被最外層的 `except` 吃掉，就變成 500 回應。

先改成這樣就好：

python

 複製程式碼

```
return jsonify({
    "success": True,
    "download_url": presigned_url
})
```

另外你現在 `return` 寫在 `for item in suno_items:` 迴圈裡面，

只要處理第一首就會 `return`，第二首不會存。

如果你之後真的要一次處理多首，可以改成收集完再回傳，例如：

python

 複製程式碼

```
urls = []
for item in suno_items:
    ...
    urls.append(presigned_url)

return jsonify({
    "success": True,
```

```
"download_urls": urls  
})
```

現在的狀態總整理

-  `/generate-image` : 會把圖片存到 S3 + 寫 DynamoDB , OK
-  `/generate-audio` : 已經證明會成功呼叫 Suno , 拿到 `taskId` , OK
-  `/audio-callback` :
還沒確認 Suno 有沒有真的打回來 , 也還沒確認 S3 / Dynamo 這條實際有跑

下一步就照我上面第 1 點 :

先用 `curl` / `Postman` 手動打 `/audio-callback` 測一次 ,

如果你打完發現 :

- 有 S3 檔案、Dynamo 有資料 → 我幫你再一起看怎麼從 Discord `/create_audio` 串到這整條流程
- 如果沒有 , 貼 CloudWatch 裡的錯誤訊息給我 , 我直接幫你定位是哪一行出問題 🙌

     ...

Status: Succeeded

Test Event Name: test_callback

Response:

```
{  
  "statusCode": "500",  
  "headers": {  
    "Content-Type": "application/json",  
    "Content-Length": "41"  
  },  
  "isBase64Encoded": false,  
  "body": "{\\\"error\\\":\\\"name 'prompt' is not defined\\\"}\\n"  
}
```

The area below shows the last 4 KB of the execution log.

Function Logs:

START RequestId: cc788f0f-19bd-45de-bfc1-

```
f47ed52299c6 Version: $LATEST
=== 收到 Suno callback ===
callback error: name 'prompt' is not defined
END RequestId: cc788f0f-19bd-45de-bfc1-f47ed52299c6
REPORT RequestId: cc788f0f-19bd-45de-bfc1-f47ed52299c6 Duration: 1100.31 ms Billed Duration: 2382 ms Memory Size: 128 MB Max Memory Used: 100 MB Init Duration: 1281.26 ms

Request ID: cc788f0f-19bd-45de-bfc1-f47ed52299c6

from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import requests
from PIL import Image
import awsgi
import boto3
from db import insert_record, get_history
import json

load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-inference/models/stabilityai/stable-diffusion-xl-base-1.0"
SUNO_URL =
"https://api.sunoapi.org/api/v1/generate"

app = Flask(__name__)

s3 = boto3.client("s3") # 建立 s3 client

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
```

```
prompt")
    user_id = data.get("user_id", "unknown")

    if not prompt:
        return jsonify({"success": False, "error": "缺少
prompt"}), 400

    headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
    payload = {"inputs": prompt}

    response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)

    if response.status_code != 200:
        return jsonify({
            "success": False,
            "error": f"Hugging Face API 錯誤:
{response.status_code}",
            "details": response.text
        }), 500

    content_type = response.headers.get('Content-
Type', '')

    if 'application/json' in content_type:
        error_data = response.json()
        return jsonify({
            "success": False,
            "error": "Hugging Face 回傳錯誤",
            "details": error_data
        }), 500

    if 'image' not in content_type:
        return jsonify({
            "success": False,
            "error": f"非預期的回傳格式: {content_type}"
        }), 500

    # 嘗試解析圖片
    try:
        image =
Image.open(io.BytesIO(response.content))
```

```
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

# 存到 S3
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = f"Images/{filename}"

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content,    # 直接用原始 bytes
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"
# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="image",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
```

```
        "body": json.dumps({
            "message": "❌ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    "reply": f"Flask 收到圖片 prompt: {prompt}"
})

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")

        data = request.json
        print("收到的 JSON:", data)
```



```
prompt = data.get('prompt')
# discord_channel_id =
data.get('discord_channel_id')
# discord_user_id = data.get('discord_user_id')

if not prompt:
    print("❌ 缺少 prompt")
    return jsonify({"success": False, "error": "缺少
prompt"}), 400

payload = {
    "prompt": prompt,
    "style": "古典",
    "title": "AI Generated Music",
    "customMode": True,
    "instrumental": True,
    "model": "V3_5",
    "callbackUrl": "https://xpapysqd2i.execute-
api.us-east-1.amazonaws.com/prod/audio-callback"
}

print("Suno payload:", payload)

headers = {
    "Authorization": f"Bearer {Suno_Token}",
    "Content-Type": "application/json"
}

print("正在呼叫 Suno API...")
response = requests.post(SUNO_URL,
json=payload, headers=headers, timeout=60)

print("Suno 回應狀態:", response.status_code)
print("Suno 回應內容:", response.text)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Suno API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500
```

```
suno_resp = response.json()
print("解析後 Suno JSON:", suno_resp)

# ★ 抓 taskId (Suno 回傳格式有時不同)
suno_request_id = (
    suno_resp.get("id")
    or suno_resp.get("requestId")
    or suno_resp.get("data", {}).get("taskId")
)

if not suno_request_id:
    print("✗ Suno 沒有回傳 taskId")
    return jsonify({
        "success": False,
        "error": "Suno 未回傳 taskId"
    }), 500

# 回傳給 Discord bot，可以用來顯示“開始生成中”
return jsonify({
    "success": True,
    "task_id": suno_request_id,
    "message": "音樂生成中"
})

except Exception as e:
    print("✗ Flask 在 /generate-audio 發生錯誤:", e)
    return jsonify({"success": False, "error": str(e)}), 500

@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    try:
        print("=== 收到 Suno callback ===")
        data = request.json
        suno_items = data["data"]["data"]

        for item in suno_items:
            audio_url = item.get("source_audio_url")
            music_id = item["id"]

            if not audio_url:
                print("✗ 沒有 audio_url")
                return jsonify({
```

```
        "success": False,
        "error": "Suno 沒有回傳 audio_url"
    }}, 500

# 下載音檔
mp3 = requests.get(audio_url).content

# 上傳 S3
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
s3_key = f"Audios/{filename}"

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=mp3,    # 直接用原始 bytes
        ContentType="audio/mpeg"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"

# 存進 DynamoDB
try:
    result = insert_record(
        user_id="no user id",
        prompt="no prompt",
        file_url=public_url,
        record_type="audio",
        status="success"
    )
except Exception as e:
```

```
        return jsonify({
            "statusCode": 500,
            "body": json.dumps({
                "message": "✖ 插入失敗",
                "error": str(e)
            })
        })
    # 提供可存取的 url
    try:
        presigned_url = s3.generate_presigned_url(
            ClientMethod="get_object",
            Params={
                "Bucket": S3_BUCKET_NAME,
                "Key": s3_key
            },
            ExpiresIn=3600 # 有效時間秒數
        )
    except Exception as e:
        return jsonify({
            "success": False,
            "error": f"產生下載網址失敗: {str(e)}"
        }), 500

    return jsonify({
        "success": True,
        "download_url": presigned_url, # 👉 這是 S3
        "reply": f"Flask 收到音樂prompt: {prompt}"
    })

    except Exception as e:
        print("callback error:", e)
        return jsonify({"error": str(e)}), 500

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
        query 取得 user_id

        records = get_history(user_id)

        return jsonify({
```

```

        "success": True,
        "records": records # 形式 : [{ "prompt": "...",
        "file_url": "..."}], ...]
    })

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
    app.run(host='0.0.0.0', port=5000, debug=True)
為甚麼沒有存進去

```

已思考 1m 22s >

關鍵原因其實已經寫在 log 了：

```
callback error: name 'prompt' is not defined
```

就是你 `audio_callback` 裡面最後在組回傳 JSON 的時候，用到一個 **沒有定義的變數** `prompt`，所以整個 `try` block 丟出錯誤，被外層 `except` 接住，回傳 500，然後你看到 `test_callback` 那個結果。

問題點在哪？

這一段👉

python

📄 複製程式碼

```

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3 presigned URL
    "reply": f"Flask 收到音樂prompt: {prompt}"
})

```

`audio_callback()` 這個 function 裡面 **根本沒有** `prompt` 這個變數，所以 Python 在這一行會直接噴：

text

📄 複製程式碼

```
NameError: name 'prompt' is not defined
```

被最外層這段 catch 到：

python

複製程式碼

```
except Exception as e:
    print("callback error:", e)
    return jsonify({"error": str(e)}), 500
```

因此你看到：

json

複製程式碼

```
{"error": "name 'prompt' is not defined"}
```

⚠ 這個錯誤發生在 最尾巴 return 的時候，

理論上在這之前的：

- `requests.get(audio_url).content`
- `s3.put_object(...)`
- `insert_record(...)`

都已經執行了，不會被 rollback。

如果你去 DynamoDB / S3 都完全沒看到資料，那就代表**根本還沒跑到那裡就出錯**（例如 JSON 結構不對、`audio_url` 為 None 等），這時就要靠多印 log 來查。

先修掉 prompt 的 bug（必做）

請先把 `audio_callback` 改成這樣：

python

複製程式碼

```
@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    try:
        print("=== 收到 Suno callback ===")
        data = request.json
        print("callback body:", json.dumps(data, ensure_ascii=False))

        suno_items = data["data"]["data"]

        saved_urls = [] # 如果一次有多首，可以全部收集起來

        for item in suno_items:
```

```
audio_url = item.get("source_audio_url")
music_id = item["id"]

print("處理音樂 id:", music_id)
print("audio_url:", audio_url)

if not audio_url:
    print("❌ 沒有 audio_url")
    return jsonify({
        "success": False,
        "error": "Suno 沒有回傳 audio_url"
    }), 500

# 下載音檔
mp3 = requests.get(audio_url).content
print("下載完成, size:", len(mp3))

# 上傳 S3
timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
filename = f"{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
s3_key = f"Audios/{filename}"

print("準備上傳 S3, key =", s3_key)

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=mp3,      # 直接用原始 bytes
        ContentType="audio/mpeg"
    )
    print("✅ 上傳 S3 成功")
except Exception as e:
    print("❌ 上傳 S3 失敗:", e)
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

public_url = f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
print("public_url:", public_url)

# 存進 DynamoDB
```

```
try:
    result = insert_record(
        user_id="no user id",
        prompt="no prompt",
        file_url=public_url,
        record_type="audio",
        status="success"
    )
    print("✅ DynamoDB insert 成功:", result)
except Exception as e:
    print("❌ DynamoDB insert 失敗:", e)
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "❌ 插入失敗",
            "error": str(e)
        })
    })

# 產生 presigned_url (可選)
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
    print("presigned_url:", presigned_url)
except Exception as e:
    print("❌ 產生 presigned_url 失敗:", e)
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

saved_urls.append(presigned_url)

# ✅ 全部處理完再回傳一次 200
return jsonify({
    "success": True,
    "download_urls": saved_urls
})
```



```

    })

    except Exception as e:
        print("callback error:", e)
        return jsonify({"error": str(e)}), 500

```

重點修改：

1. 刪掉 `reply": f"Flask 收到音樂prompt: {prompt}"` (因為沒有 `prompt`)
2. 加一堆 `print(...)` 幫你 debug :
 - 收到的 JSON
 - 處理到哪一首音樂
 - `audio_url` 是什麼
 - S3 Key
 - `public_url`
3. 把 `return` 移到迴圈外，支援多首一起處理 (現在 Suno 一次回兩首會比較合理)

再跑一次 test_callback

用剛剛那個 test event 再測一次 `/audio-callback`：

- 如果這次 `statusCode` 是 `200` 並且 `body` 是 `{"success": true, "download_urls": [...]}`
→ 代表 callback 整條 OK
- 然後去：
 - S3 看 `Audios/xxxx.mp3` 在不在
 - DynamoDB `UserRecords` 看有沒有多一筆 `type = "audio"` 的紀錄

如果還是「沒有存進去」

這時候你就去 CloudWatch 看 log，靠我們加的 `print` 來查：

- 有沒有印出 `準備上傳 S3, key = ...` ？
- 有沒有印出 `✅ 上傳 S3 成功` ？
- 有沒有印出 `✅ DynamoDB insert 成功` ？
- 失敗的話，`❌ 上傳 S3 失敗`：或 `❌ DynamoDB insert 失敗`：會把真正的 AWS error message 印出來 (像是權限不夠、Region 不對、Table 名稱打錯等)

你可以直接把那一小段錯誤 log 貼給我，我可以幫你看是 IAM 權限、region、還是 table name 問題。

總之，現在這個 500 是因為 `prompt` 沒定義的純程式 bug，

先修掉這個，S3 / DynamoDB 真正存不存得進去，我們就可以用 log 做第二層確認 🍌

📄 👍 💬 ↗️ ↺️ ...

我如果把callbackurl設定成s3 bucket的其中一個folder(這個bucket已公開)，是不是suno就能把音樂url存進去

已思考 14s >

不行 🤔

`callbackUrl` 不能設成 S3 bucket 或 folder 的網址，就算 bucket 是 public 也一樣「完全沒用」。

為什麼不行？

1. Suno 的 `callbackUrl` 是「要發 HTTP POST 的地方」

Suno 做完歌之後，會對你給的 `callbackUrl` 做一個 **HTTP POST**，body 是像你之前給我的那個 JSON：

json

📄 複製程式碼

```
{
  "code": 200,
  "data": {
    "callbackType": "complete",
    "data": [
      {
        "source_audio_url": "https://cdn1.suno.ai/xxx.mp3",
        ...
      }
    ]
  }
}
```

也就是說，Suno 期待的是：

「我把 JSON 丟給你這個 URL，你自己想辦法處理」

這個 URL 必須是一個 **會跑程式的 HTTP 服務**：

- 例如：API Gateway + Lambda 的 `/audio-callback`
- 或你自己開在 EC2 / Render / Railway / Vercel 的後端
- 你現在的 `https://...execute-api.../prod/audio-callback` 就是正確做法 

2. S3 bucket / folder 不是「會跑程式的 HTTP 服務」

S3 做的事情比較像「檔案雲端硬碟」，不是後端伺服器：

- 它可以讓你：
 - `GET`：讀取檔案
 - `PUT` / `POST`：上傳檔案（但通常要用你事先簽好的 pre-signed URL 或 AWS credentials）
- 但它 **不會幫你讀 JSON、也不會幫你解析** `source_audio_url` 再去幫你抓 mp3、存檔

所以如果你把 `callbackUrl` 設成像這樣：

text

 複製程式碼

```
https://my-bucket.s3.amazonaws.com/audio-callback/
```

Suno 會嘗試 `POST` JSON 到這個 URL：

- S3 不會處理這個 JSON
- 很大的機會直接回 `403` 或 `405`
- 更不可能替你「把音樂 URL 轉存進去」

正確的流程其實就是你現在這套

1. Discord 呼叫 `/generate-audio`
2. 你的 Flask（跑在 Lambda 後面）呼叫 Suno API，帶上：

json

 複製程式碼

```
"callbackUrl": "https://你的-api-gateway-url/prod/audio-callback"
```

3. Suno 生成完歌 → 對 `audio-callback` 發出 **HTTP POST(JSON)**。
4. 你的 `/audio-callback`：
 - 從 JSON 取出 `source_audio_url`

- `requests.get(source_audio_url)` 把 mp3 抓下來
- `s3.put_object(...)` 上傳到 你自己的 S3 bucket/folder
- `insert_record(...)` 存 DynamoDB 紀錄 (包含 S3 public URL)

✓ 只有你自己的程式，才有你設定好的 IAM 權限可以寫入 S3、寫入 DynamoDB。

✗ Suno 並不知道你的 S3 bucket 結構，也沒有你的 AWS 權限。

那我要「讓知道連結的人都能聽音樂」要怎麼做？

你現在已經走在正確路上了，總結一下做法：

1. S3 bucket 或 `Audios/` 這個 folder 設成：
 - **ACL : Object 可 public-read** (或用 Bucket policy 允許 `s3:GetObject` 給 `Principal: *`)
2. 上傳 mp3 後組 public URL (你現在這行是對的)：

python

複製程式碼

```
public_url = f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
```

3. 把 `public_url` 存進 DynamoDB，或回傳給 Discord bot 去貼在頻道裡。

這樣：

- 誰拿到這個 `public_url`，就可以直接點開播放 / 下載 mp3。
- Discord 看到這種 URL，也會自動做預覽 (就跟圖片一樣)。

小結

- ✗ `callbackUrl` 設成 S3 folder → 完全不會自動幫你存任何東西
- ✓ `callbackUrl` 一定要是「你的後端 API」，由你：
 - 收 JSON
 - 下載音檔
 - 上傳 S3
 - 寫 DynamoDB

如果你願意，可以把 目前最新的 `/audio-callback` 程式 + 你真實拿到的 Suno callback JSON 再貼一次，我可以幫你逐行檢查「哪一段可能出錯」，順便幫你加更精準的 debug `print`。



```
from flask import Flask, request, jsonify,
send_from_directory
from dotenv import load_dotenv
import os, io, uuid, datetime
import requests
from PIL import Image
import awsgi
import boto3
from db import insert_record, get_history
import json

load_dotenv()
HuggingFace_Token =
os.getenv("HUGGINGFACE_TOKEN")
Suno_Token = os.getenv("SUNO_TOKEN")
S3_BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
HuggingFace_URL = "https://router.huggingface.co/hf-
inference/models/stabilityai/stable-diffusion-xl-base-
1.0"
SUNO_URL =
"https://api.sunoapi.org/api/v1/generate"

app = Flask(__name__)

s3 = boto3.client("s3") # 建立 s3 client

@app.route("/generate-image", methods=["POST"])
def generate_image():
    try:
        data = request.json
        prompt = data.get("prompt", "沒有收到圖片
prompt")
        user_id = data.get("user_id", "unknown")

        if not prompt:
            return jsonify({"success": False, "error": "缺少
prompt"}), 400

        headers = {"Authorization": f"Bearer
{HuggingFace_Token}"}
        payload = {"inputs": prompt}
```

```
response = requests.post(HuggingFace_URL,
headers=headers, json=payload, timeout=60)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Hugging Face API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

content_type = response.headers.get('Content-
Type', '')

if 'application/json' in content_type:
    error_data = response.json()
    return jsonify({
        "success": False,
        "error": "Hugging Face 回傳錯誤",
        "details": error_data
    }), 500

if 'image' not in content_type:
    return jsonify({
        "success": False,
        "error": f"非預期的回傳格式: {content_type}"
    }), 500

# 嘗試解析圖片
try:
    image =
Image.open(io.BytesIO(response.content))
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"圖片解析失敗: {str(e)}"
    }), 500

# 存到 S3
timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S
")
```

```
filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.png"
s3_key = f"Images/{filename}"

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=response.content,    # 直接用原始 bytes
        ContentType="image/png"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"
# 存進 DynamoDB
try:
    result = insert_record(
        user_id=user_id,
        prompt=prompt,
        file_url=public_url,
        record_type="image",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
        ClientMethod="get_object",
```

```
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

return jsonify({
    "success": True,
    "download_url": presigned_url, # 👉 這是 S3
    "reply": f"Flask 收到圖片prompt: {prompt}"
})

except Exception as e:
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500

@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        print("=== /generate-audio 被呼叫 ===")

        data = request.json
        print("收到的 JSON:", data)

        prompt = data.get('prompt')
        # discord_channel_id =
        data.get('discord_channel_id')
        # discord_user_id = data.get('discord_user_id')

        if not prompt:
            print("❌ 缺少 prompt")
            return jsonify({"success": False, "error": "缺少
prompt"}), 400
```



```
payload = {
    "prompt": prompt,
    "style": "古典",
    "title": "AI Generated Music",
    "customMode": True,
    "instrumental": True,
    "model": "V3_5",
    "callbackUrl": "https://xpapysqd2i.execute-
api.us-east-1.amazonaws.com/prod/audio-callback"
}

print("Suno payload:", payload)

headers = {
    "Authorization": f"Bearer {Suno_Token}",
    "Content-Type": "application/json"
}

print("正在呼叫 Suno API...")
response = requests.post(SUNO_URL,
json=payload, headers=headers, timeout=60)

print("Suno 回應狀態:", response.status_code)
print("Suno 回應內容:", response.text)

if response.status_code != 200:
    return jsonify({
        "success": False,
        "error": f"Suno API 錯誤:
{response.status_code}",
        "details": response.text
    }), 500

suno_resp = response.json()
print("解析後 Suno JSON:", suno_resp)

# ★ 抓 taskId (Suno 回傳格式有時不同)
suno_request_id = (
    suno_resp.get("id")
    or suno_resp.get("requestId")
    or suno_resp.get("data", {}).get("taskId")
)
```

```
if not suno_request_id:
    print("✖ Suno 沒有回傳 taskId")
    return jsonify({
        "success": False,
        "error": "Suno 未回傳 taskId"
    }), 500

# 回傳給 Discord bot，可以用來顯示“開始生成中”
return jsonify({
    "success": True,
    "task_id": suno_request_id,
    "message": "音樂生成中"
})

except Exception as e:
    print("✖ Flask 在 /generate-audio 發生錯誤:", e)
    return jsonify({"success": False, "error": str(e)}), 500

@app.route('/audio-callback', methods=['POST'])
def audio_callback():
    try:
        print("=== 收到 Suno callback ===")
        data = request.json
        suno_items = data["data"]["data"]

        for item in suno_items:
            audio_url = item.get("source_audio_url")
            music_id = item["id"]

            if not audio_url:
                print("✖ 沒有 audio_url")
                return jsonify({
                    "success": False,
                    "error": "Suno 沒有回傳 audio_url"
                }), 500

            # 下載音檔
            mp3 = requests.get(audio_url).content

            # 上傳 S3
            timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")
```

```
)

filename = f"
{timestamp}_{uuid.uuid4().hex[:8]}.mp3"
s3_key = f"Audios/{filename}"

try:
    s3.put_object(
        Bucket=S3_BUCKET_NAME,
        Key=s3_key,
        Body=mp3,    # 直接用原始 bytes
        ContentType="audio/mpeg"
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"上傳 S3 失敗: {str(e)}"
    }), 500

public_url=f"https://{S3_BUCKET_NAME}.s3.amazonaws.com/{s3_key}"
local_url=f"s3://{S3_BUCKET_NAME}/{s3_key}"

# 存進 DynamoDB
try:
    result = insert_record(
        user_id="no user id",
        prompt="no prompt",
        file_url=public_url,
        record_type="audio",
        status="success"
    )
except Exception as e:
    return jsonify({
        "statusCode": 500,
        "body": json.dumps({
            "message": "✖ 插入失敗",
            "error": str(e)
        })
    })

# 提供可存取的 url
try:
    presigned_url = s3.generate_presigned_url(
```

```
        ClientMethod="get_object",
        Params={
            "Bucket": S3_BUCKET_NAME,
            "Key": s3_key
        },
        ExpiresIn=3600 # 有效時間秒數
    )
except Exception as e:
    return jsonify({
        "success": False,
        "error": f"產生下載網址失敗: {str(e)}"
    }), 500

    return jsonify({
        "success": True,
        "download_url": presigned_url, # 👉 這是 S3
presigned URL
        "reply": "Flask 收到音樂prompt"
    })

except Exception as e:
    print("callback error:", e)
    return jsonify({"error": str(e)}), 500

@app.route("/history", methods=["GET"])
def history():
    try:
        user_id = request.args.get("user_id") # 從 URL
query 取得 user_id

        records = get_history(user_id)

        return jsonify({
            "success": True,
            "records": records # 形式 : [{ "prompt": "...",
"file_url": "..."}], ...]
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    print("🚀 Flask 伺服器啟動中...")
```

```
app.run(host='0.0.0.0', port=5000, debug=True)
```

目前的做法callbackurl就是apigateway+audio_callback
· 但是audio_callback一直沒有被呼叫，問題在哪

已思考 4m 44s >

關鍵點先講結論：

你現在的 Lambda/Flask 程式本身大致沒問題，`/audio-callback` 你用 **Lambda 測試事件** 是會進來的；但是 **Suno** 那邊根本沒有成功打到這個 URL，所以 S3、DynamoDB 都沒東西。

我幫你整理「為什麼會這樣」跟「你可以怎麼一步一步確認」。

一、你現在程式裡真的有一個 bug

在 `audio_callback` 裡這一段：

python

複製程式碼

```
return jsonify({
    "success": True,
    "download_url": presigned_url,  # 👉 這是 S3 presigned URL
    "reply": "Flask 收到音樂prompt"
})
```

👉 這裡你已經把 `prompt` 改成字串 `"Flask 收到音樂prompt"`，
所以就不會再出現 `name 'prompt' is not defined` 的錯誤了。
你剛剛 `test_callback` 的錯誤訊息是舊版本（還沒改掉時）才會發生的。

另外還有一個小問題：

python

複製程式碼

```
for item in suno_items:
    ...
    return jsonify({...})
```

`return` 放在 `for` 迴圈裡面，代表只要處理到第一首歌就直接回傳，不會處理第二首。
這不會影響「有沒有被呼叫」，但未來如果一次回傳多首歌，只會存第一首。

建議：把 `return jsonify(...)` 移到 `for` 迴圈外面。

二、為什麼 Suno 一直沒有打進 `/audio-callback`

重點：

現在可以確定 **Lambda 測試事件打 `/audio-callback` 是 OK 的**（你看到 `=== 收到 Suno callback ===`）。

但「真正從外面 HTTP 打進來」有兩件事要確認：

1. 你要測試「從網路打 API Gateway」

請用 Postman / curl / 瀏覽器，直接打你的 API Gateway：

```
bash 複製程式碼  
  
curl -X POST \  
  https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/audio-callback \  
  -H "Content-Type: application/json" \  
  -d '{  
    "code": 200,  
    "msg": "All generated successfully.",  
    "data": {  
      "callbackType": "complete",  
      "task_id": "test-task-id",  
      "data": [  
        {  
          "id": "test-id-123",  
          "source_audio_url": "https://cdn1.suno.ai/test.mp3"  
        }  
      ]  
    }  
  }'
```

如果這樣呼叫之後：

- CloudWatch Log 裡有出現

`=== 收到 Suno callback ===`

而且你的 S3 有多一個 `Audios/xxxx.mp3`

DynamoDB `UserRecords` 多一筆 `audio` 記錄

👉 代表 **API Gateway → Lambda → Flask → `/audio-callback`** 這條線是通的。

問題就不在你的 AWS，而是在 Suno 那邊「根本沒有成功打這個 URL」。

如果這個 curl 測試都失敗，才是 API Gateway 設定有問題（例如 `/audio-callback` 這個 path 沒有掛到 Lambda、只建 `/generate-audio` 路由、等等）。

但你現在的情況看起來比較像：

- `/generate-audio` 可以從 Discord / Test Event 正常
- `/audio-callback` 只有用 Lambda 「測試事件」呼叫過，還沒有確認 HTTP 入口

所以這一步 **一定要先做**。

2. Suno 那邊到底有沒有真的「打你的 callback URL」

你在 payload 裡這樣寫：

```
python 複製程式碼  
  
payload = {  
    "prompt": prompt,  
    "style": "古典",  
    "title": "AI Generated Music",  
    "customMode": True,  
    "instrumental": True,  
    "model": "V3_5",  
    "callbackUrl": "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/a  
}
```

這裡有幾個重點：

1. 參數名稱要完全符合 Suno / sunoapi 的規範

- 有些文件是 `callbackUrl` (小寫 b)
- 有些是 `callBackUrl` (大寫 B)

你現在用的是 `callBackUrl`，這要跟你看到的官方文件一模一樣才行，大小寫錯一個它就直接忽略。

2. 即使 `generate` 回傳 `taskId`，

也不代表 callback 一定設定成功，可能只是排了非 callback 的任務。

通常他們會有「查詢 task 狀態」的 API (例如 `/getTaskStatus` 或 `/get-music-detail` 類似的)，

你可以用 `task_id` 去查，看看裡面有沒有記錄 callback URL 或錯誤。

但因為我看不到你 sunoapi 的後台 / 文件細節，只能推論：

最有可能的兩種狀況：

1. “Suno 的 API 參數名跟你寫的不完全一樣 (例如要 `callbackUrl` 而不是 `callBackUrl`)。”

2. “Suno 真的有打，但打到的 URL 回傳 4xx / 5xx，而你還沒從 Suno 那邊的 log / dashboard 看錯誤訊息。”

三、你接下來可以怎麼一步一步驗證

我幫你排成「照順序檢查」的 checklist，你照做就能縮小問題範圍：

1. 測試 API Gateway → Lambda → Flask → /audio-callback 是否正常

用上面的 `curl` or Postman 測試：

```
bash
```

[複製程式碼](#)

```
curl -X POST https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/audi
```

- 看 CloudWatch Log 有沒有印：`=== 收到 Suno callback ===`
- 看 S3 `Audios/` 有沒有多一個檔案
- 看 DynamoDB `UserRecords` 有沒有多一筆 `audio` 資料

2. 如果 1 正常，就代表 AWS 端沒問題

下一步就要檢查 Suno 那邊：

- 確認文件上 callback 參數名稱是不是真的叫 `callbackUrl`
- 如果它有「任務查詢 API」，用 `task_id` 查看看，是否有 callback 相關資訊 / 錯誤
- 看 sunoapi 後台有沒有請求 log

3. 修正 `audio_callback` 的小細節

- 把 `return jsonify({...})` 移出 `for item in suno_items:` 迴圈
- 如果之後想用真的 `user_id` / `prompt`，可以把它們在 `generate-audio` 時傳給 Suno，
再在 callback body 裡一起帶回來（例如 Suno 支援的 `ext` / `metadata` 欄位）。

四、順帶一提：Discord Token 千萬要換掉

你在前面有貼出整串

```
python
```

[複製程式碼](#)

```
DC_token = "MTQ3....."
```


這個是 **超機密資訊**，一旦被別人看到，就可以直接控制你的 Bot。

建議你立刻：

1. 到 Discord Developer Portal，把這個 Bot 的 Token 重新產生一個新的
2. 在程式裡改成用 `.env`：

```
python
```

[複製程式碼](#)

```
load_dotenv()  
DC_token = os.getenv('DISCORD_TOKEN')
```

3. 把舊 Token 相關設定刪掉。

如果你願意，你可以先照我說的第 1 步，用 curl 打一下 `/audio-callback`，

確認 S3 & DynamoDB 都會寫入，

這一步成功之後，我就可以再幫你推進「怎麼從 Suno 那邊確認 callback 有沒有打過來 / 參數名要不要改」。

< 2/2 > [📄](#) [👍](#) [💬](#) [📌](#) [🔄](#) [...](#)

👉 請用 Postman / curl / 瀏覽器，直接打你的 API Gateway

[怎麼做](#)

已思考 35s >

好，我一步一步教你怎麼「真的從外面打到」

`https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/audio-callback` 來測試。

目標：

- ✓ 確認 **API Gateway → Lambda → Flask** `/audio-callback` 這條路是通的
- ✓ 看 CloudWatch 有沒有印出 `=== 收到 Suno callback ===`
- ✓ 看 S3 / DynamoDB 有沒有資料

方法一：用 Postman 測試（最直覺）

1. 開 Postman → 點「New」→ 選 HTTP Request
2. 上方設定：

- Method：選 **POST**
- URL：貼上

```
https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/audio-callback
```

3. 點下方 **Body** 分頁：

- 選 **raw**
- 右邊格式改成 **JSON**
- 貼上這段 JSON (是模擬 Suno 回傳給你的格式)：

json

 複製程式碼

```
{
  "code": 200,
  "msg": "All generated successfully.",
  "data": {
    "callbackType": "complete",
    "task_id": "test-task-id-123",
    "data": [
      {
        "id": "test-music-id-001",
        "source_audio_url": "https://cdn1.suno.ai/test-audio-001.mp3"
      }
    ]
  }
}
```

4. 上方 Headers 確認有：

- Content-Type: application/json
(如果 Body 已選 JSON，Postman 通常會自動幫你加)

5. 按 **Send**

預期結果

- Postman 會收到 Flask 的回應，正常的話類似：

json

 複製程式碼

```
{
  "success": true,
  "download_url": "https://你的bucket.s3.amazonaws.com/Audios/xxxxx.mp3",
  "reply": "Flask 收到音樂prompt"
}
```

- 這時候你再去：
 - 🔍 **CloudWatch Logs** 找對應 Lambda，看有沒有印：
=== 收到 Suno callback ===
 - 🔍 看 S3 bucket 裡面 **Audios/** 資料夾有沒有多出 mp3 檔
 - 🔍 看 DynamoDB **UserRecords** 有沒有多一筆 **type = audio** 的紀錄

只要這三個都有 ☒，就代表：

你的 AWS 端完全沒問題，剩下就是 Suno 那邊 callback 有沒有打過來的問題。

方法二：用 PowerShell / cmd 打 curl (如果你喜歡指令)

在 PowerShell

powershell

複製程式碼

```
curl -Method POST `
  -Uri "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/audio-callback"
  -Headers @{ "Content-Type" = "application/json" } `
  -Body '{
    "code": 200,
    "msg": "All generated successfully.",
    "data": {
      "callbackType": "complete",
      "task_id": "test-task-id-123",
      "data": [
        {
          "id": "test-music-id-001",
          "source_audio_url": "https://cdn1.suno.ai/test-audio-001.mp3"
        }
      ]
    }
  }'
```

在 WSL / Git Bash / Linux

bash

複製程式碼

```
curl -X POST \
  "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/audio-callback" \
  -H "Content-Type: application/json" \
```

```
-d '{
  "code": 200,
  "msg": "All generated successfully.",
  "data": {
    "callbackType": "complete",
    "task_id": "test-task-id-123",
    "data": [
      {
        "id": "test-music-id-001",
        "source_audio_url": "https://cdn1.suno.ai/test-audio-001.mp3"
      }
    ]
  }
}'
```

效果跟 Postman 是一樣的，只是用指令送。

方法三：用一個小 Python 檔測試

在你本機隨便開一個 `test_callback.py`：

python

 複製程式碼

```
import requests
import json

url = "https://xpapysqd2i.execute-api.us-east-1.amazonaws.com/prod/audio-callback"

payload = {
  "code": 200,
  "msg": "All generated successfully.",
  "data": {
    "callbackType": "complete",
    "task_id": "test-task-id-123",
    "data": [
      {
        "id": "test-music-id-001",
        "source_audio_url": "https://cdn1.suno.ai/test-audio-001.mp3"
      }
    ]
  }
}
```

```
resp = requests.post(url, json=payload)
print("status:", resp.status_code)
print("body:", resp.text)
```

然後執行：

```
bash
```

 複製程式碼

```
python test_callback.py
```

再去看 CloudWatch / S3 / DynamoDB 。

接下來你可以做的事

1. 先用 **Postman / curl / Python 三選一**，
只要有一個方式能讓 `/audio-callback` 成功跑完，
就代表你 AWS 這一邊沒問題了。
2. 確認沒問題後，再回頭到 Suno 那邊：
 - 檢查 `callbackUrl` 參數名稱是否完全符合官方文件（大小寫也要一樣）
 - 如果有查詢 task 狀態的 API，用 `task_id` 看有沒有 callback 的錯誤 / log

你先用 Postman 打一次，結果丟給我（回傳的 JSON + 有沒有看到 S3 / DynamoDB 新資料）。

我再幫你判斷下一步要調哪裡。

     ...

```
[ERROR] KeyError: 'httpMethod'
Traceback (most recent call last):
  File "/var/task/lambda_function.py", line 6, in
lambda_handler
    return awsgi.response(app, event, context)
  File "/var/task/awsgi/__init__.py", line 172, in response
    output = app(environ(event, context), sr)
  File "/var/task/awsgi/__init__.py", line 121, in environ
    'REQUEST_METHOD': event['httpMethod'],
```

已思考 1m 36s >

這個錯誤其實是「測試事件(event)格式不對」，不是 `/audio-callback` 程式本身壞掉。

`KeyError: 'httpMethod'`

awscli 在把 Lambda event 轉成 WSGI 環境的時候，會去讀 `event['httpMethod']`，但你現在丟進去的 event 根本沒有這個欄位（你直接把 S3 的 JSON 當成整個 event 了）。

也就是說：

- 正式情況：API Gateway 會幫你包成「API Gateway Proxy event」，裡面有 `httpMethod`，`path`，`body` 等欄位 → `awscli` OK
- 你現在在 Lambda console 測試：直接丟

json

複製程式碼

```
{"code":200,"data":{...}}
```

→ 沒有 `httpMethod` → `awscli` 崩潰

正確做法：用「API Gateway AWS Proxy」格式來測 `/audio-callback`

1 在 Lambda console 建 test event 的方式

1. 進 Lambda function 頁面（就是你掛在 API Gateway 上那隻）
2. 點上面的 **Test** 按鈕右邊的小三角形 → **Configure test event**
3. 在 **Event template** 下拉選單選：
 - `API Gateway AWS Proxy`（或類似字樣）
4. 給它一個名字：`test_callback`
5. 把內容改成像這樣（重點是 `httpMethod`，`path`，`body` 都要在）：

json

複製程式碼

```
{
  "resource": "/audio-callback",
  "path": "/audio-callback",
  "httpMethod": "POST",
  "headers": {
    "Content-Type": "application/json"
  },
  "multiValueHeaders": {
    "Content-Type": [
```

```

    "application/json"
  ],
  "queryStringParameters": null,
  "multiValueQueryStringParameters": null,
  "pathParameters": null,
  "stageVariables": null,
  "requestContext": {
    "path": "/prod/audio-callback",
    "stage": "prod",
    "httpMethod": "POST",
    "requestId": "test-request-id",
    "requestTimeEpoch": 0,
    "identity": {}
  },
  "body": "{\\"code\\":200,\\"msg\\":\\"All generated successfully.\\",\\"data\\":{\\"call\\":\\"\\",\\"isBase64Encoded\\":false}"
}
```

注意：`body` 一定要是字串，裡面的雙引號要加 `\` 轉義，這樣 Flask 的 `request.json` 才會拿到你模擬的 Suno JSON。

6. 按 **Save** → 再按上面的 **Test**

2 預期結果

成功時：

- Lambda 回傳類似：

json 複製程式碼

```

{
  "statusCode": "200",
  "headers": { ... },
  "body": "{\\"success\\": true, \\"download_url\\": \\"https://你的bucket.s3.amazonaws.com/\\", \\"isBase64Encoded\\": false}"
}
```

- CloudWatch Logs 裡應該會看到：

text 複製程式碼

```
=== 收到 Suno callback ===
```

- S3 bucket 裡 `Audios/` 多出一個 mp3 檔
- DynamoDB `UserRecords` 多一筆 `type = audio` 的紀錄

3 重點總結

- 你現在的錯誤只是：用「一般 JSON」測 Lambda + awsgi，但 awsgi 只懂 **API Gateway proxy event** 的格式。
- 你 **不需要改任何程式碼**，只要：
 - Lambda console 測試時，用 **API Gateway AWS Proxy** 模板
 - 把 Suno 回傳的 JSON 放到 **body (字串)** 裡

你可以照上面的 event 貼進去跑一次，把新的 log / 回傳結果貼給我，我可以幫你再確認 S3 / DynamoDB 有沒有成功寫進去。

< 2/2 >      ...